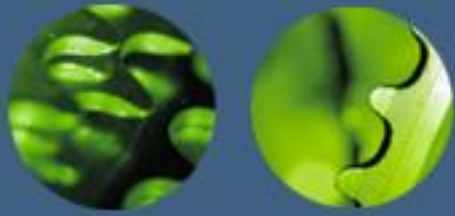




UML

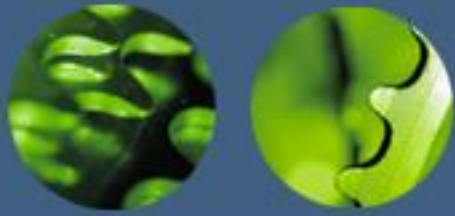
Unified Modeling
Language
(Lenguaje de
Modelamiento unificado)

Presentado por: Lic. Ricardo Gastón Plazas
Especialista en Innovación Institucional



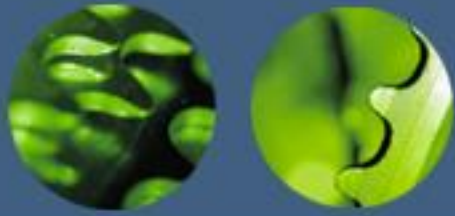
Contenido

- Introducción.
- Introducción a UML.
- Programación Orientación a Objetos (OOP).
 - Objetos y Clases.
 - Los Pilares.
 - Concepción de Clases.
 - Paquetes.
 - Las Relaciones.
 - Asociaciones, Herencia y Generalización, Dependencia, Agregación y Composición.



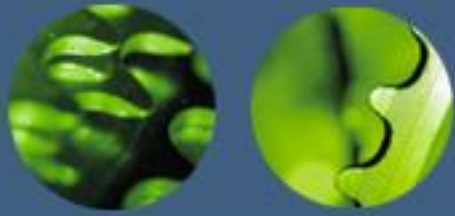
Contenido

- Diagrama de Contexto.
- Otras Características de las Clases.
- Notas.
- **Introducción a los Casos de Usos.**
- **Fase de Captura de Requerimientos y Análisis**
 - Diagramas de Casos de Usos.
 - Diagramas de Actividades.



Contenido

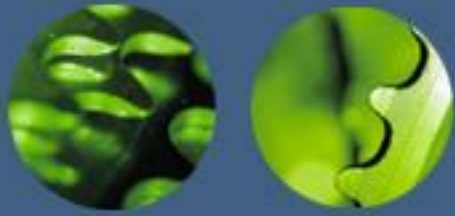
- **Fase de Diseño**
 - Diagramas de Clases y Objetos.
 - Diagramas de Secuencias.
 - Diagramas de Colaboraciones.
 - Diagramas de Estados.
 - Diagramas de Componentes.
 - Diagramas de Despliegue o Distribución.
- **Conclusiones.**



¿Qué es un Modelo?

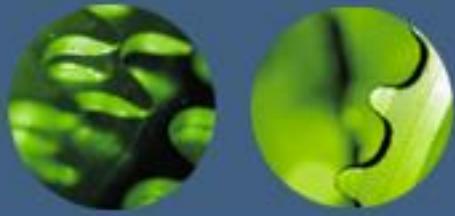


**Un Modelo es
una Simplificación de la
Realidad**



Conceptos Importantes

- **Modelo:** captura una vista de un sistema del mundo real. Es una abstracción de dicho sistema, considerando un cierto propósito. Así, el modelo describe completamente aquellos aspectos del sistema que son relevantes al propósito del modelo, y a un apropiado nivel de detalle.
- **Diagrama:** una representación gráfica de una colección de elementos de modelado, a menudo dibujada como un grafo con vértices conectados por arcos.
- **Metodología:** Conjunto de procedimientos, técnicas, herramientas y un soporte documental que ayuda a los desarrolladores a realizar nuevo software



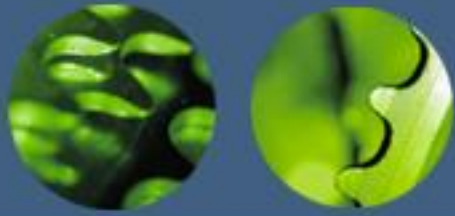
Conceptos Importantes

Modelos y Diagramas

- Un proceso de desarrollo de software debe ofrecer un conjunto de modelos que permitan expresar el producto desde cada una de las perspectivas de interés.
- El código fuente del sistema es el modelo más detallado del sistema (y además es ejecutable). Sin embargo, se requieren otros modelos ...



- Cada modelo es completo desde su punto de vista del sistema, sin embargo, existen relaciones de trazabilidad entre los diferentes modelos.

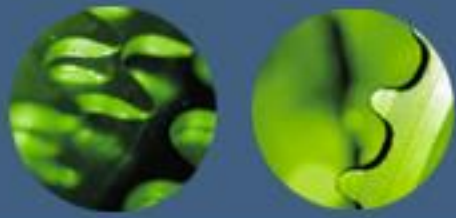


Conceptos Importantes

Metodología Vs Ciclo de Vida

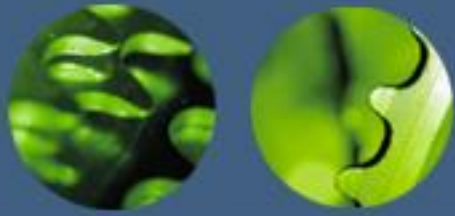
Una metodología puede seguir uno o varios modelos de ciclo de vida, es decir, el ciclo de vida indica qué es lo que hay que obtener a lo largo del desarrollo del proyecto pero no cómo hacerlo.

La metodología indica cómo hay que obtener los distintos productos parciales y finales.



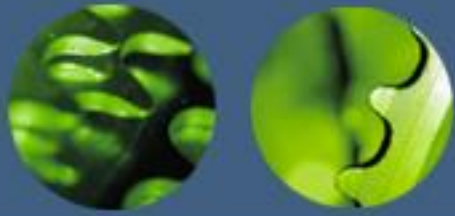
Paradigmas de Programación

- Hay para todos los gustos
 - Estructurados (C, Pascal, Basic, etc.)
 - Funcionales (CAML)
 - Declarativos (Prolog)
 - Orientados a Objetos (C#, VB.NET, Smalltalk, Java, Visual FoxPro)
 - Orientados a Aspectos
 - Híbridos (Lisp, Visual Basic)
 - Incomprensibles....
- Cada enfoque tiene sus ventajas y desventajas
- Cada uno es más apropiado para ciertas cosas



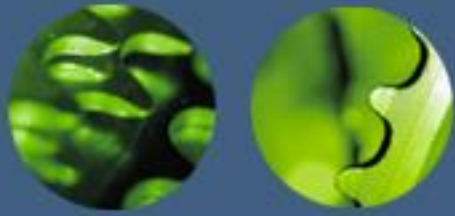
Historia del Software

- Primeros años (1950-1960)
 - Orientación a Batch (Lotes), Distribución limitada, Software a la medida.
- Segunda Era (1975 - 1986)
 - Multiusuario, Tiempo real, Base de Datos (Jerarquias, Redes), Paquetes de Software.
- Tercera Era (1975 - 1986)
 - Sistemas Distribuidos, Incorporación de Inteligencia.
 - Bajo costo de Hardware, Software de uso final, aparición del Micro.
- Cuarta Era (1986 - 1993)
 - Base de Datos Relacionales.
 - Informix, PL-SQL, Dbase, FoxPro.
 - Gran consumo de Paquetes de Software.



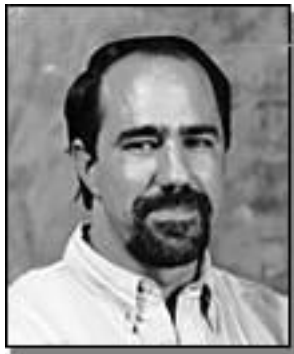
Historia del Software

- 5ª Generación (1994-1996)
 - Cliente/Servidor:
 - Developer, Power-Builder, Forte, Visual Basic, Java, Visual FoxPro.
- 6ª Generación (1997 - 2002)
 - WIS: Web Information System, Open Source.
 - Perl, PHP, Java, DHTML, MySql, PostGress.
- 7ª Generación (2002 - ...)
 - Arquitectura x Componentes:
 - JEEE, .Net, BPMS, SOA.



Introducción a UML

- Lenguaje escrito por:



Grady
Booch



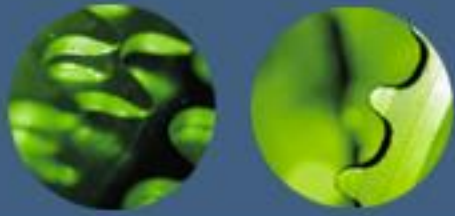
Ivar
Jacobson



James
Rumbaugh

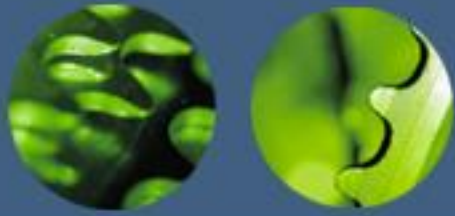
- Basado en las experiencias de los autores.
- Actualmente es un estándar y pertenece a la OMG (Object Management Group)
- Última Versión: 2.0 y la 2.1 es Beta.





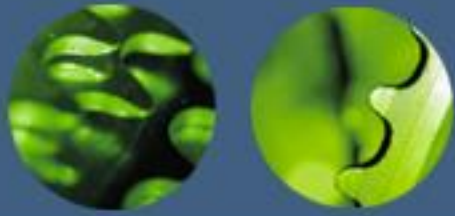
¿Qué es UML?

Es una herramienta o Lenguaje de Modelamiento Unificado que permite a los creadores de Sistemas generar diseños que capturen sus ideas en una forma convencional y fácil de comprender y así poder comunicárselas a otras personas.



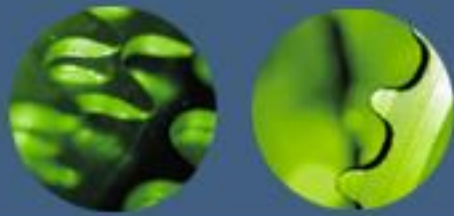
¿Qué es UML?

- UML define una notación que se expresa como diagramas que sirven para representar modelos/subsistemas o partes de ellos
- UML es un lenguaje de propósito general para el modelado orientado a objetos.
- Define una estructura para ir del análisis al diseño y de éste a la implementación.

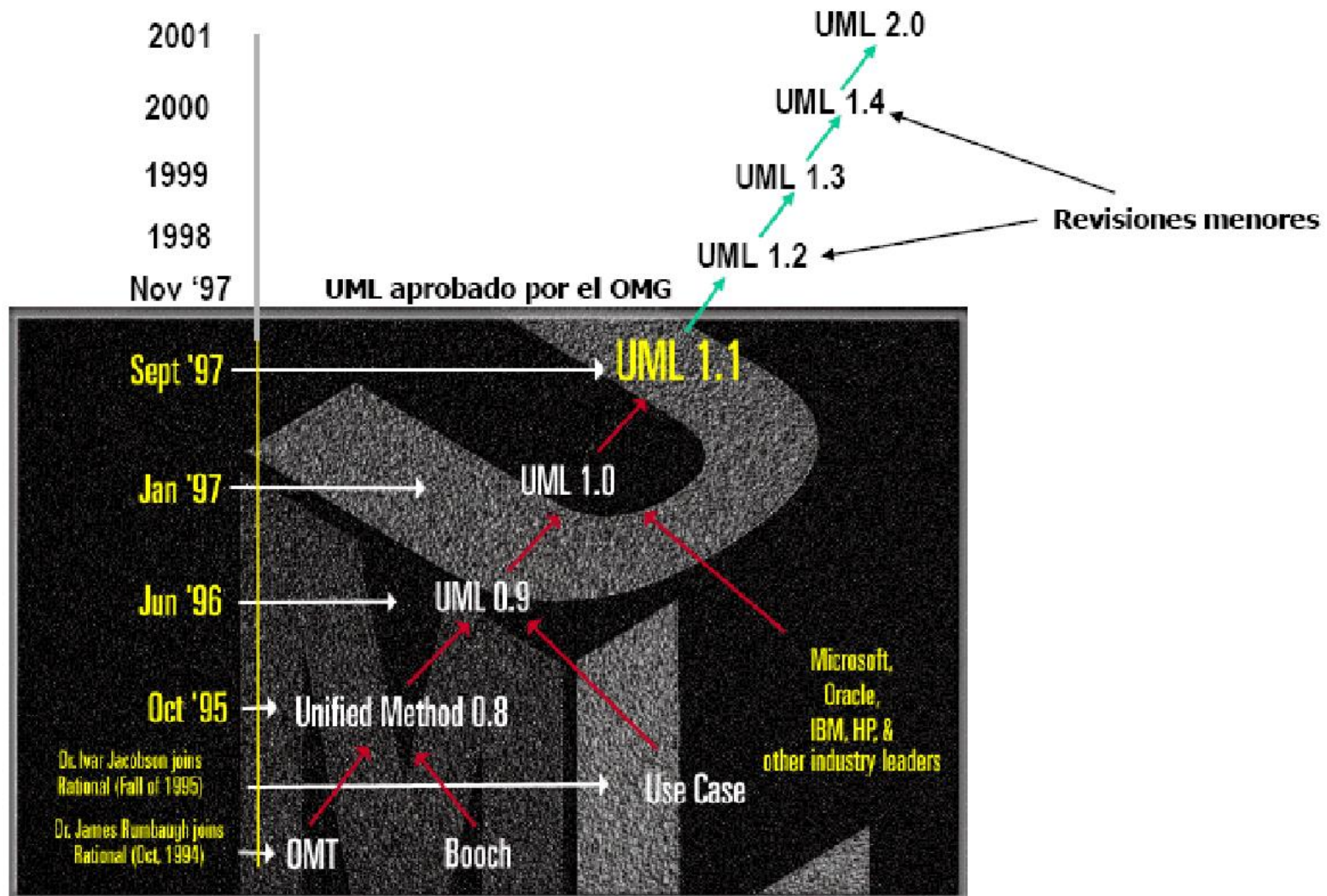


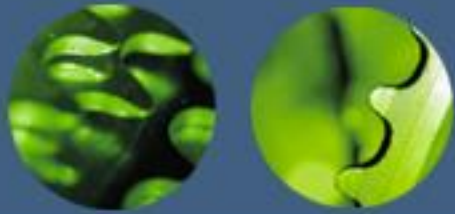
¿Qué es UML?

- “UML es un lenguaje visual para especificar, construir y documentar sistemas” (OMG - Object Management Group)
- Unified (UNIFICADO):
 - El aporte de muchos métodos y notaciones
 - Independiente de implementaciones, plataformas y lenguajes
- Modeling (MODELADO):
 - Los modelos son utilizados en todas las ingenierías
- Language (LENGUAJE):
 - Si hay gente, requieren comunicarse. Si se tienen que comunicar, se tienen que entender. Para entenderse necesitan un lenguaje común
- **¡UML no es Metodología!**



Historia de UML



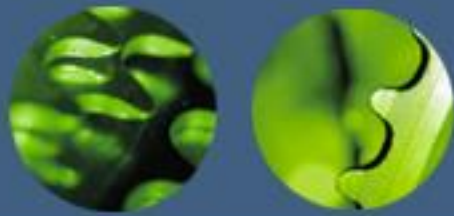


Estructura de UML

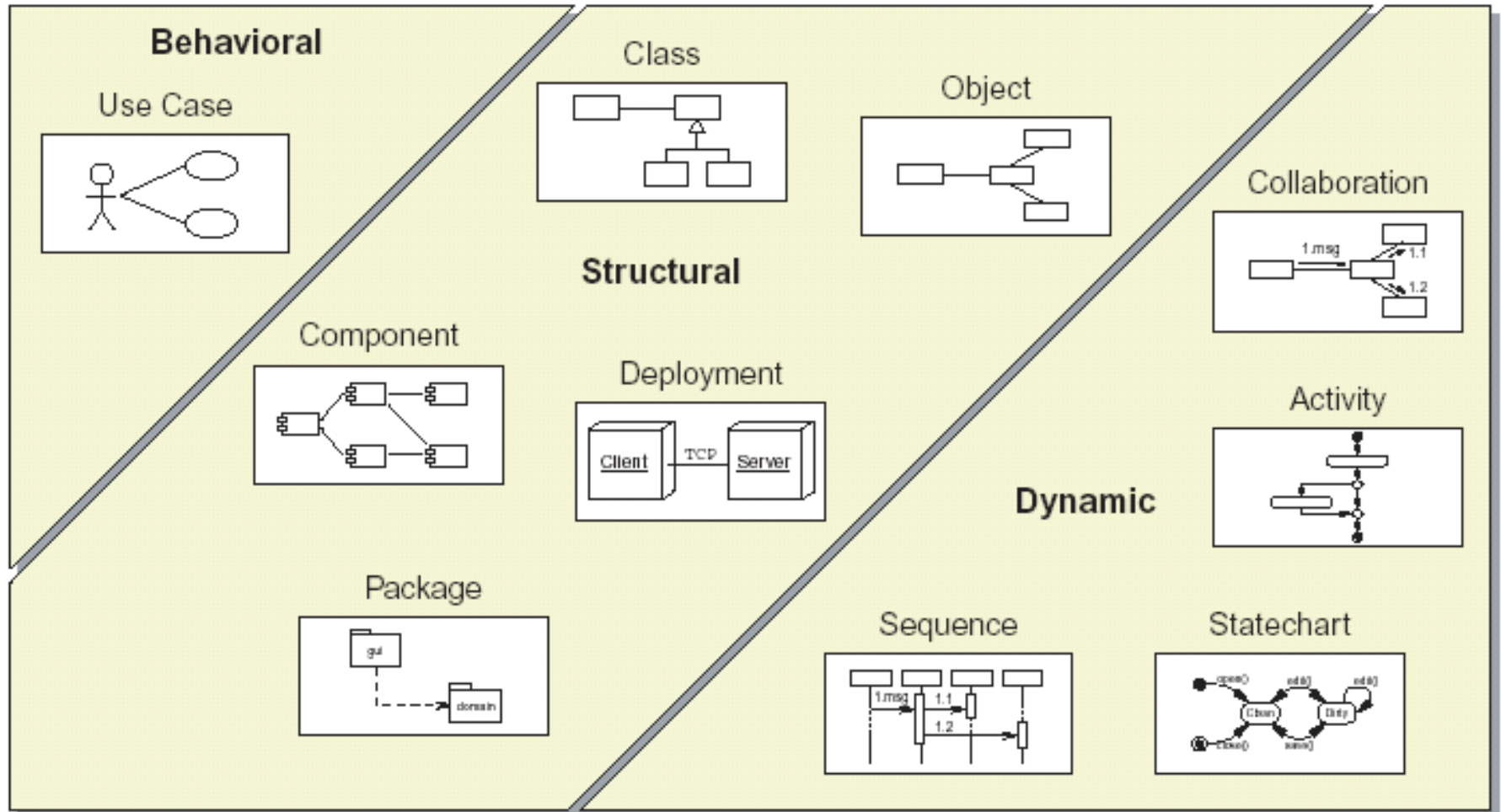
- **Vistas de UML: Arquitectura 4 + 1**

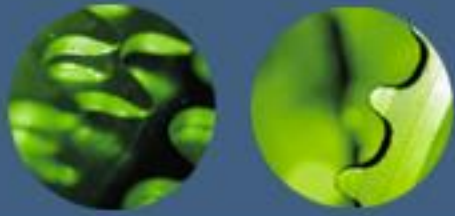
- 5 Vistas
- 9 Diagramas





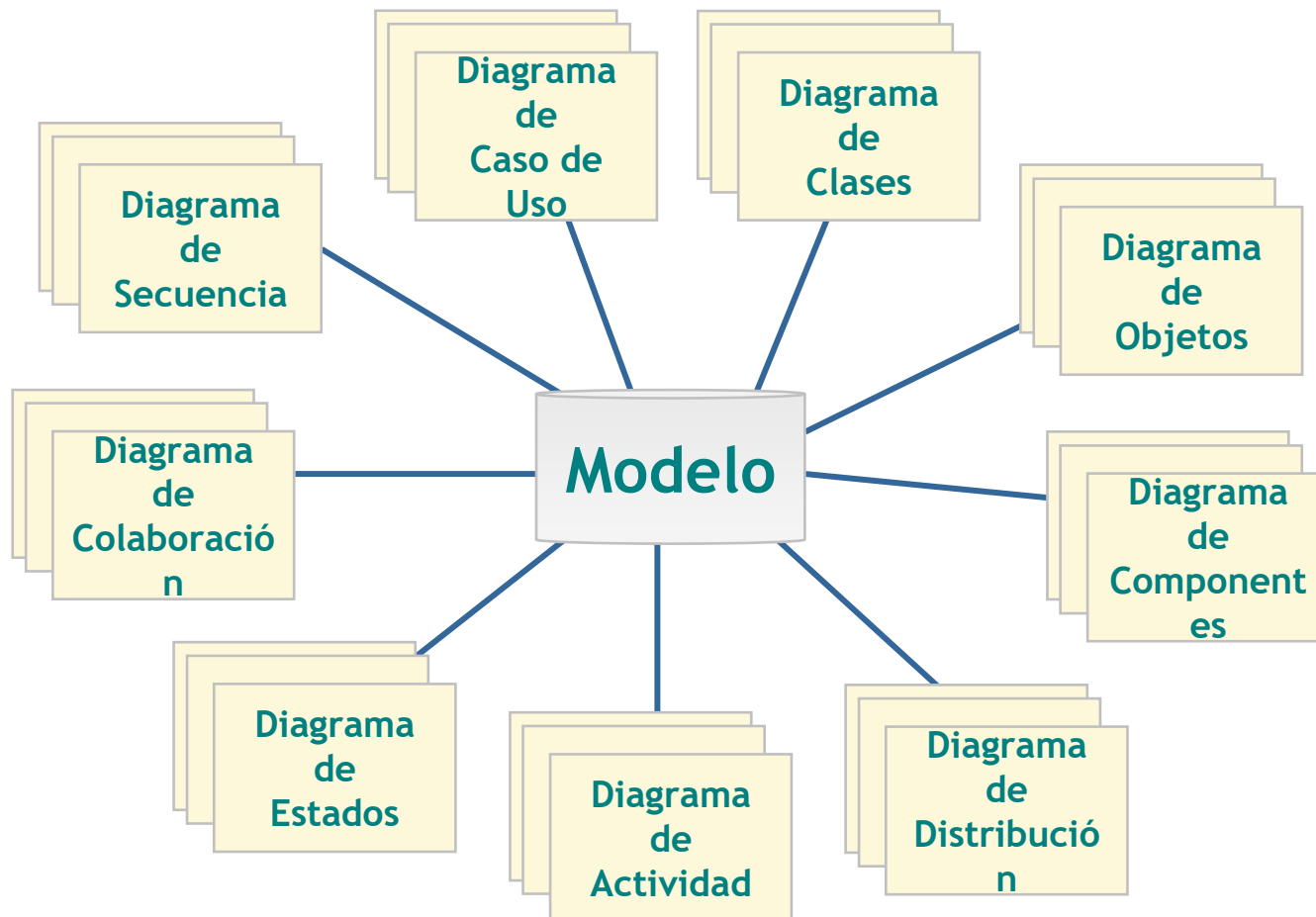
Vista de UML

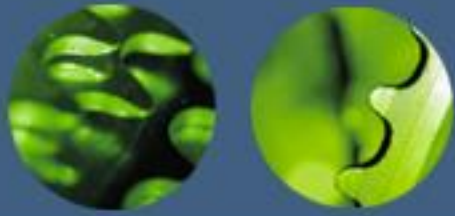




Diagramas de UML

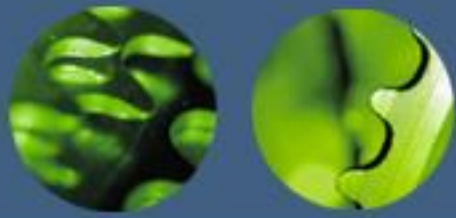
Los diagramas expresan gráficamente partes de un modelo.





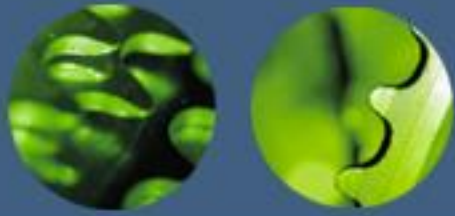
Diagramas de UML

- La finalidad de los Diagramas es presentar diversas perspectivas de un Sistema, a los cuales se le conoce como MODELO.
- El Modelo UML de un Sistema es similar a un Modelo de Escala de un Edificio.
- Es importante destacar que el Modelo UML describe lo que supuestamente hará un Sistema, pero no dice como implementar dicho sistema.



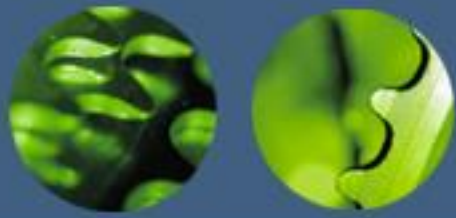
¿Por qué tantos Diagramas?

- Los Diagramas UML permite examinar un Sistema desde distintos puntos de vista.
- En necesario contar con diferentes perspectiva en un Sistema por que se cuenta con diferentes personas implicadas, los cuales tienen enfoque particulares en diferentes aspectos del Sistema.
- El Objetivo es satisfacer a cada Persona involucrada.
- Cabe recalcar que en UML no es necesario que aparezcan todos los Diagramas.



Orientación a Objetos

- La Programación Orientada a Objeto fomenta una metodología basada en Componentes en la Ingeniería de Software.
- El Sistema se genera mediante un conjunto de Objetos, después se amplía agregándole funcionalidad y finalmente reutilización de los Objetos en los nuevos Sistemas, reduciendo el tiempo en Desarrollo.



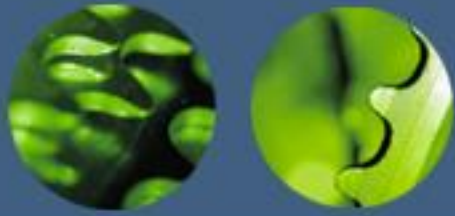
Orientación a Objetos

- La Programación Estructurada tradicional se basa en la Ecuación de Wirth:

Algoritmos + Estructuras de Datos = Programas

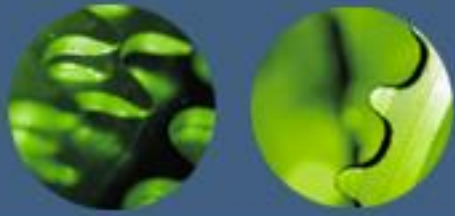
Esto significa que los Datos y el Código se trata por separado.

- La OOP es una técnica de programación cuyo soporte es el Objeto.
- Objeto: es una extensión de un *Tipo Abstracto de Datos (TAD)*.
- El TAD es un tipo definido por el Usuario, que encapsula un conjunto de datos y las operaciones sobre estos datos.



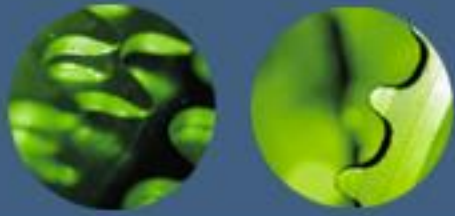
Orientación a Objetos

- Un Objeto es una cosa, es una Instancia de una Clase. Todos nosotros somos instancia de una Clase llamada Persona. Informalmente, un objeto representa una entidad del mundo real
- Un objeto posee (Booch): Estado, Comportamiento e Identidad.
- Un Objeto cuenta con una Estructura: Atributos (Propiedades) y Métodos (Acciones).
- Atributos es una característica concreta de una clase.
- Los Métodos o Acciones son todas las Actividades que el Objeto es capaz de realizar.
- El Conjunto de Atributos y Métodos se conocen como Características o Rasgos.



Orientación a Objetos

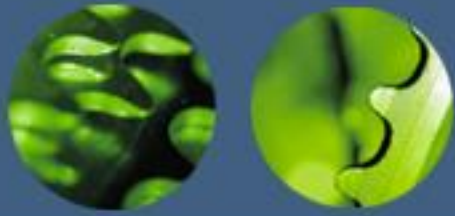
- ¿Por qué Orientación a Objetos (OO)?
 - Se parece más al mundo real.
 - Permite representar modelos complejos.
 - Muy apropiada para aplicaciones de negocios.
 - Las empresas ahora sí aceptan la OO.
 - Las nuevas plataformas de desarrollo la han adoptado (Java / .NET).



Un objeto posee Estado

Lo que el objeto sabe

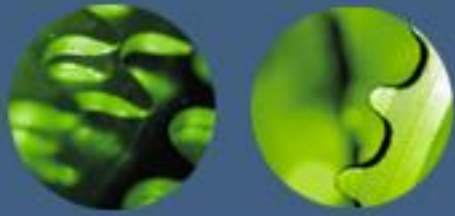
- El estado de un objeto es una de las posibles condiciones en que el objeto puede existir
- El estado normalmente cambia en el transcurso del tiempo
- El estado de un objeto es implementado por un conjunto de propiedades (atributos), además de las conexiones que puede tener con otros objetos



Un objeto posee Comportamiento

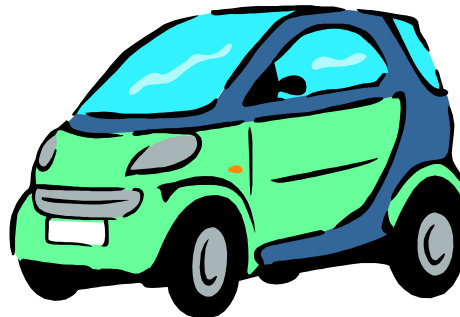
Lo que el objeto puede hacer

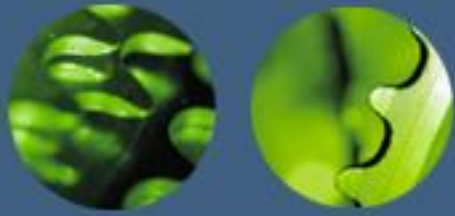
- El comportamiento de un objeto determina cómo éste actúa y reacciona frente a las peticiones de otros objetos
- Es modelado por un conjunto de mensajes a los que el objeto puede responder (operaciones que puede realizar)
- Se implementa mediante métodos



Un objeto posee Identidad

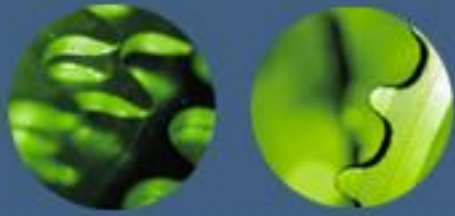
- Cada objeto tiene una identidad única, incluso si su estado es idéntico al de otro objeto





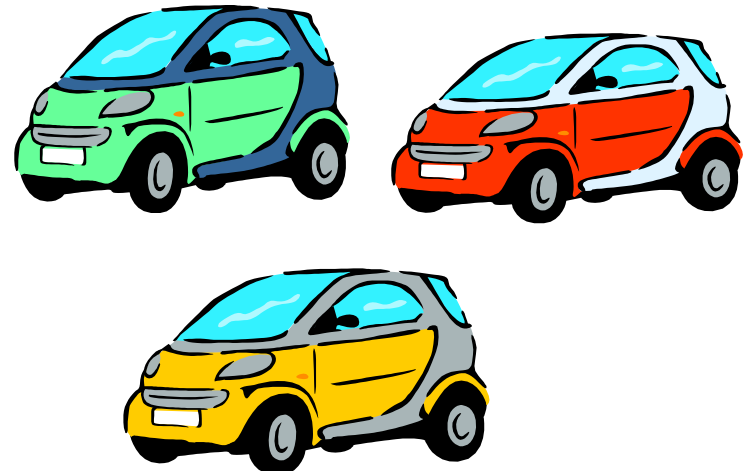
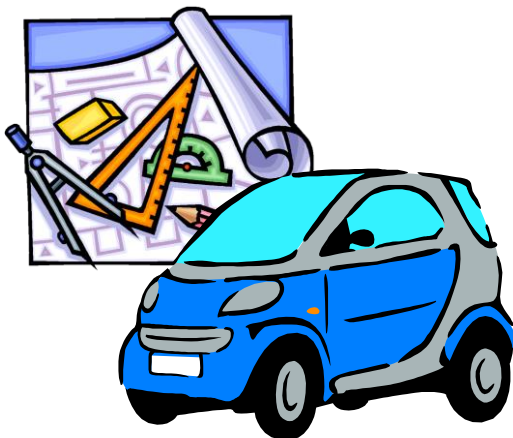
Orientación a Objetos

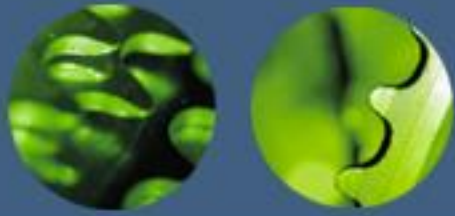
- La Clase es una descripción de un conjunto de objetos similares. Es una plantilla para fabricar Objetos.
- La OOP no es solo Objetos, Clases, Atributos y Métodos, son: Abstracción, Herencia, Polimorfismo y Encapsulamiento o Encapsulación.
- Otro Aspecto importante de la OOP son: Envío de Mensajes, las asociaciones y agregaciones.



Objetos y Clases

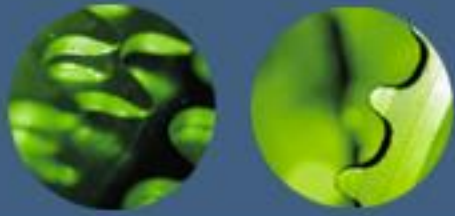
- Una clase es una definición abstracta de un objeto
 - Define la estructura y el comportamiento compartidos por los objetos
 - Sirve como modelo para la creación de objetos
- Los objetos pueden ser agrupados en clases





Ejemplo de una Clase

- Clase: Curso
- Estado (Atributos)
 - Nombre
 - Ubicación
 - Días Ofrecidos
 - Horario de Inicio
 - Horario de Término
- Comportamiento (Métodos)
 - Agregar un Alumno
 - Borrar un Alumno
 - Entregar un Listado del Curso
 - Determinar si está Completo



Pilares de la Orientación a Objetos

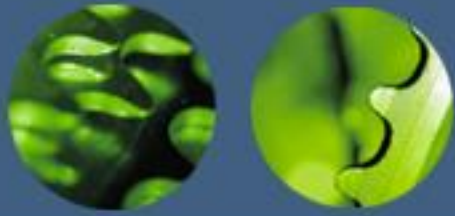
Abstracción

Herencia

Polimorfismo

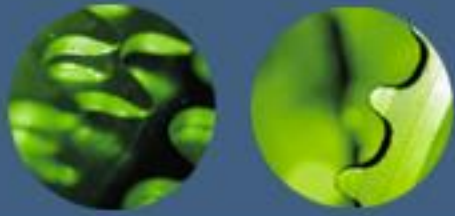
Encapsulamiento

Relaciones



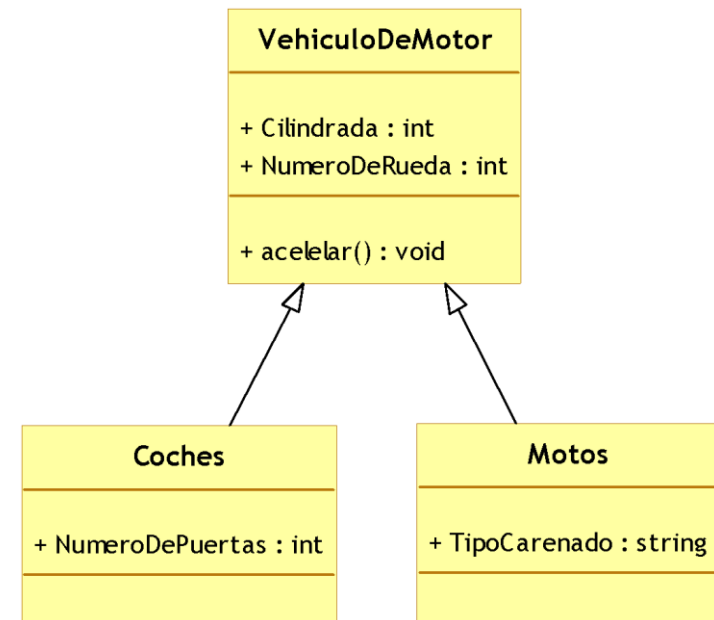
Abstracción

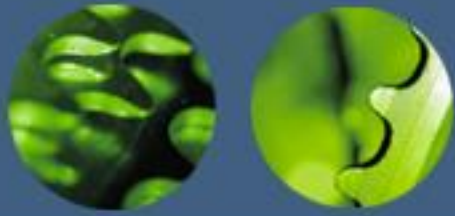
- Es quitar las Propiedades y Acciones de un Objeto y dejar solo las necesarias.
- Ignorancia Selectiva
 - La abstracción nos ayuda a trabajar con cosas complejas
 - Se enfoca en lo importante
 - Ignora lo que no es importante (simplifica)
- Una clase es una abstracción en la que:
 - Se enfatizan las características relevantes
 - Se suprimen otras características
- Una clase debe capturar una y solo una abstracción clave



Herencia

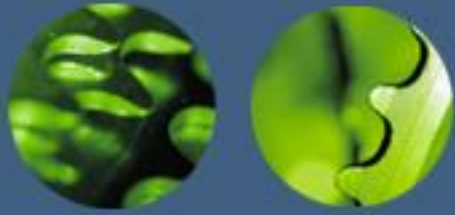
- Es la Calidad mas importante de la OOP.
- Es un mecanismo mediante el cual se puede crear una nueva clase partiendo de una existente, se dice que la nueva clase hereda las características de la clase existente, aunque se le puede añadir mas capacidades o modificar las que tiene.





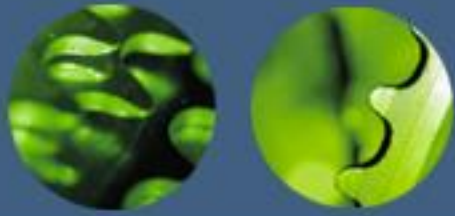
Polimorfismo

- En ocasiones una acción tiene el mismo nombre en diferentes Clases o en la misma, pero realizara una operación diferente.
- En la OOP cada Clase “SABE” como realizar cada operación.
- Es la posibilidad de que dos Métodos implementen distintas acciones, aun teniendo el mismo nombre, dependiendo del Objeto que lo ejecuta o de los parámetros que recibe.



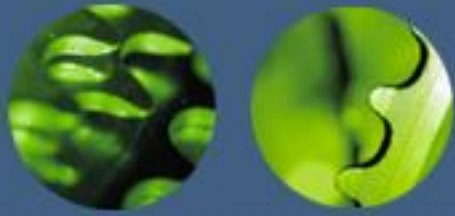
Polimorfismo

- La Sobrecarga es un tipo especial del Polimorfismo.
- Varios Métodos con el mismo nombre, siempre y cuando que el tipo de parámetros que recibe o el numero sean diferentes.



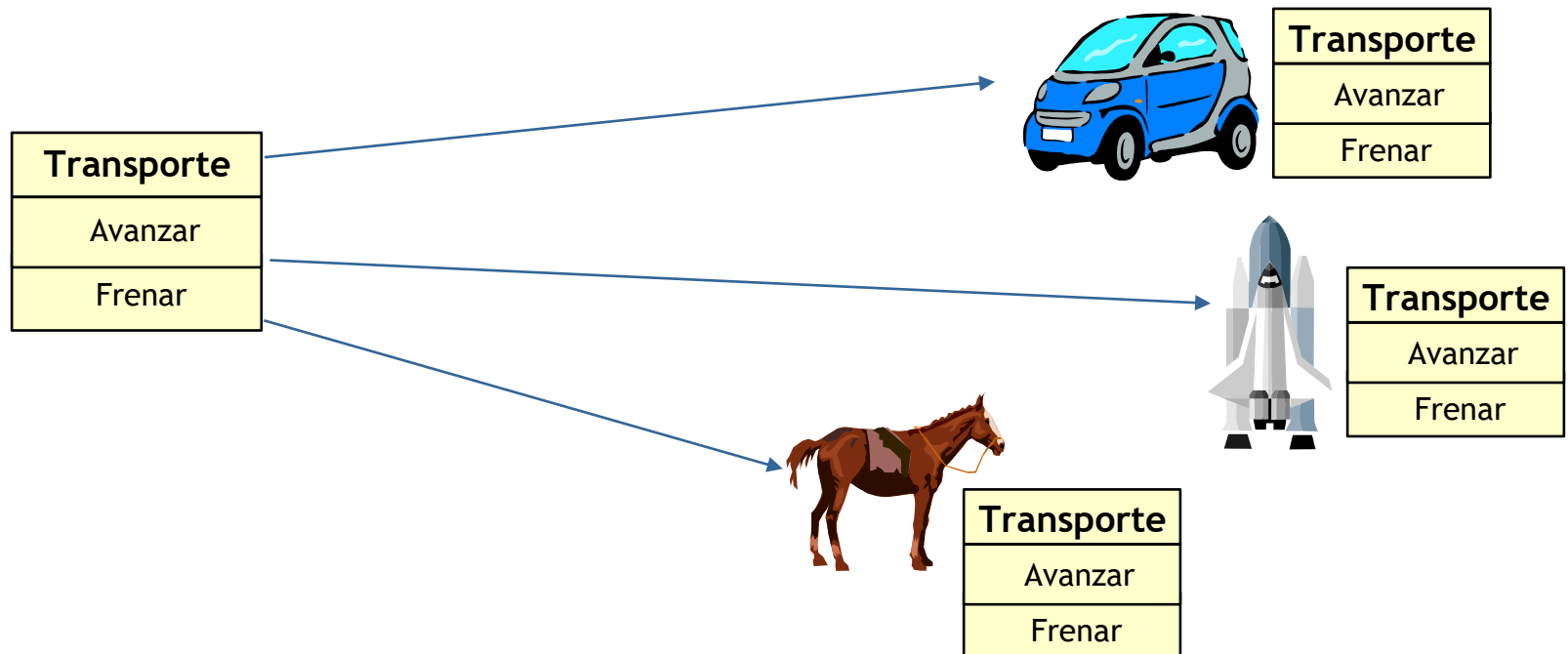
Polimorfismo

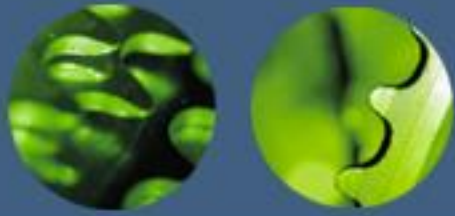
- Es la propiedad que tienen los objetos de permitir invocar genéricamente un comportamiento (método) cuya implementación será delegada al objeto correspondiente recién en tiempo de ejecución
- El polimorfismo tiende a existir en las relaciones de herencia, pero no siempre es así



Polimorfismo - Ejemplo

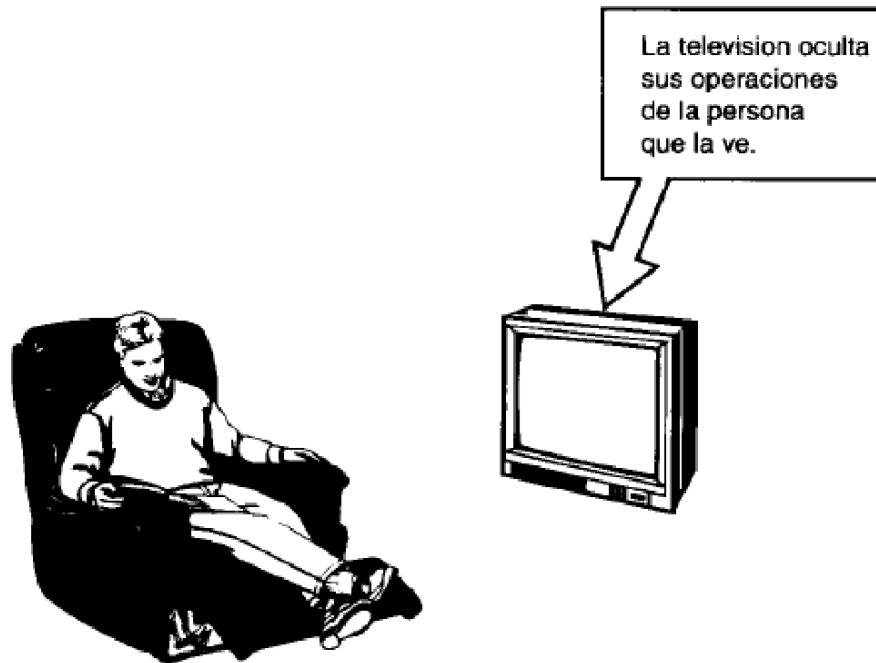
- La definición del método reside en la clase base
- La implementación del método reside en la clase derivada
- La invocación es resuelta al momento de ejecución

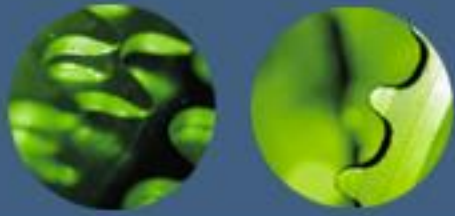




Encapsulamiento

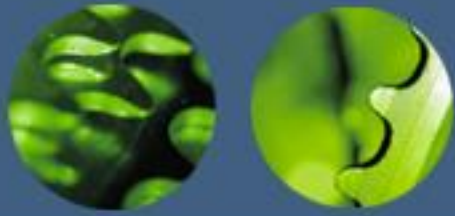
- Es el ocultamiento de la Funcionalidad interna de sus operaciones, de otros objetos y del mundo exterior.





Encapsulamiento

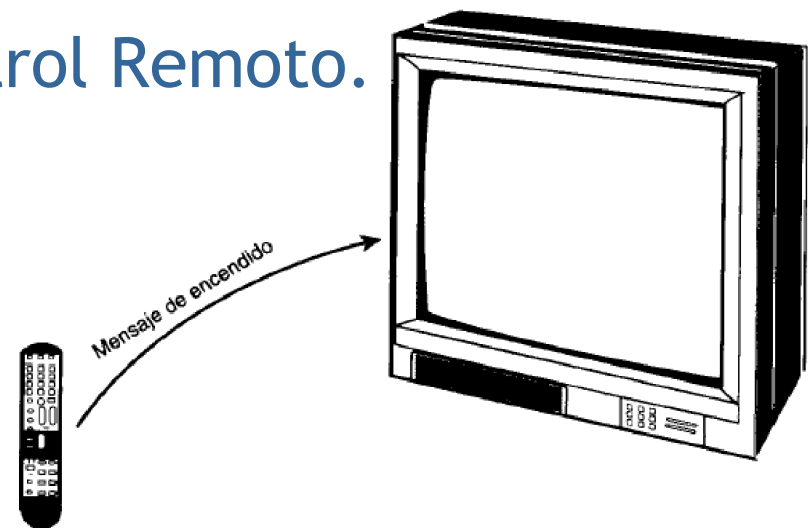
- Principio que establece que los atributos propios de un objeto no deben ser visibles desde otros objetos
 - Deben ser declarados como privados
- Permite abstraer al resto del mundo de la complejidad de la implementación interna
- Permite exponer el estado del objeto sólo a través del comportamiento que le hayamos definido mediante miembros públicos
- ¿Por qué es útil?
 - Punto de Control/Validación
 - Mejor respuesta ante los Cambios

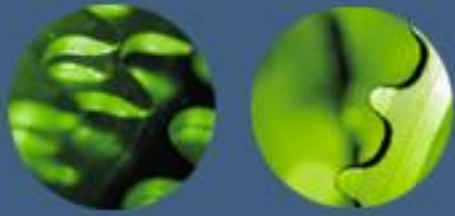


Envió de Mensajes

- Los Objetos en un Sistema trabajan en conjunto, esto se logra por intermedio de mensajes entre ellos.
- Un Objeto envía a otro un mensaje para realizar una operación y el Objeto receptor recibe dicho mensaje para su ejecución.

Ejemplo: El Televisor y su Control Remoto.





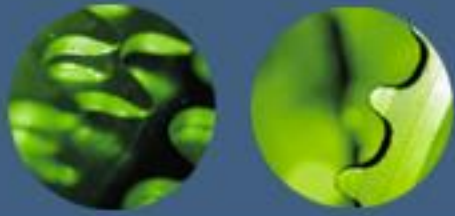
Concepción de Clases

- La Clase se representa con un Rectángulo.
- Existen diferentes tipo de Clases:
 - **Abstracta:** Es de apoyo y solo se construye solo para derivar de ellas otras Clases, pero no se puede hacer ninguna instancia. También se le llama Clase Virtual.
 - **Base:** Es la que se halla al inicio del Árbol de las Jerarquías de Clases. La raíz de ese árbol es la clase base o superclase.

<Nombre Clase>

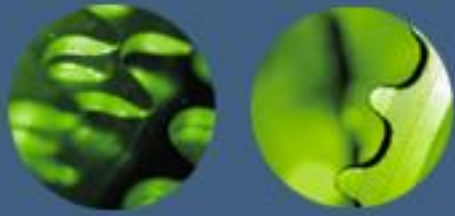
<Atributos>

<Operaciones
o Métodos>



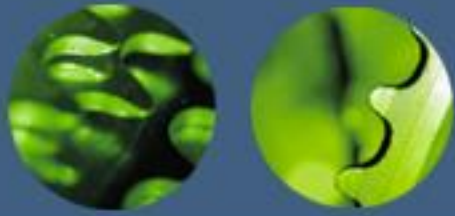
Concepción de Clases

- **Contenedora o Compuesta:** Al hecho de crear nuevas clases utilizando otras clases como componentes, se le llama composición, y a la clase compuesta se le llama contenedora.
- **Derivada:** cuando hemos aplicado la herencia de una sobre otra. La clase B deriva de la clase A cuando B hereda los datos y métodos de A.
- **Hija:** Clase que es derivada directamente de otra. Decimos que la clase B es hija de la clase A si B deriva directamente de A (está conectada directamente en el árbol de jerarquías de las clases).



Concepción de Clases

- **Padre:** La clase de la cual otra deriva directamente. Decimos que la clase A es padre de la clase B si B deriva directamente de A (está conectada directamente en el árbol de jerarquías de las clases).
- **SuperClase:** Cualquier clase de la que derivan una o más clases. Normalmente, a la clase que se halla directamente por encima de otra determinada, la llamamos clase padre, y aquella de la que derivan todas -la que se halla a la raíz del árbol de jerarquías- la llamamos *Superclase* o *Clase Base*.



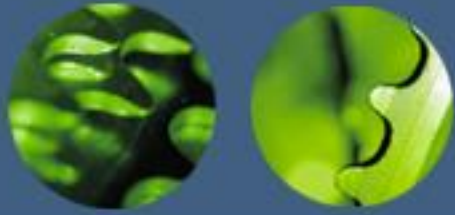
Concepción de Clases

- **SubClase:** Cualquier clase que es derivada de otra (u otras si el sistema permite herencia múltiple) es una subclase. También llamada Clase Hija o Clase derivada.

Ejemplo de una Clase:

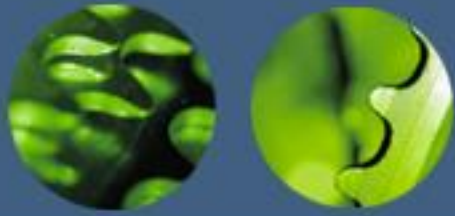
VehiculosDeMotor
◆ Cilindrada ◆ Numero de Ruedas
◆ acelerar()

VehiculoDeMotor
+ Cilindrada : int + NumeroDeRueda : int
+ acelerar() : void



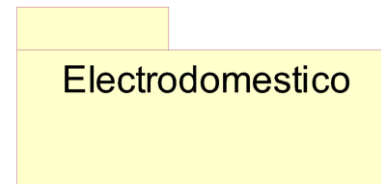
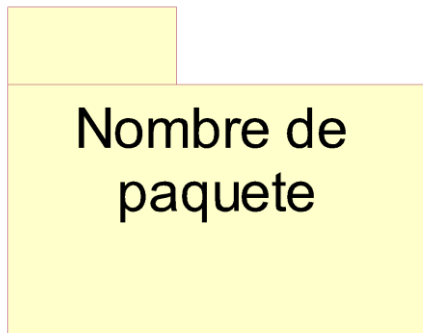
Concepción de Clases

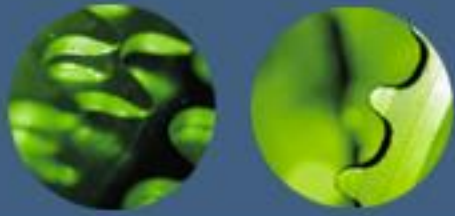
- Las Clases son el Vocabulario terminología del área del Conocimiento.
- Las Clases son los Sustantivos del Requerimiento y los Verbos son las Operaciones o Métodos que conforman la Clase.



Paquetes

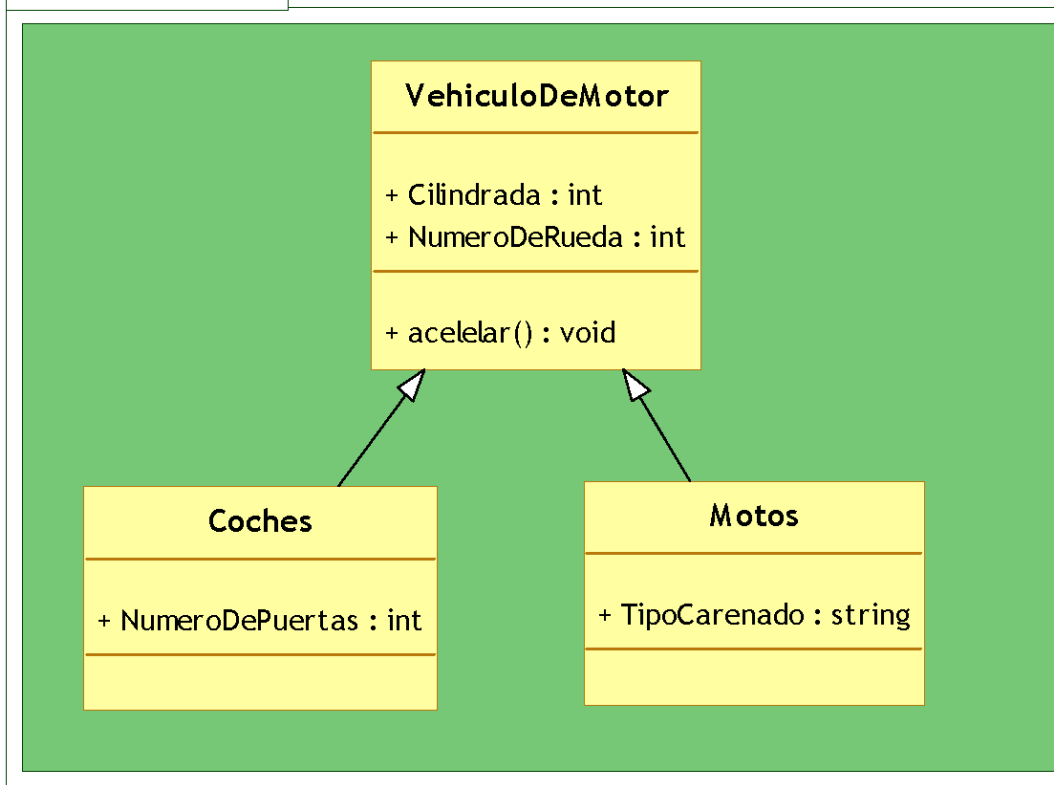
- **Paquetes:** Es la manera en que UML organiza un diagrama de elementos. También sirve para la organización de un Modelo de Sistema/SubSistemas agrupando elementos del Modelo.
- Los modelos contienen múltiples clases y pueden estar agrupadas en paquetes

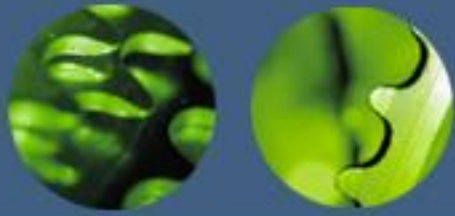




Paquetes

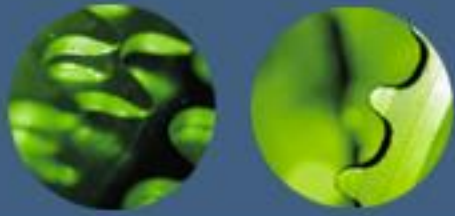
Paquete Vehiculo





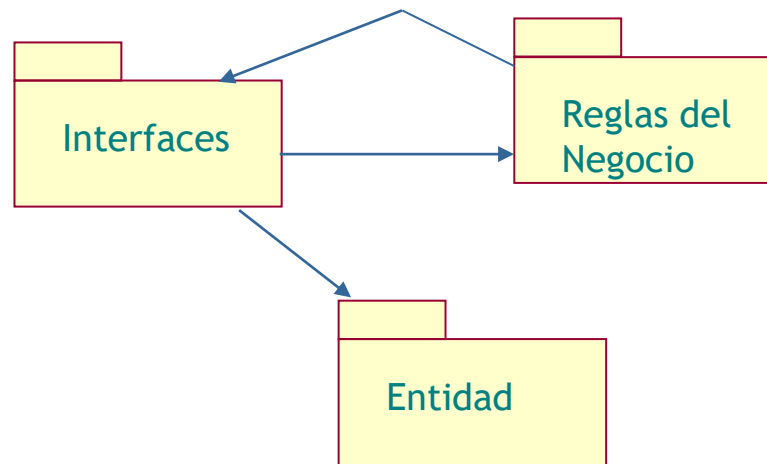
Paquetes

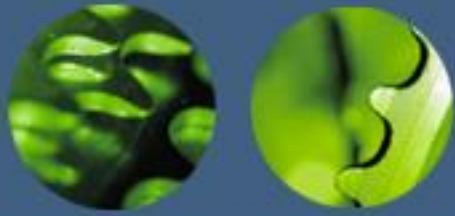
- En las primeras fases del desarrollo del sistema es posible utilizar los paquetes para los siguientes objetivos:
 - Tener una vista del sistema sin mucho detalle.
 - Tener vistas de pequeñas porciones del sistema.
 - Crear pequeñas porciones del sistema que pueden trabajar independientemente.
 - Existe una dependencia entre paquetes cuando por lo menos una clase de un paquete depende de una clase dentro de un segundo paquete.



Paquetes

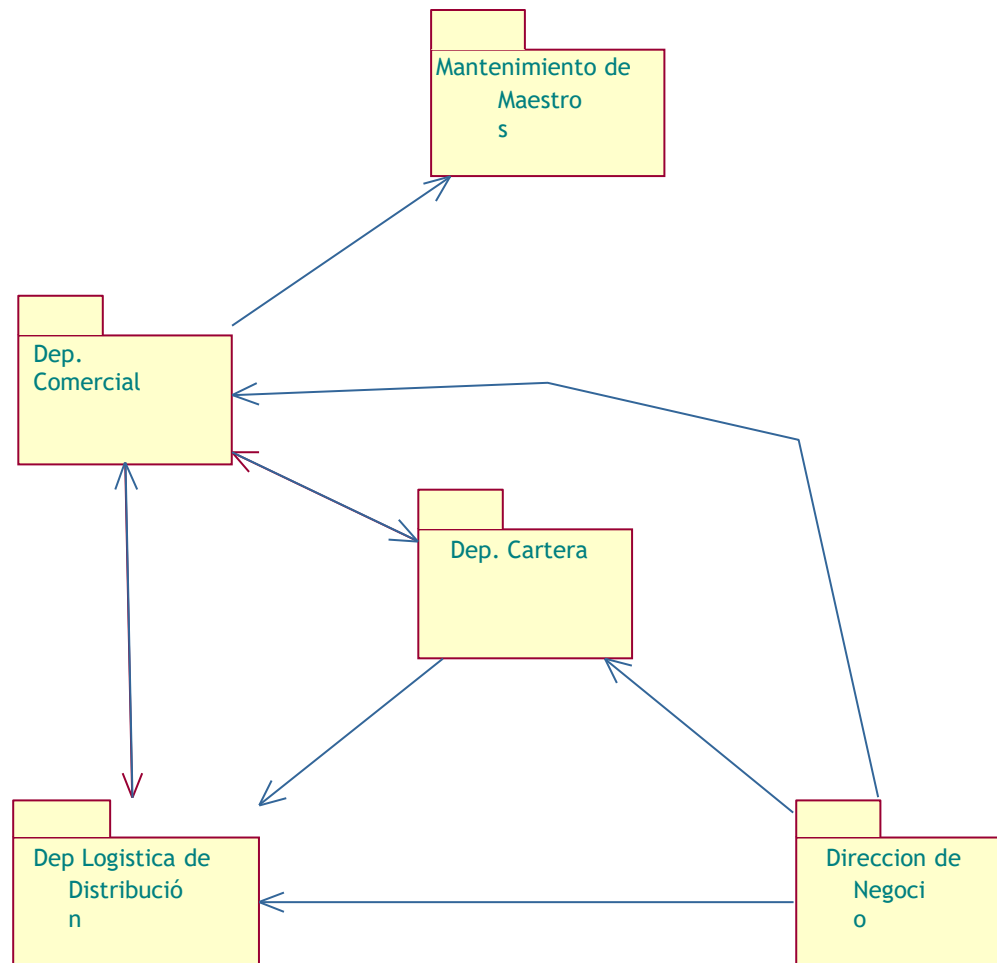
Cuando existe dependencia cíclica entre paquetes, es recomendable dividir los paquetes por funcionalidad, para romper estas dependencias cíclicas.

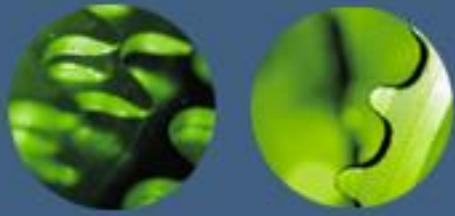




Paquetes

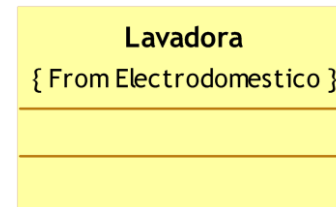
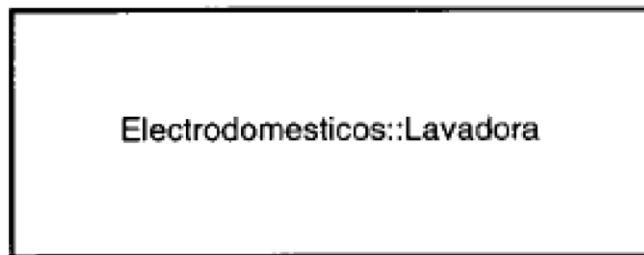
Ejemplo de Paquetes en el modelamiento de un Sistema.

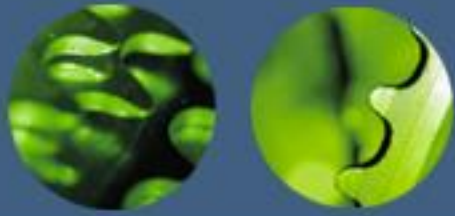




Paquetes

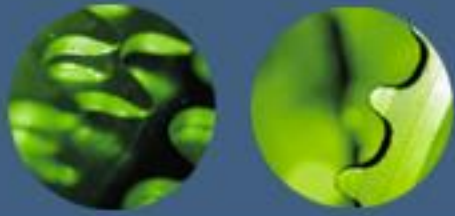
Si la Clase Lavadora pertenece al Paquete llamado Electrodoméstico, su representación seria:





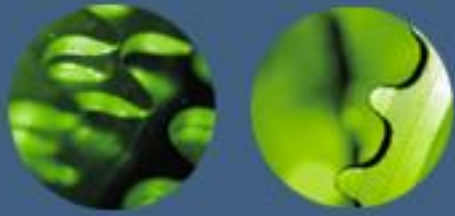
Relaciones

- Todo sistema abarca muchas clases y objetos
- Los objetos contribuyen en el comportamiento de un sistema colaborando entre si
 - La colaboración se logra a través de las relaciones
- Existen dos tipos principales de relaciones
 - Asociación
 - Agregación



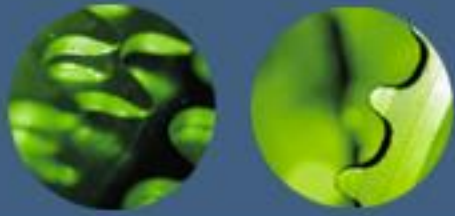
Asociaciones

- Son las relaciones entre los Objetos (Clases).
- Es una relación estructural que especifica que los Objetos de un elemento están conectados con los Objetos de otro.
- Los Objetos se pueden asociar con otro en mas de una forma y dirección.
- Un Objeto se puede asociar con mas de un Objeto y de diferentes Clase o Característica.
- Es posible que la Asociación se dé de manera recursiva en un Objeto



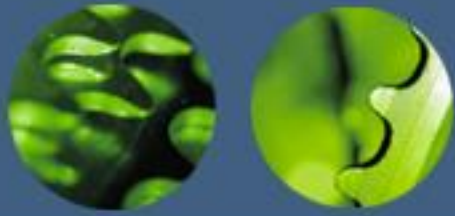
Asociaciones

- Existen cuatro adornos que se aplican a las asociaciones para facilitar su comprensión:
 - **Nombre:** describe la naturaleza de la relación.
 - **Rol:** Cuando una clase participa en una Asociación esta tiene un rol específico. Es la cara que dicha Clase presenta a la Clase que se encuentra en el otro extremo
 - **Multiplicidad:** Es señalar cuantos Objetos se pueden conectar a través de una instancia de la Asociación.



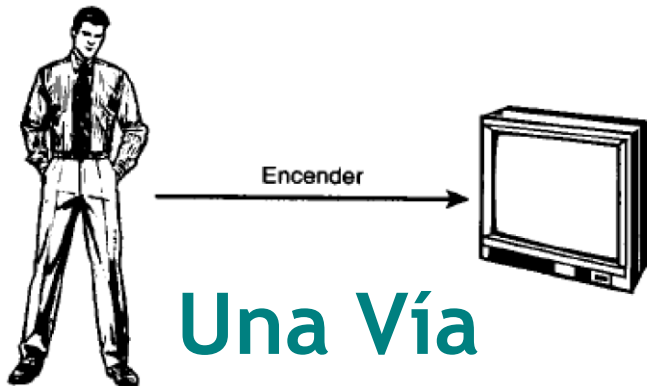
Asociaciones

- **Agregación:** Representa una relación del tipo “tiene-un”. Es un tipo especial de Asociación.
- **Composición:** Es una variación de la Agregación simple. Es la forma de Agregación, con una fuerte relación de pertenencia y vidas coincidentes de la parte del todo.

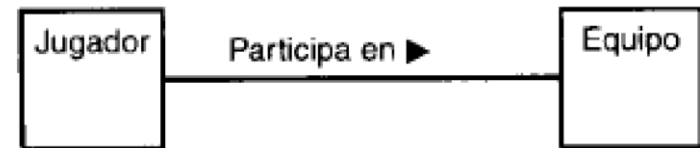


Asociaciones

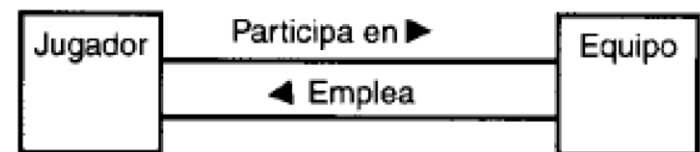
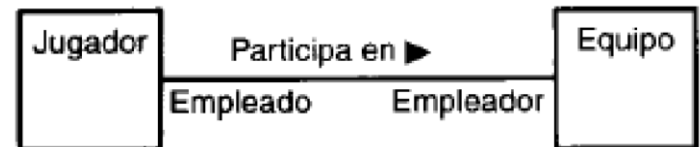
Es la Asociación entre un Jugador y un Equipo



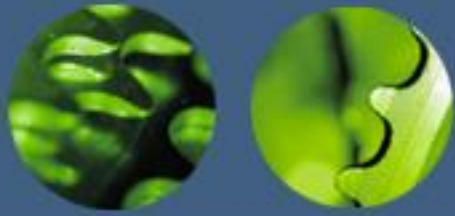
Una Vía



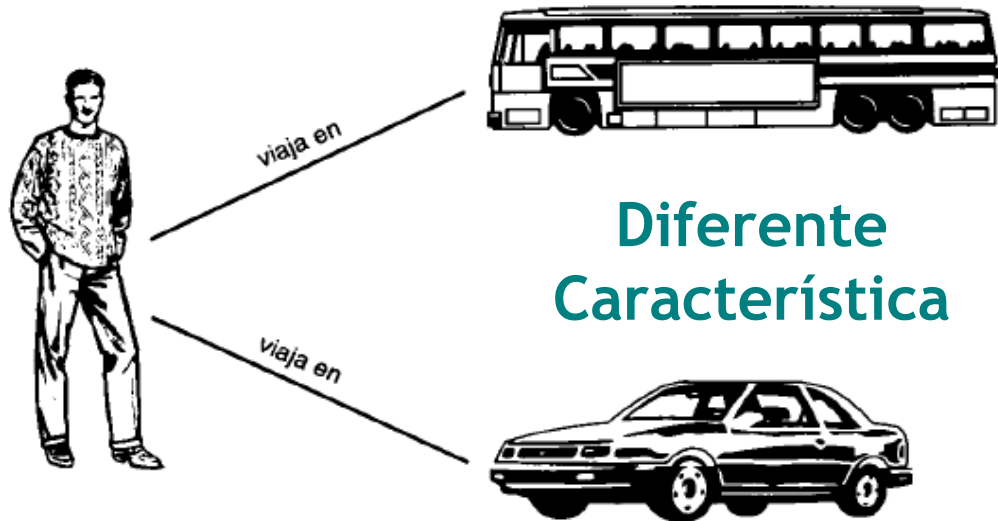
Es el papel que representa cada Clase en la Asociación



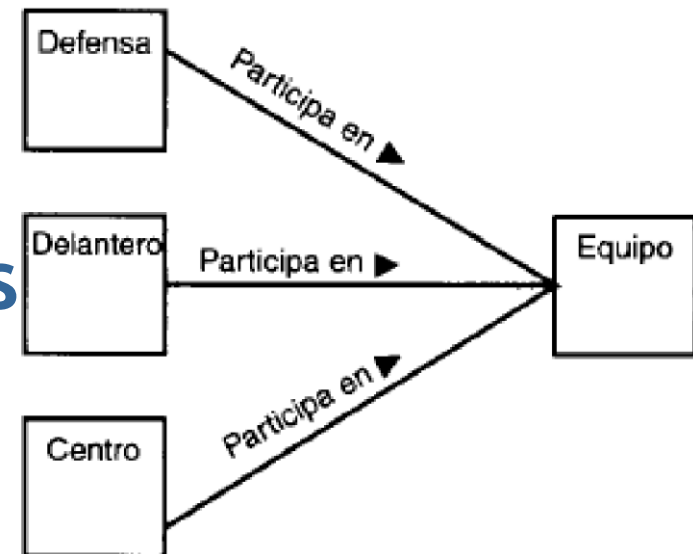
Dos Vías

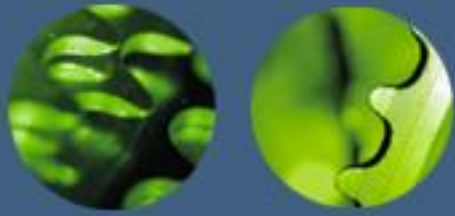


Asociaciones



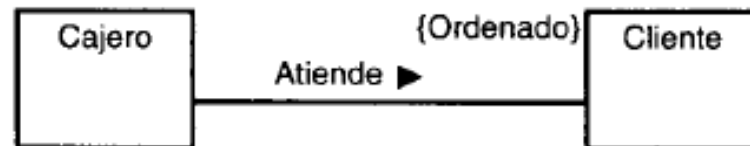
Relaciones Complejas

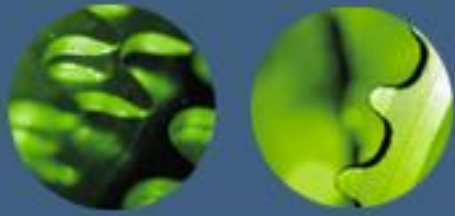




Restricciones en las Asociaciones

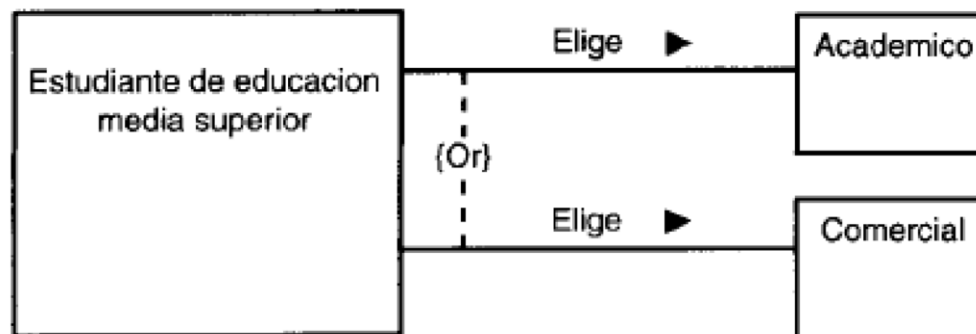
- En Asociaciones entre Clases pueden existir ciertas reglas.
- Se establece una Restricción en una Asociación. En este caso, la Asociación “Atiende” está restringida para que el Cajero atienda al Cliente en turno.

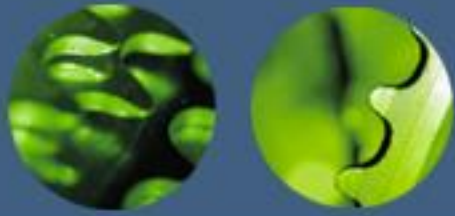




Restricciones en las Asociaciones

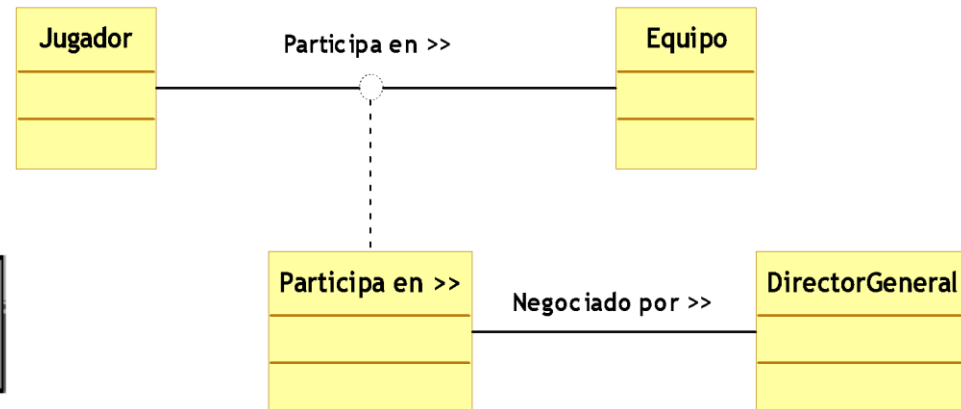
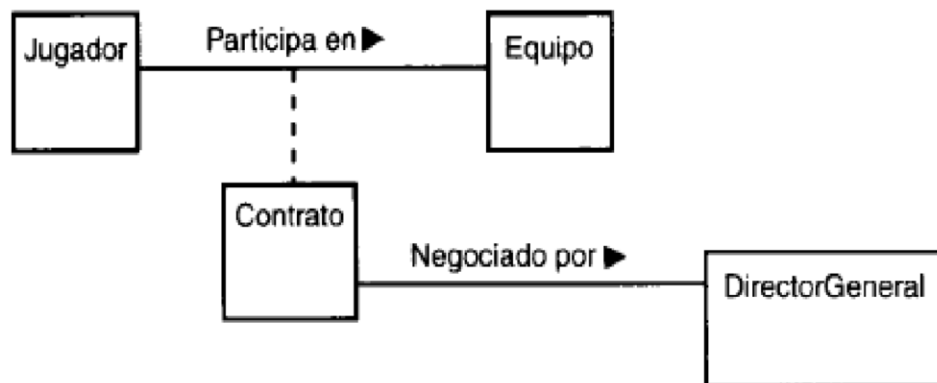
- Otro tipo de Restricción es la relación O (distinguida como {Or}) en una línea discontinua que conecte a dos líneas de Asociación.
- La siguiente figura modela a un Estudiante que elegirá entre un Curso Académico o Comercial

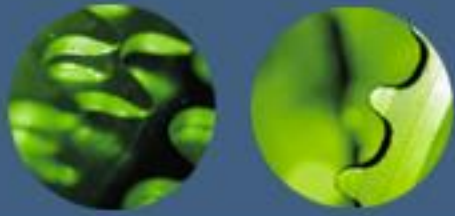




Clase de Asociación

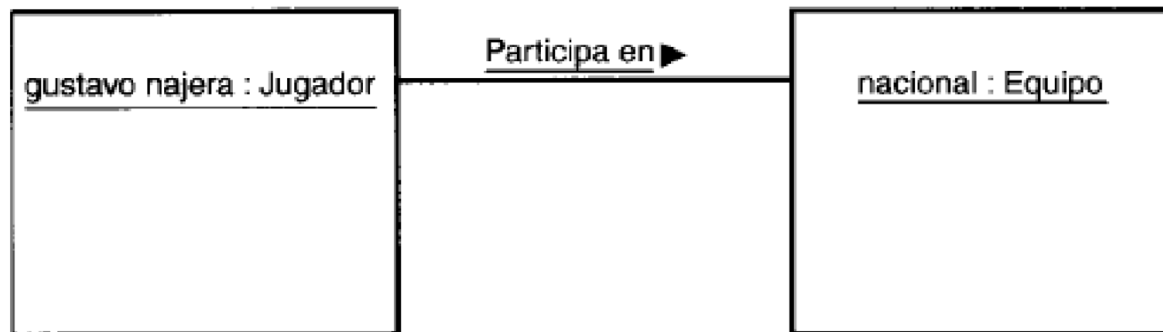
- Una Asociación igual que una Clase, puede contener Atributos y Métodos. Esto se llama Clase de Asociación.
- Una Clase de Asociación puede tener asociaciones con otras Clases.

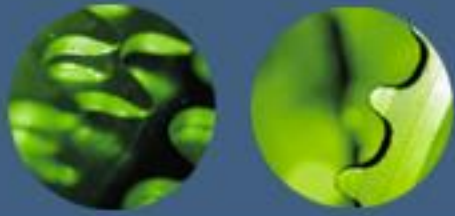




Vínculos

- Así como un Objeto es una Instancia de una Clase, una Asociación también se puede instanciar.

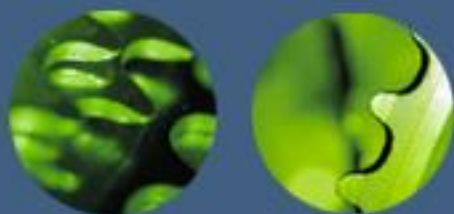




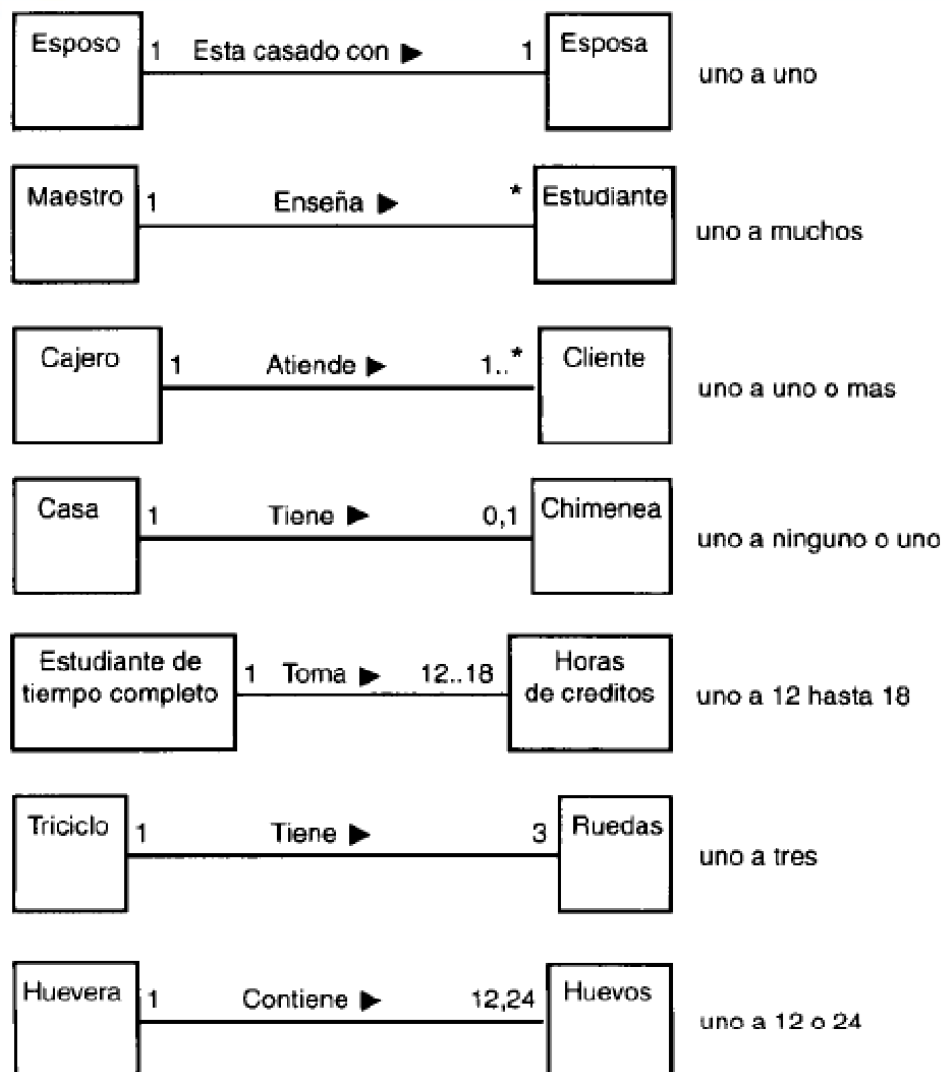
Multiplicidad

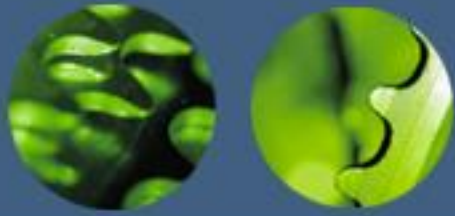
- Es un aspecto importante en las Asociaciones entre Objetos.
- Indica la cantidad de Objetos de una Clase que se relacionan con otro Objeto particular de la Clase Asociada.
- Las Multiplicidad pueden ser: 1 a 1, 1 a muchos, 1 a 5, etc.





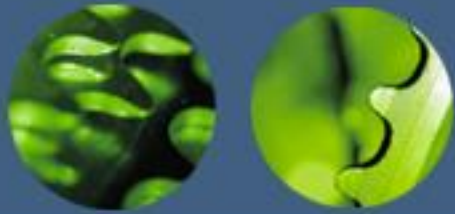
Multiplicidad



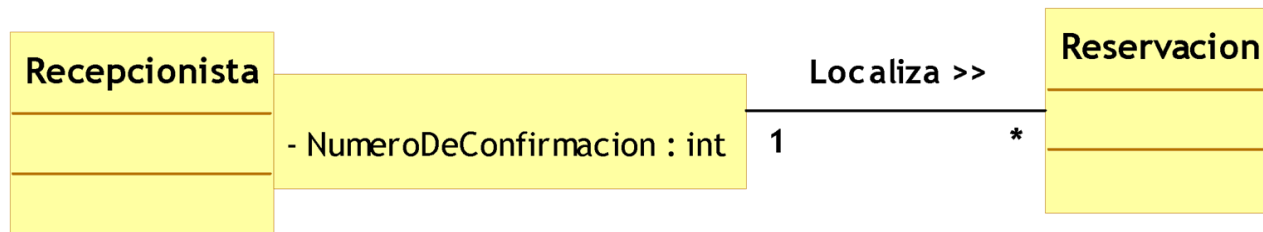
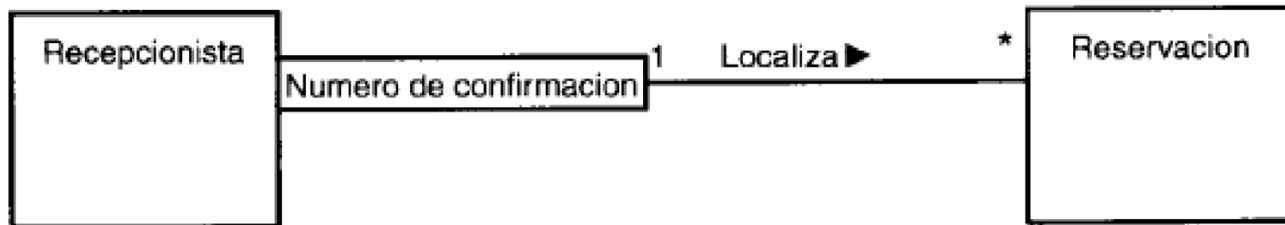


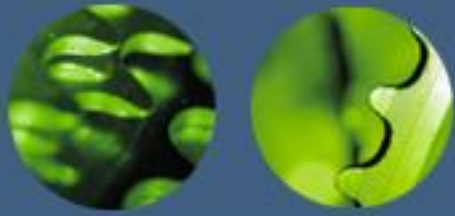
Asociaciones Calificadas

- Cuando la Multiplicidad es de Uno a Muchos, se presenta un reto importante, La Búsqueda.
- Cuando un Objeto de una Clase tiene que seleccionar un Objeto en particular de otro tipo para cumplir con un papel en la Asociación, la primera Clase deberá atenerse a un atributo en particular para localizar al Objeto adecuado.
- El Atributo identificador se conoce como *Calificador*.



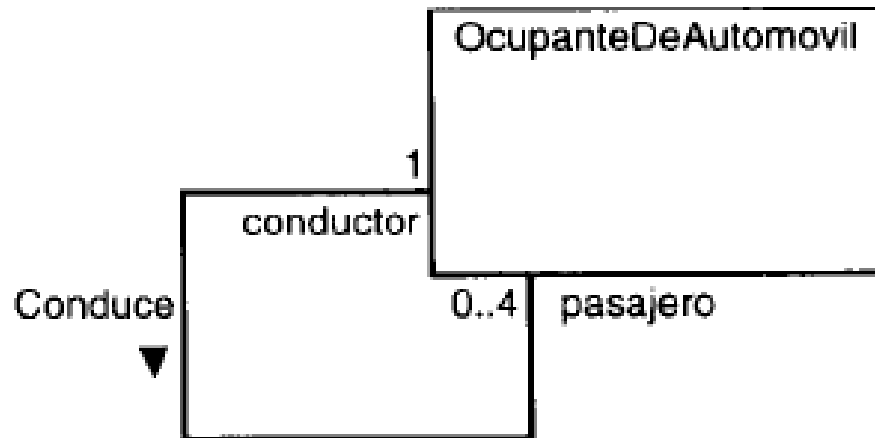
Asociaciones Calificadas

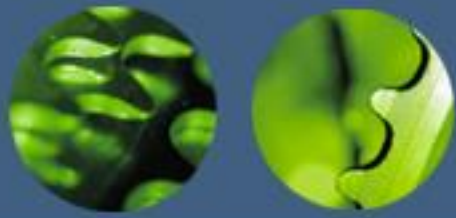




Asociaciones Reflexivas

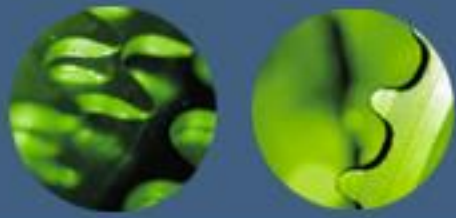
- Es una Relación consigo mismo.
- Esto ocurre cuando una Clase tiene Objetos que pueden jugar diversos papeles.



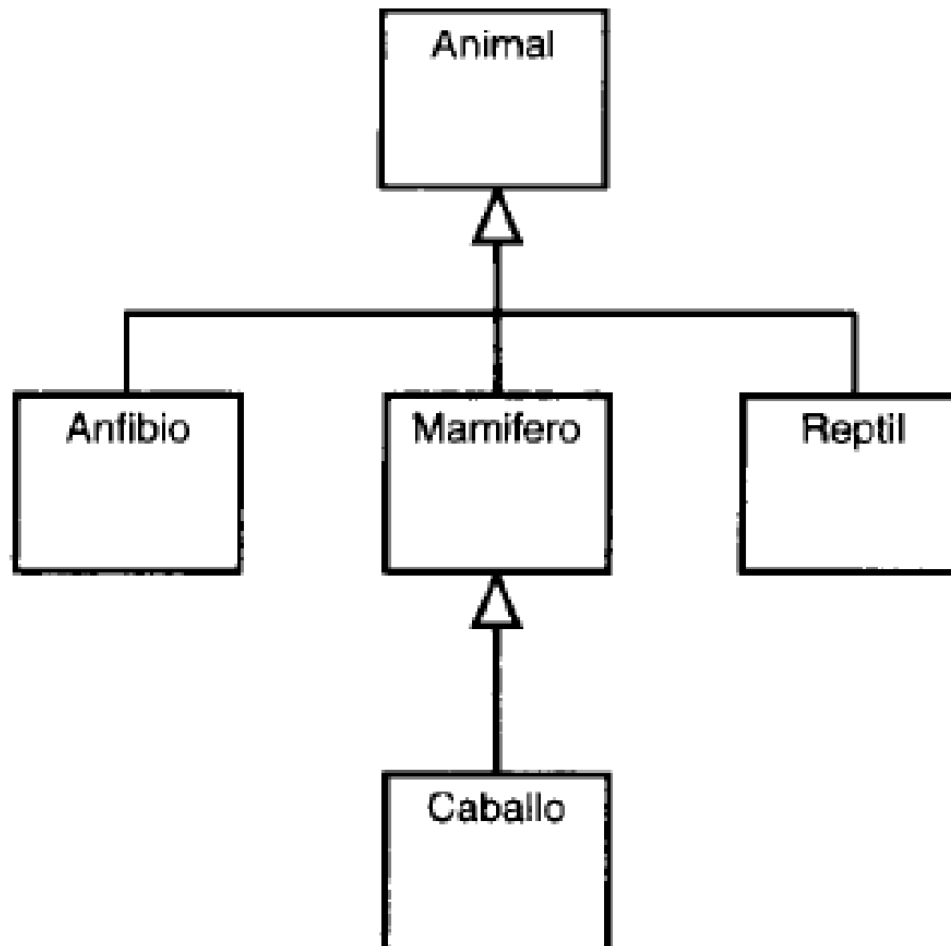


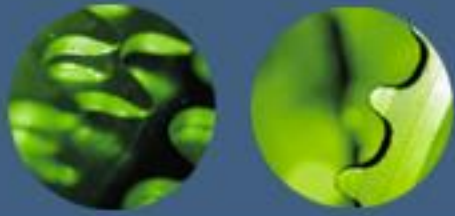
Herencia y Generalización

- La Herencia y Generalización es lo mismo.
- Como se dijo anteriormente, es uno de los aspectos mas importante que cuenta la OOP.
- Es cuando una SubClase o Clase Secundaria puede heredar los Atributos y Métodos de otra Clase (Clase Principal o SuperClase).
- La Clase Principal es mas genérica en su definición.



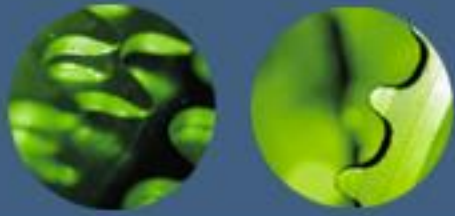
Herencia y Generalización





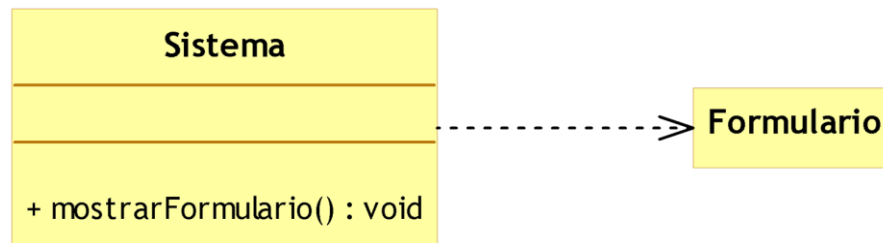
Dependencia

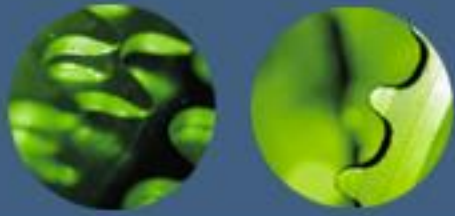
- Una relación de dependencia significa que una clase es dependiente de otra por algún servicio.
- Una relación de dependencia se indica si:
 - Las operaciones de la clase cliente crean objetos de la clase proveedora
 - Las operaciones de la clase cliente pasan argumentos a las instancias de la clase proveedora.



Dependencia

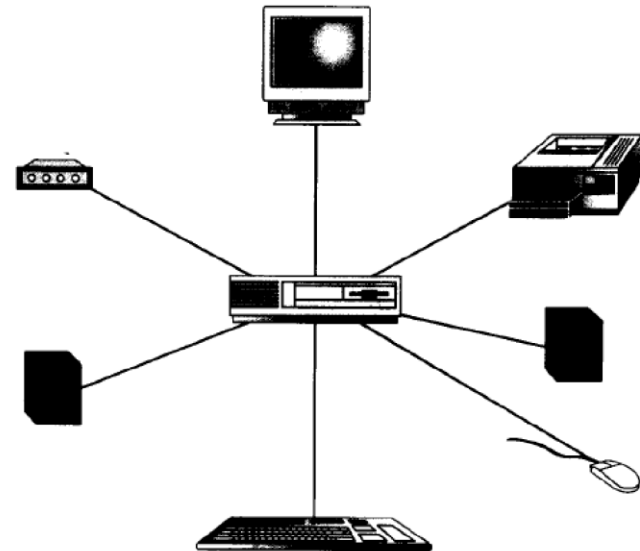
- Es cuando una Clase utiliza a otra Clase.

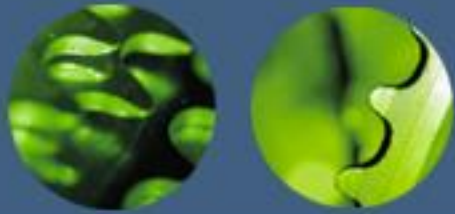




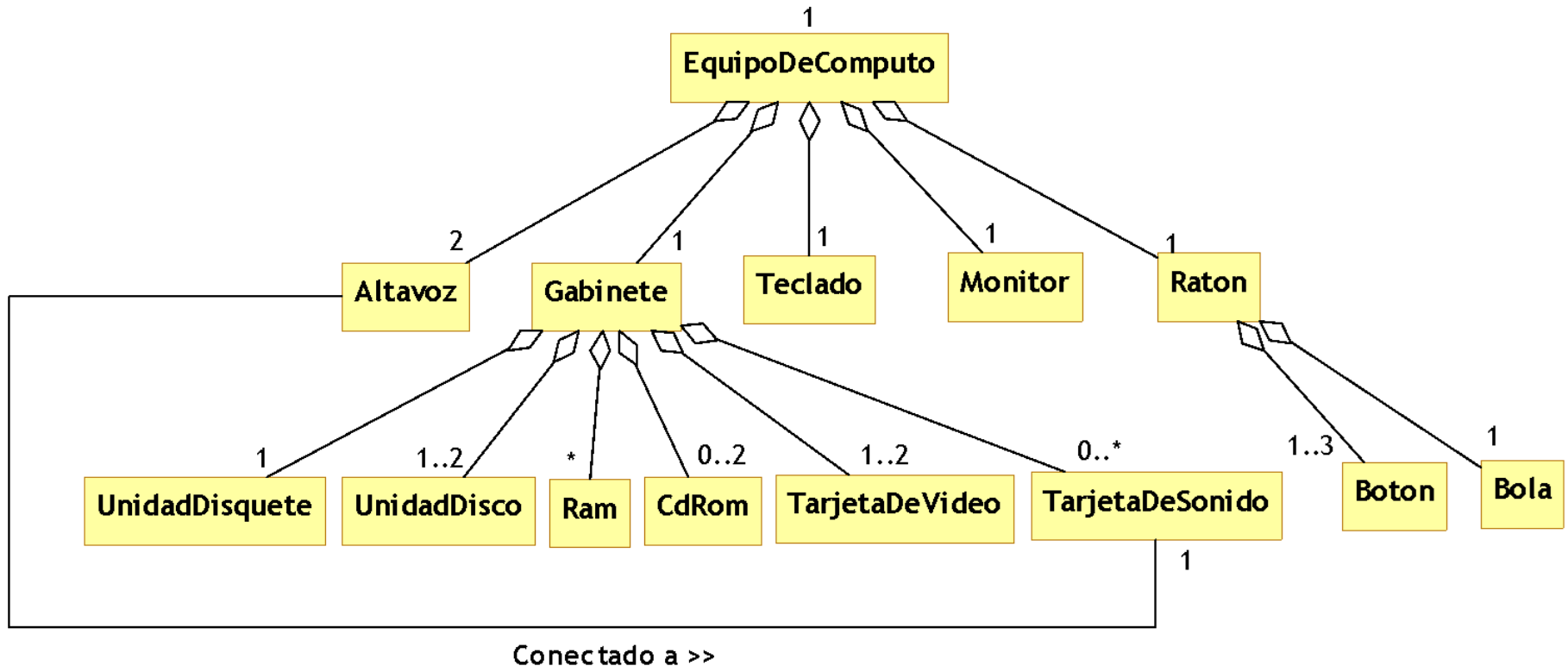
Agregación

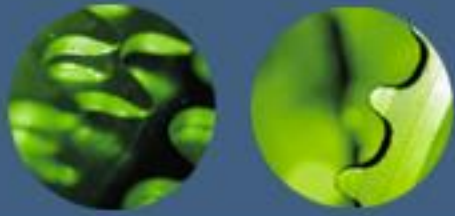
- Es una estrecha relación que existen entre varios Objetos.
- En un Objeto que se conforma de una combinación de diversos tipos de objetos.
- Una Clase consta de otra.





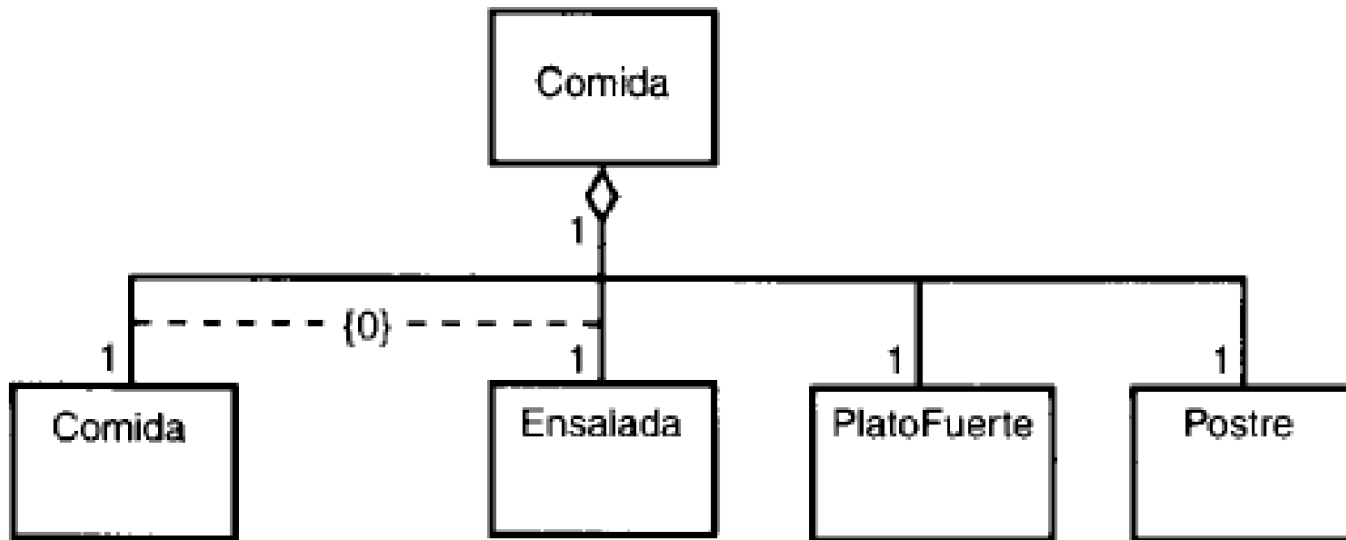
Agregación

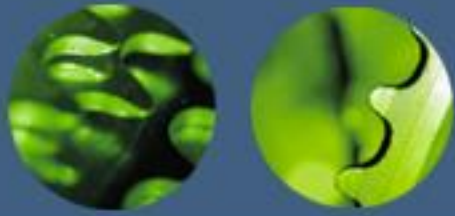




Restricciones en las Agregaciones

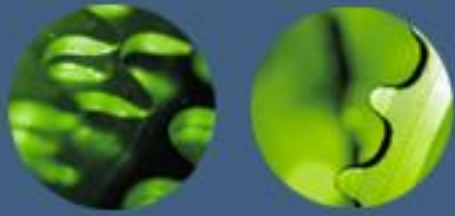
- Es posible que una Agregación existan relaciones con restricciones.





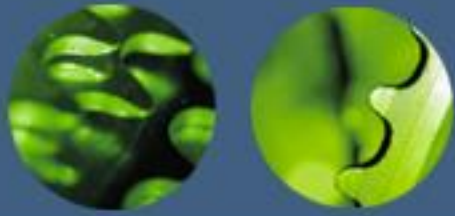
Composiciones

- Es cuando un componente se considera como tal solo como parte del Objeto compuesto.
 - Ejemplo: Una Camisa que esta compuesta por: Cuerpo, manga, cuello, botones, etc.
 - En ocasiones, un Objeto compuesto no tiene la misma Vida Útil que de sus Componentes.
 - Las partes puede crearse después de la parte que representa el Todo (la parte compuesta), una vez creada pertenecen a ella de manera que viven y mueren con ella.



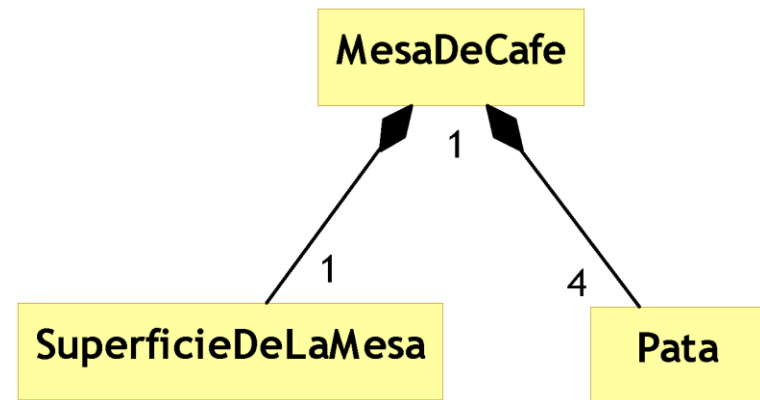
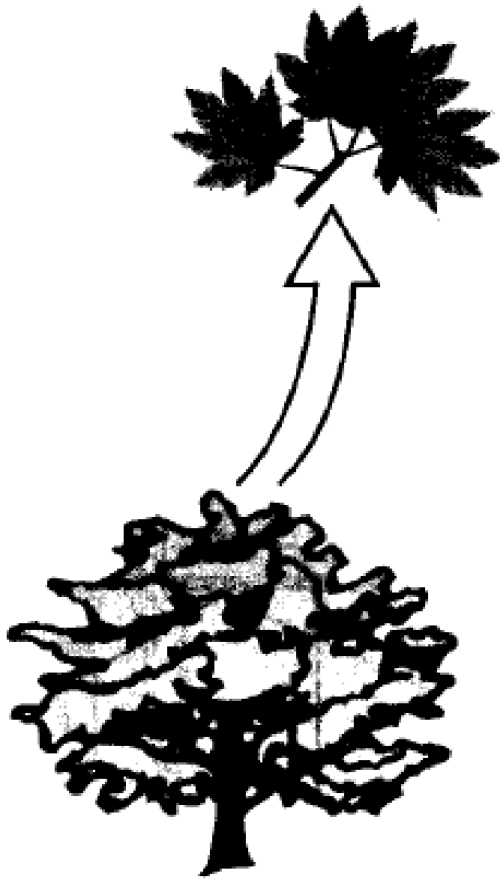
Composiciones

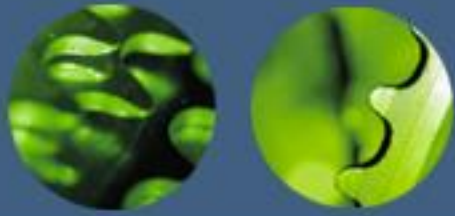
- Las partes pueden ser eliminadas antes que el Todo sea destruido, pero una vez sea eliminado el Todo, es destruido todas sus partes.
- El Todo es encargado de administrar o gestionar todas sus partes (creación, mantenimiento, disposición, etc).



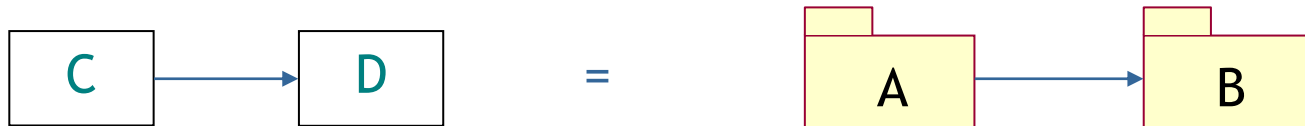
Composiciones

Ejemplo de Composición.





Relaciones de Clases entre Paquetes



La clase C pertenece al paquete A, la clase D pertenece al paquete B. Si existe una relación de asociación entre estas clases, existe una relación de dependencia entre paquetes. En este caso, se construye primero el paquete B, porque A depende de B.



La clase C pertenece al paquete A, la clase D pertenece al paquete B. Si existe una relación de agregación entre estas clases, existe una relación de dependencia entre paquetes. En este caso, se construye primero el paquete B, porque A depende de B.



La clase C pertenece al paquete A, la clase D pertenece al paquete B. Si existe una relación de herencia entre estas clases, existe una relación de dependencia entre paquetes. En este caso, se construye primero el paquete B, porque A depende de B.

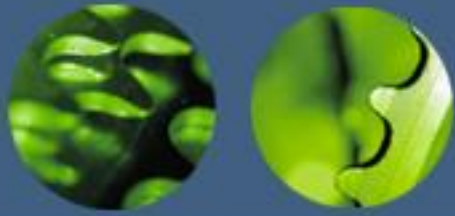


Diagrama de Contexto

- El Diagrama de Contexto es como el mapa detallado de alguna sección de un mapa de mayores dimensiones.
- Pueden ser necesarias varias secciones para capturar toda la información.
- Cuando se modele un Sistema puede producirse, con frecuencia, agrupamiento de Clases, como Agregaciones o Composiciones.

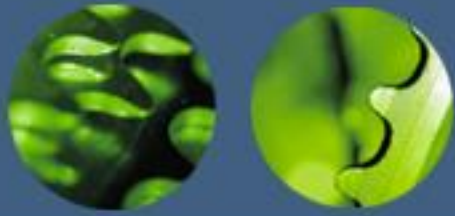
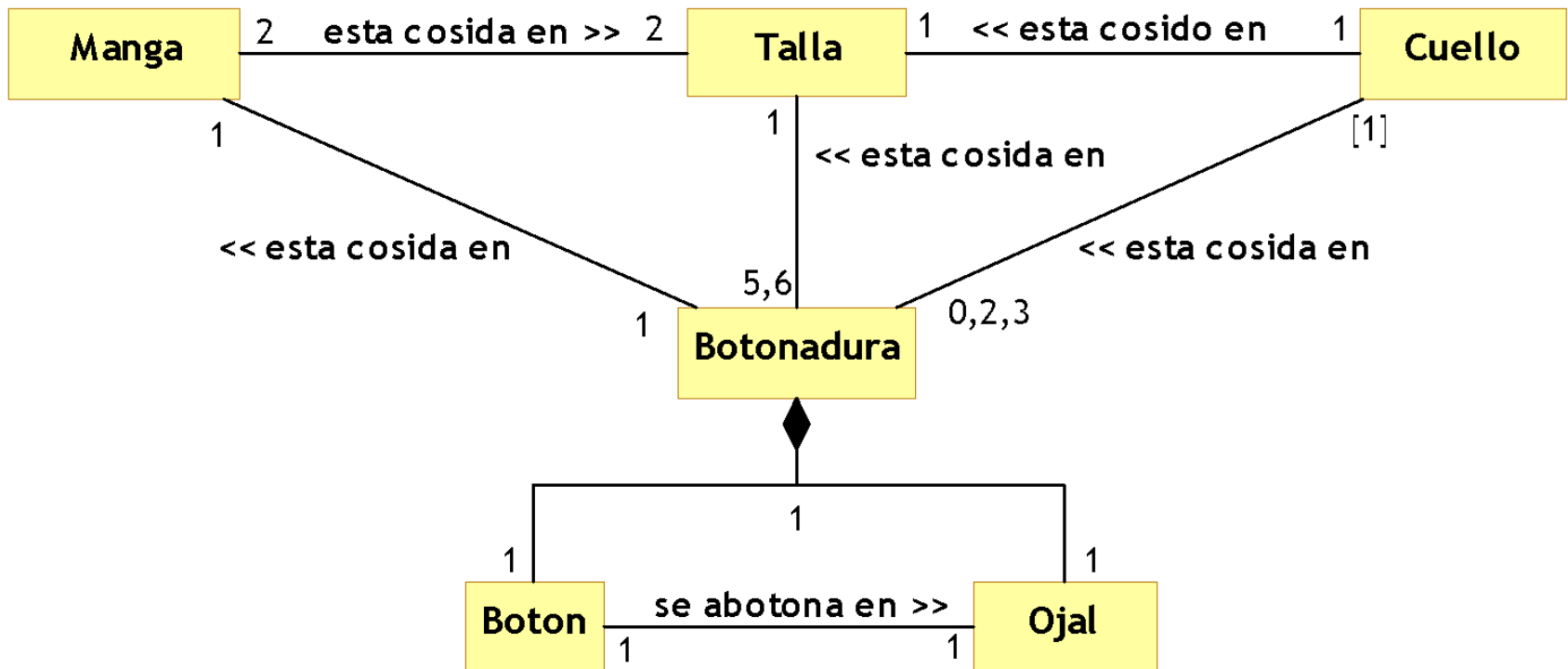
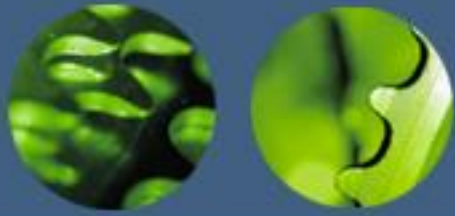


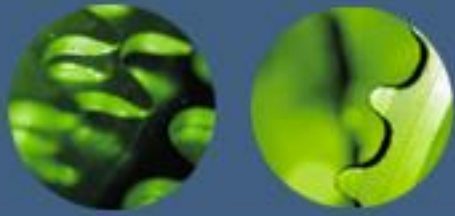
Diagrama de Contexto





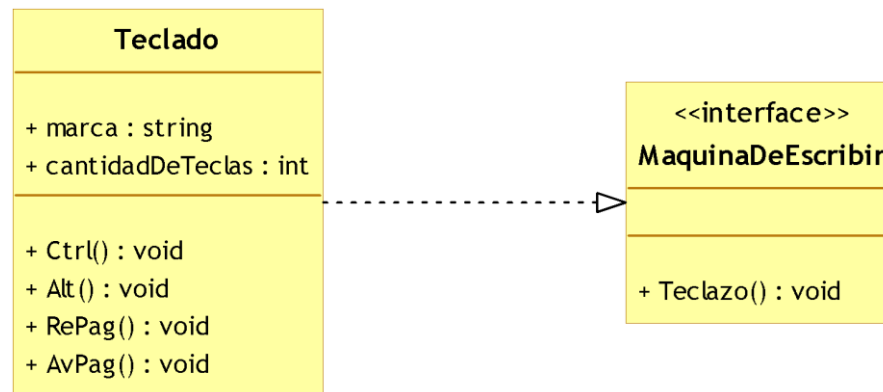
Interfaces

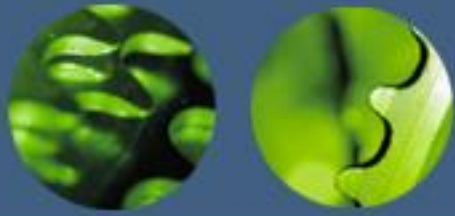
- Es un conjunto de operaciones (Métodos) que especifica cierto aspecto de la funcionalidad de una Clase, y es un conjunto de operaciones (Métodos) que una Clase presenta a otras.
- Recurso de diseño soportado por los lenguajes orientados a objetos que permite definir comportamiento.
- Permite que clases que no están estrechamente relacionadas entre sí deban tener el mismo comportamiento.
- La implementación de una interfaz es un contrato que obliga a la clase a implementar todos los métodos definidos en la interfaz.



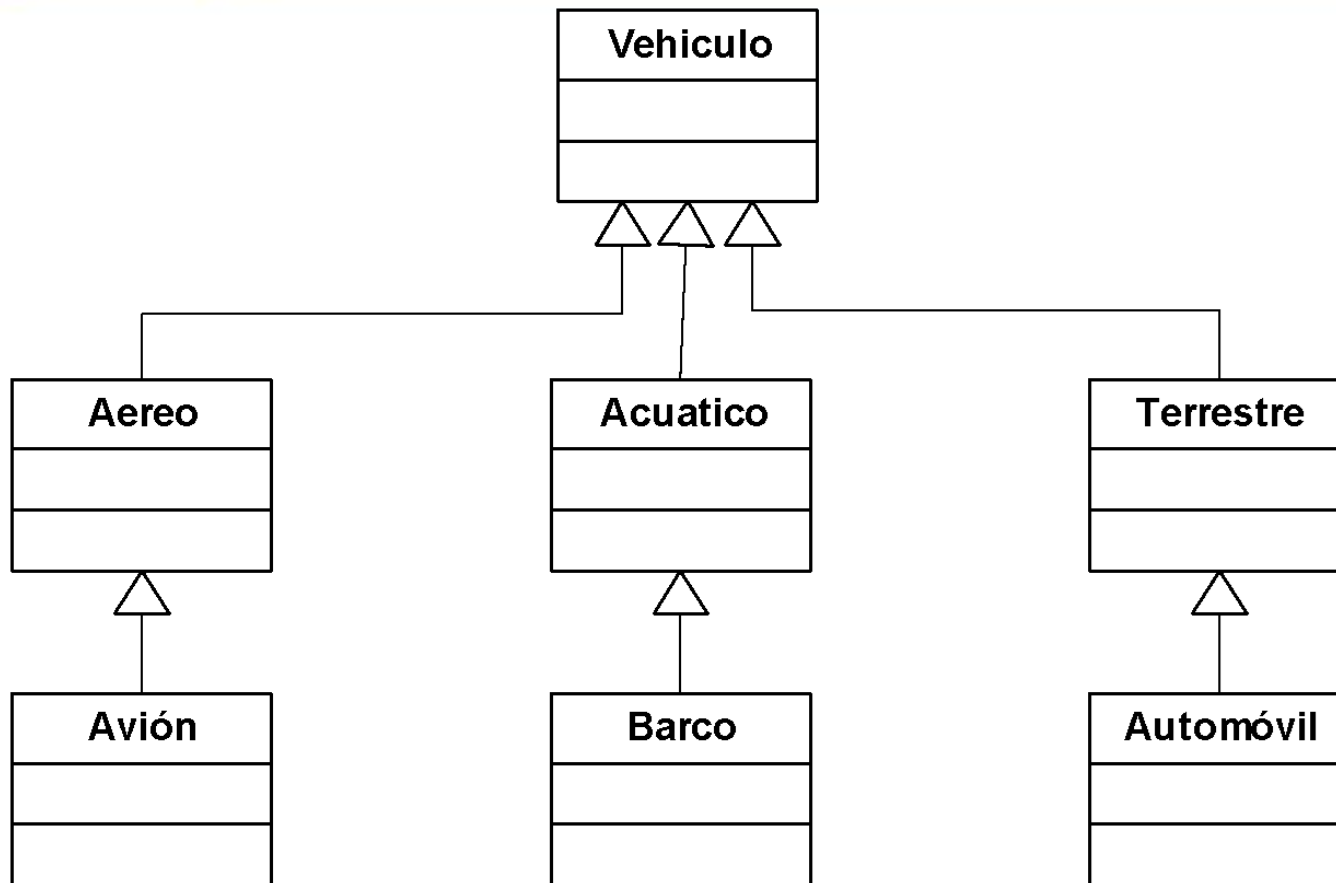
Interfaces

- Una vez que se hayan creado varias Clases, tal vez se de cuenta que no pertenecen a una Clase Principal, pero en su comportamiento debe incluir algunas de las mismas operaciones con las mismas firmas de la primera Clase.

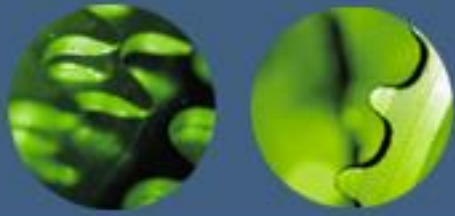




Interfaces

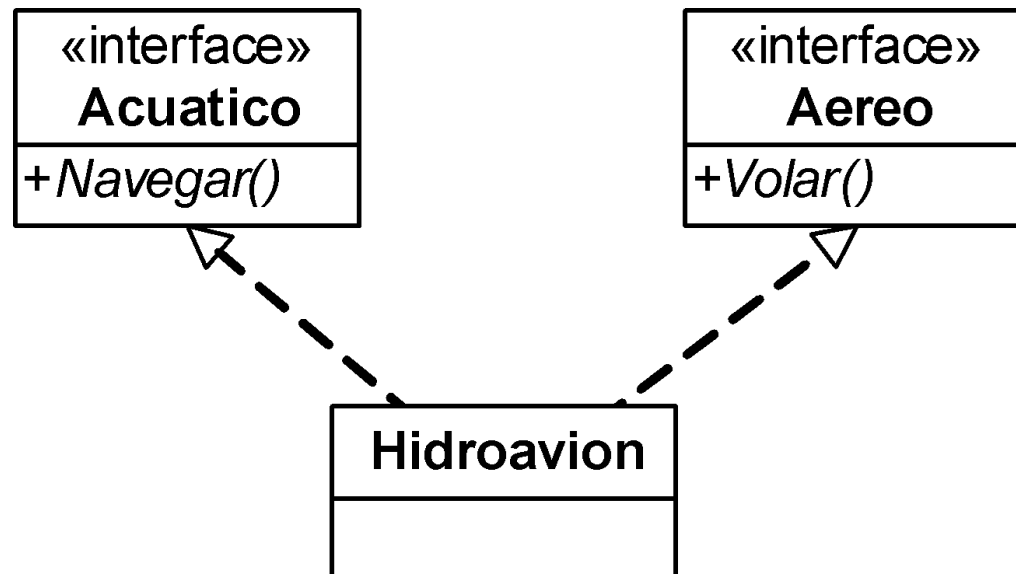


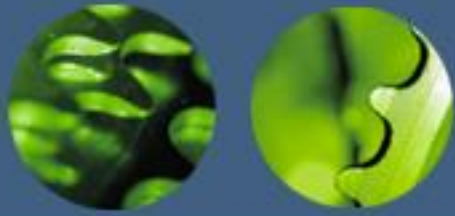
¿ De que clase heredaría la clase Hidroavión ?



Interfaces

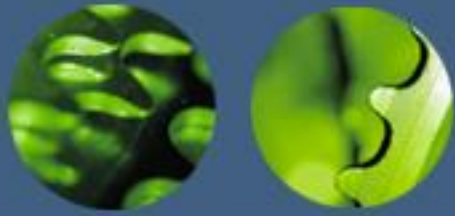
- Se crean las interfaces que definen comportamiento
- Hidroavión deberá definir los comportamientos de cada una de las interfaces que implemente





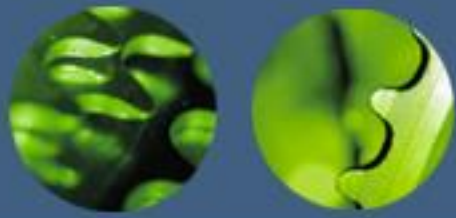
Visibilidad

- Identifica la visibilidad de un Atributo o Método.
- Tipos de Visibilidad: Públicos, Privados y Protegidos.
- **Públicos (+):** Son visibles dentro y fuera de la clase sin restricción alguna. La palabra reservada más común para denotarlos es "public".
- **Privados (-):** Los miembros privados son solo accesibles desde dentro de la clase donde existen. La palabra reservada más común para denotarlos es "private".



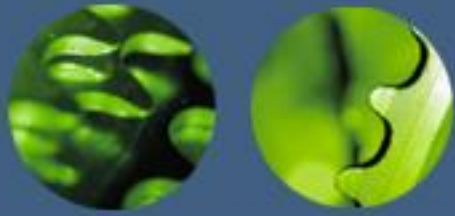
Visibilidad

- **Protegidos (#):** No serán accesible desde fuera de la clase, pero si podrá ser accesado por la clase además de las subclases que se deriven (herencia). La palabra reservada más común para denotarlos es "protected" o "friend“



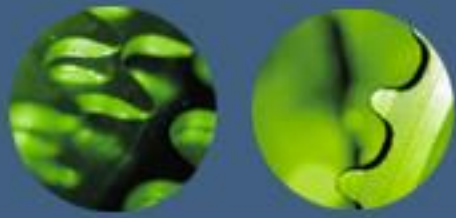
Ámbito

- Es la forma en que se relacionan los Atributos y Métodos dentro del Sistema.
- Existen dos tipos: Instancias y el de Archivador.
- **Instancias:** Cuenta con su propio valor.
- **Archivador:** solo abra un solo valor del Atributo o del Método en toda las instancias de la Clase. Este tipo de ámbito se presentan cuando los Atributos o Métodos se declaran Privados.



Constructores y Destructores

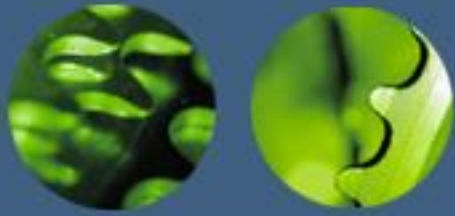
- **Constructores:** Para poder utilizar un Objeto, previamente debemos crearlo mediante el *Constructor de la Clase*. Este Método nos devuelve un objeto nuevo de una Clase. Normalmente en los lenguajes de programación estos Métodos se conocen como *New()*. Una Clase puede tener mas de un Constructor.



Constructores y Destructores

- **Destructores:** Al igual que existen constructores, en la mayoría de lenguajes de OOP, disponemos de *destructores*. Este es método es muy similar en su operatoria al constructor: existe uno interno (destructor por defecto) que siempre es llamado cuando la variable que contiene un objeto sale fuera de ámbito, y que llama, caso de existir al destructor que nosotros hayamos fabricado.

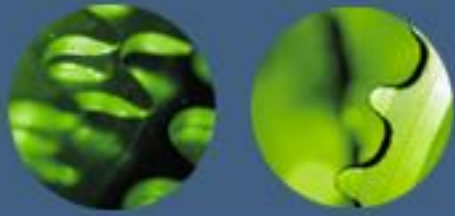
La funcionalidad del destructor por defecto es deshacer todo lo que el constructor por defecto realizó: eliminar las referencias en la tabla de símbolos, liberar la memoria ocupada, etc.



Atributos

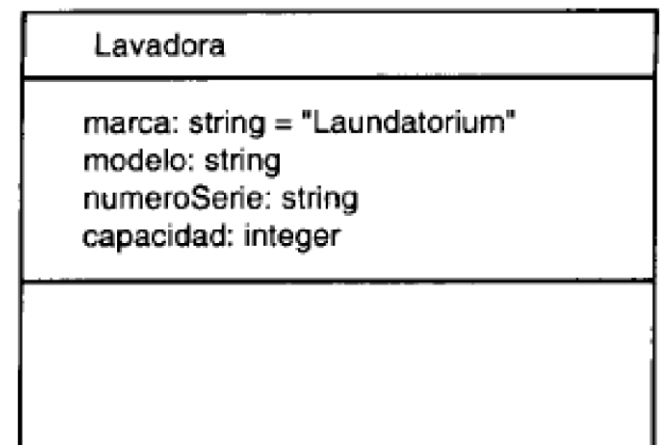
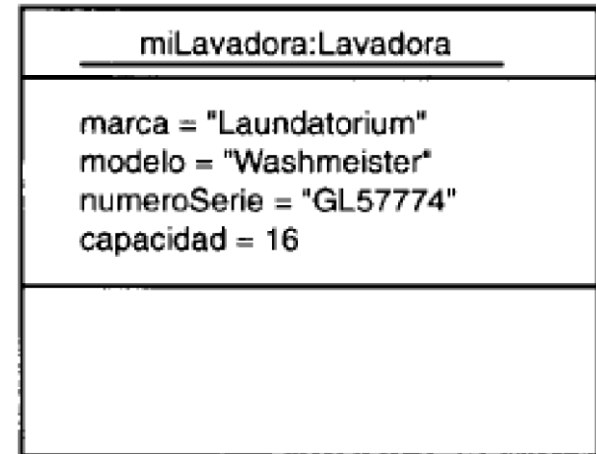
- Es una propiedad o característica de una Clase y describe un rango de valores que la propiedad podrá contener en los Objetos (esto es instancias) de la Clase.
- Una Clase podrá tener uno, varios o ningún atributo.

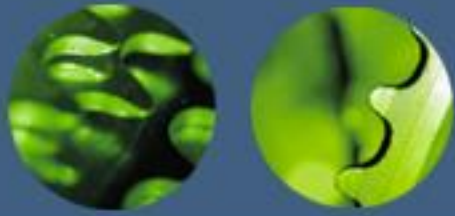
Lavadora
marca modelo numeroSerie capacidad



Atributos

- Todo Objeto de la Clase tiene un valor específico en cada atributo.
- UML da la opción de indicar información adicional a los Atributos como su tipo o valor Default.

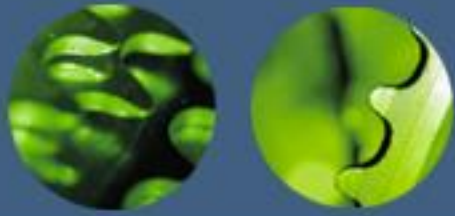




Operaciones o Métodos

- Es lo que la Clase puede realizar, o que usted (u otra Clase) pueden hacer a una Clase.

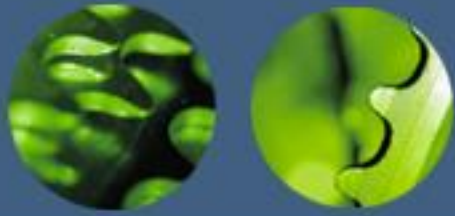
Lavadora
marca modelo numeroSerie capacidad
agregarRopa() sacarRopa() agregarDetergente() activar()



Operaciones o Métodos

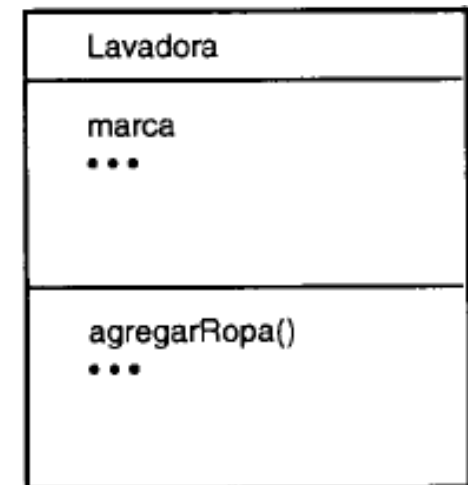
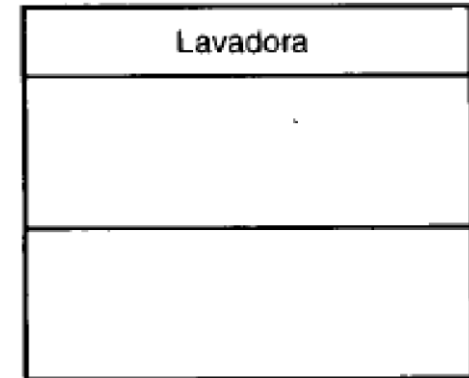
- Así como es posible adicionar información a los Atributos, también se puede hacer en los Métodos.
- Entre los paréntesis que preceden al nombre podrán mostrar el parámetro con que funcionara y su tipo de dato. Si devuelve un valor, también se puede mostrar el tipo de dato a devolver. Esto se conoce como la Firma de la Operación o del Método.

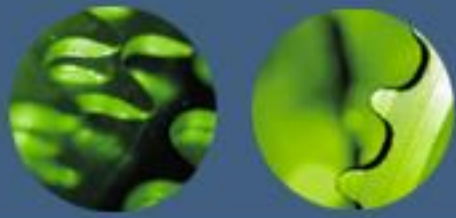
Lavadora
marca modelo numeroSerie capacidad
agregarRopa(C:String) sacarRopa(C:String) agregarDetergente(D:Integer) activar():Boolean



Atributos, Métodos y Concepción

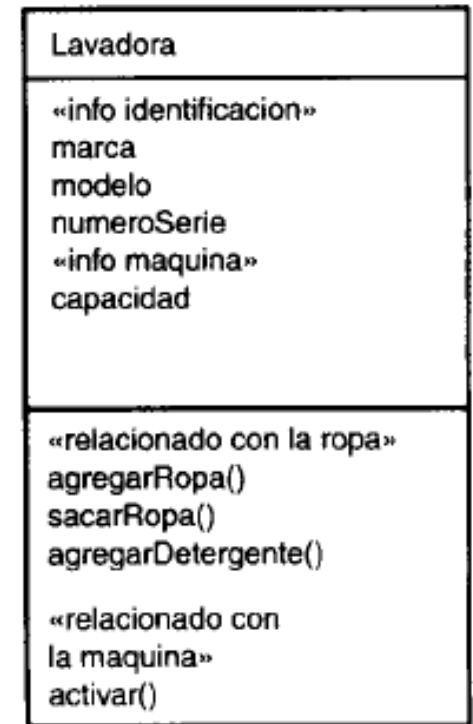
- En ocasiones para no saturar un Diagrama de Clase de tantos Atributos y Métodos, se puede representar la Clase sin sus Características.
- En ocasiones solo se necesite mostrar algunas de ellas o las mas Importantes para el Diseño.

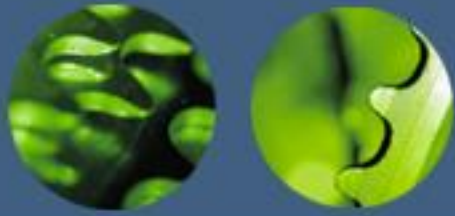




Atributos, Métodos y Concepción

- Si se tiene una larga lista de Atributos o Métodos, se podrán utilizar “*Estereotipos*” para organizarla y sea mas comprensible.
- Un Estereotipo es el modo que UML le permite extenderlo, es decir, crear nuevos elementos que son específicos de un problema en particular que intente resolver. Es una estructura flexible.

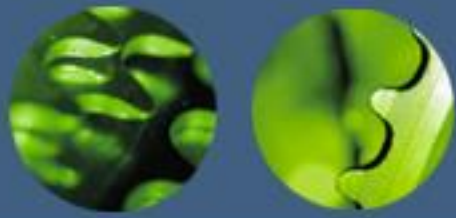




Responsabilidades y Restricciones

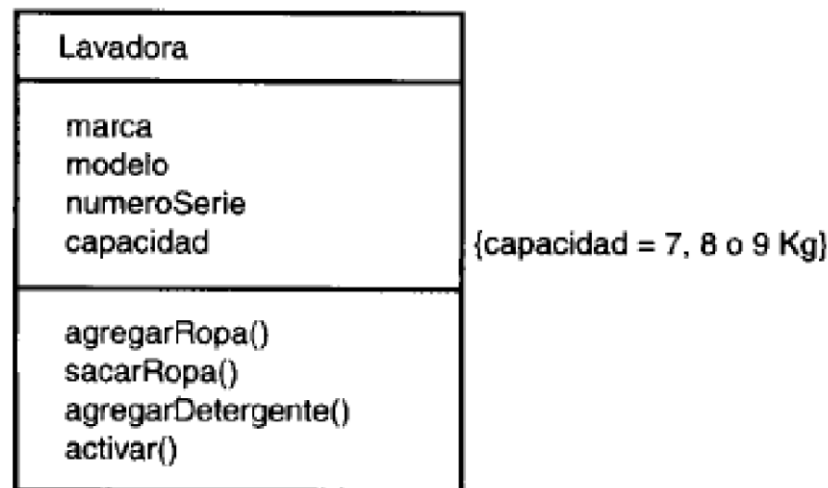
- En un área o cuadro abajo de los Métodos se puede mostrar la responsabilidad de la Clase.
- La Responsabilidad es una descripción breve de lo que hará la Clase.

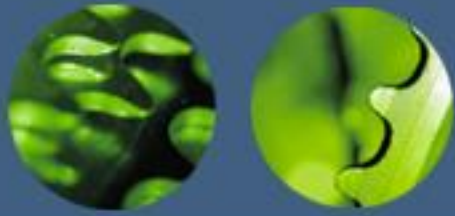
Lavadora
«info identificacion» marca modelo numeroSerie «info maquina» capacidad
«relacionado con la ropa» agregarRopa() sacarRopa() agregarDetergente() «relacionado con la maquina» activar()
Recibe ropa sucia y devuelve ropa limpia



Responsabilidades y Restricciones

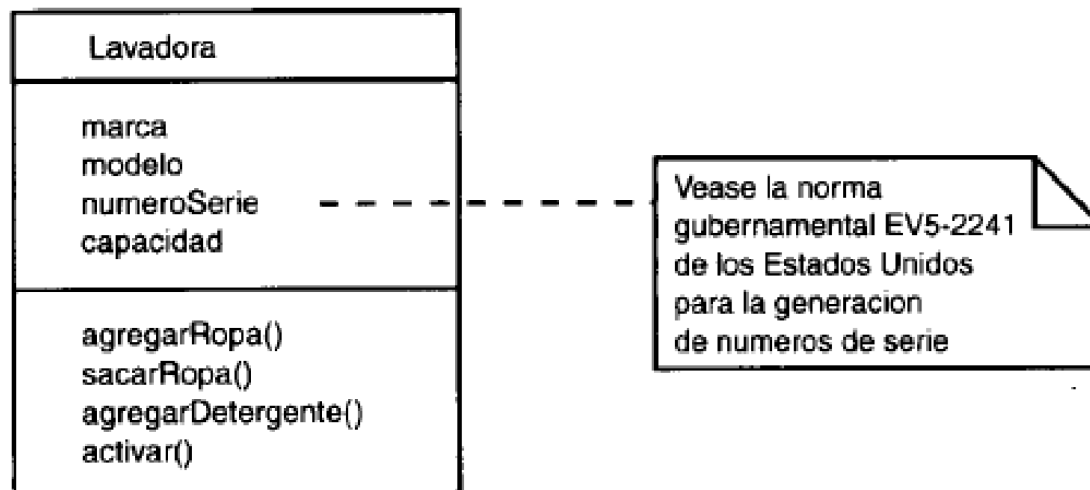
- Las Restricciones son reglas que llevan los Atributos para la capacidad de contener uno o tres posibles valores.
- La forma de representar una restricción es con un texto libre bordeado por llaves donde especifica los valores a contener.

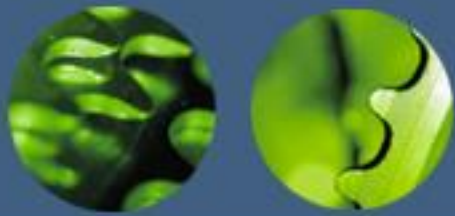




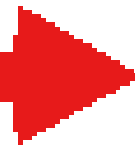
Notas Adjuntas

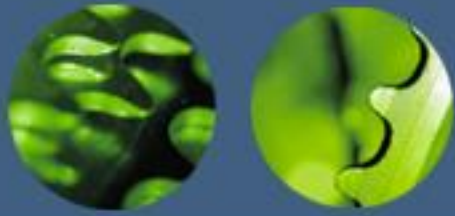
- Por encima de los Atributos, Métodos, responsabilidades y restricciones se pueden adicionar mas información por intermedio de las Notas Adjuntas.
- Una Nota puede contener tanto texto como imagen.





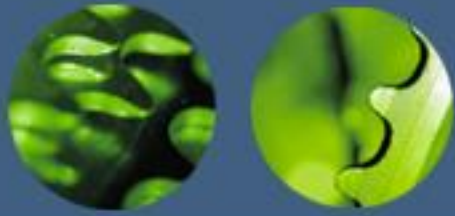
casos de uso





Casos de Uso

- Los Casos de Uso es una técnica para capturar información de cómo un sistema o negocio trabaja, o de cómo se desea que trabaje.
- Ayuda a obtener requerimientos desde el punto de vista del Usuario (actor), modelando la funcionalidad del sistema.
- No pertenece estrictamente al enfoque orientado a objeto, es una técnica para captura de requisitos.
- Es el poderoso concepto que ayuda al analista a comprender la forma en que un Sistema deberá comportarse.



Elementos de los Casos de Uso

Actor:

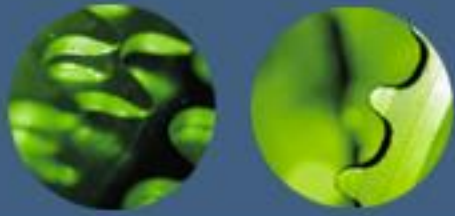


- rol que juega un usuario con respecto al sistema.
- un Actor no necesariamente representa a una persona en particular, sino más bien la labor que realiza frente al sistema.

Caso de Uso:



- l Operación o tarea específica que se realiza tras una orden de algún agente externo, originada por una petición de un actor o bien desde la invocación desde otro caso de uso



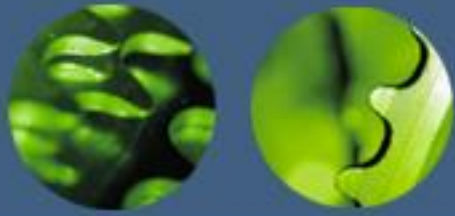
Relaciones de los Casos de Uso

- Son: Inclusión, Extensión, Generalización y Agrupamiento.
- **Asociaciones:** Es el tipo de relación más básica que indica la invocación desde un actor o caso de uso a otra operación (caso de uso).



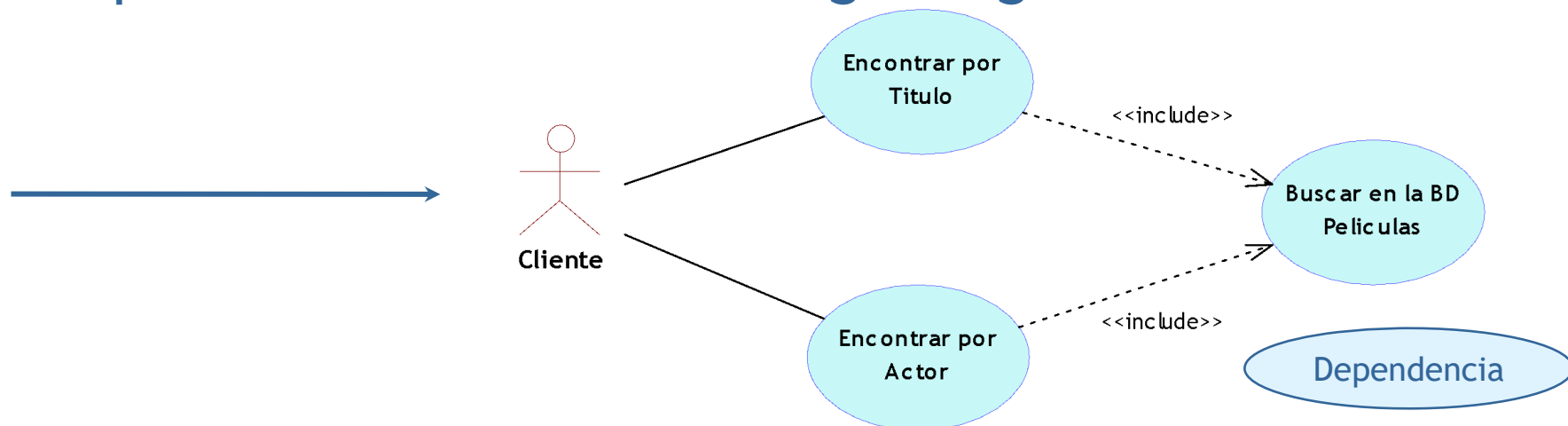
- **Dependencia o Instanciación:** Es una forma muy particular de relación entre clases, en la cual una clase depende de otra, es decir, se instancia (se crea).

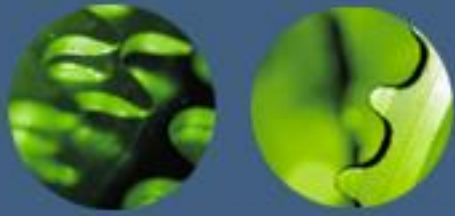




Relaciones de los Casos de Uso

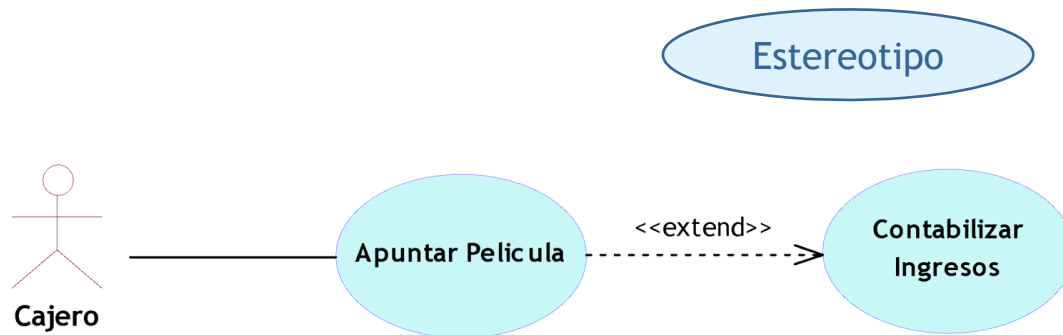
- **Inclusión <<include>>:** Volver a utilizar los pasos de un Caso de Uso dentro de otro. Permite factorizar un comportamiento en un caso de uso aparte y evitar repetir un mismo flujo en diferentes casos de uso. Incluye la funcionalidad de un Caso de Uso en otro.
- Un caso de uso base incorpora explícitamente el comportamiento de otro en algún lugar de su secuencia.

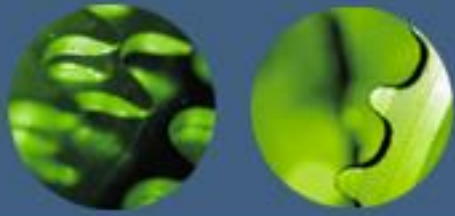




Relaciones de los Casos de Uso

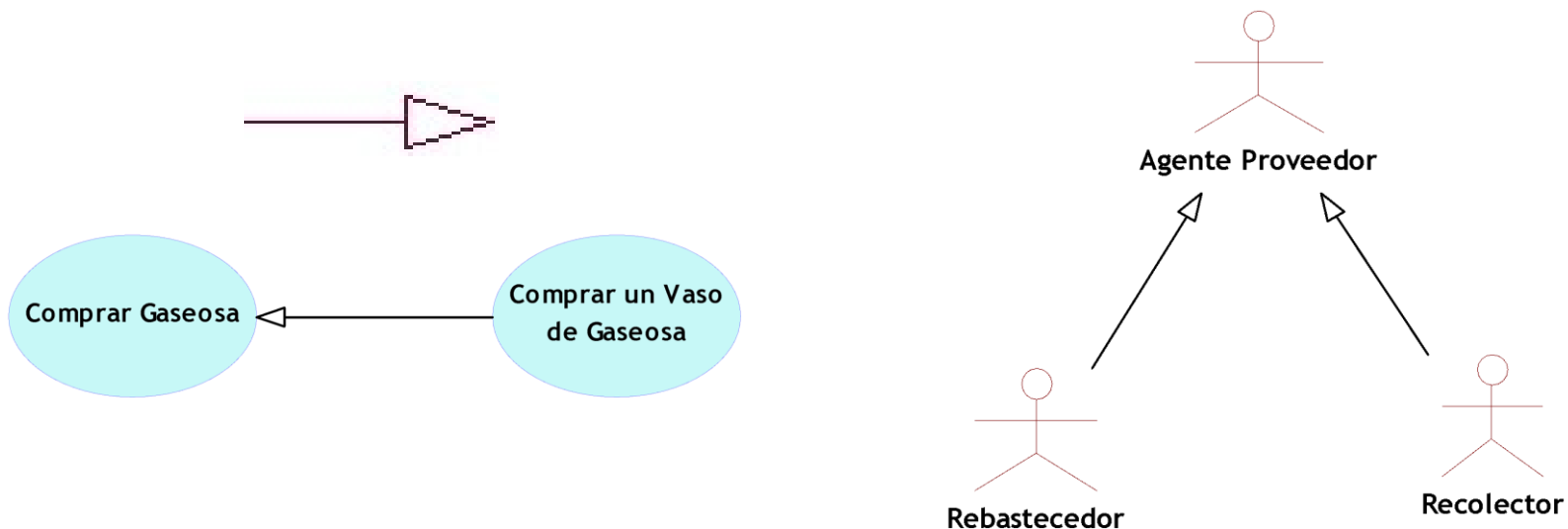
- **Extensión <<extend>>:** Un caso de uso base incorpora implícitamente el comportamiento de otro caso de uso en el lugar especificado indirectamente por este otro caso de uso.
- Extiende la funcionalidad de un Caso de Uso a otro bajo unas condiciones

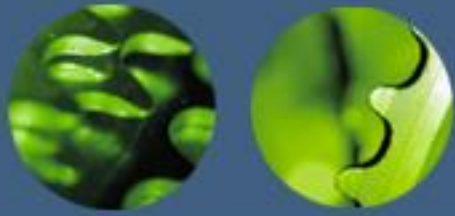




Relaciones de los Casos de Uso

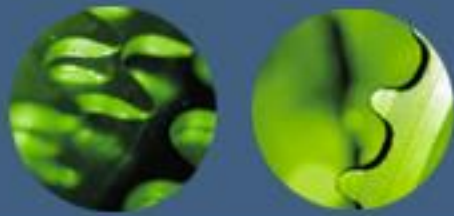
- **Generalización:** Las Clase se pueden heredar entre si, de igual forma sucede con los Casos de Uso. El Caso de Uso secundario hereda las acciones y significados del Primario, y además agrega sus propias acciones.



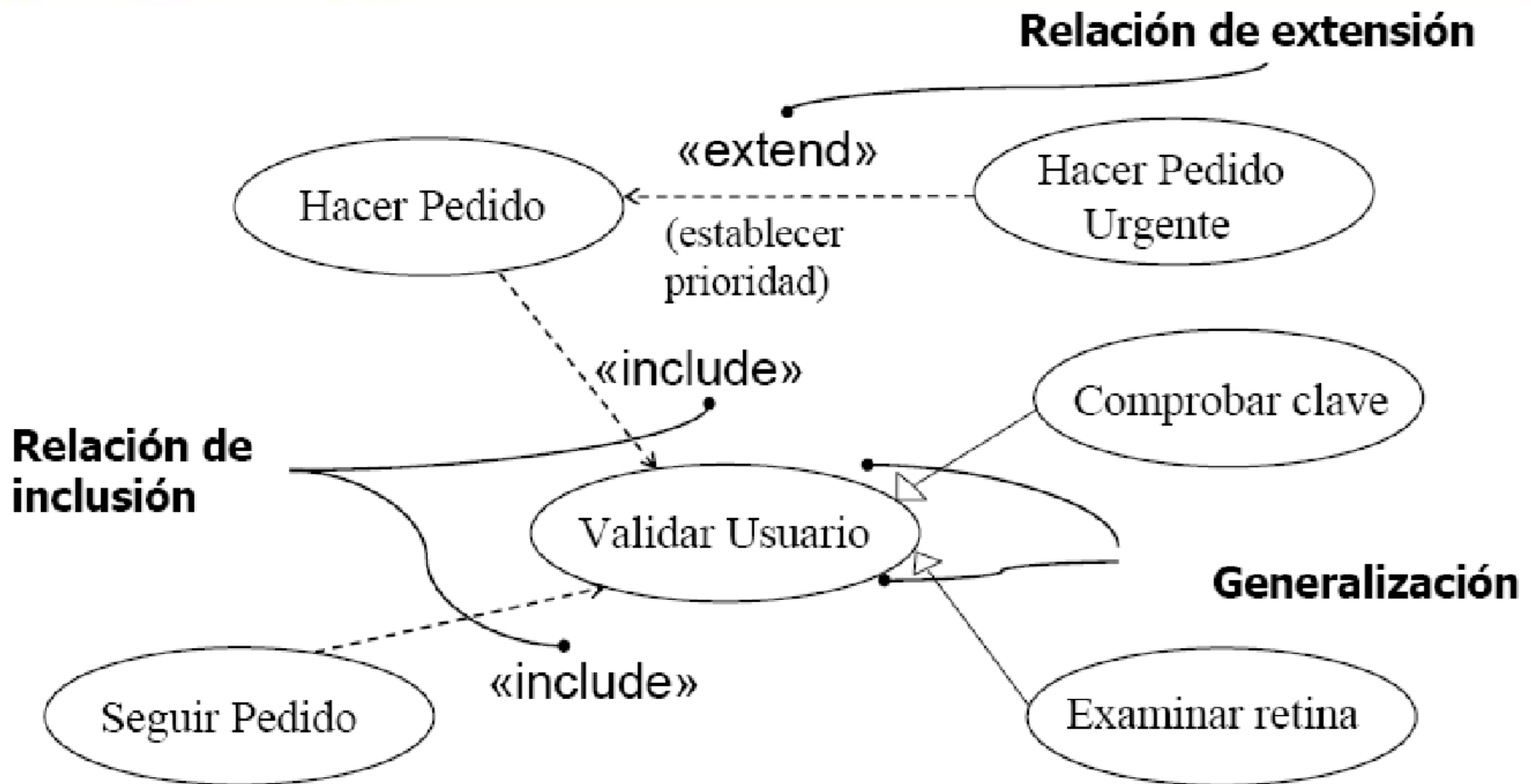


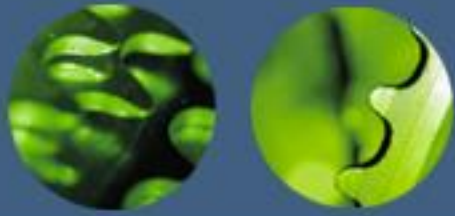
Relaciones de los Casos de Uso

- Se diferencian por el estereotipo <<uses>> (uso) o (<<extends>>) (herencia).
- **extends:** Se recomienda utilizar cuando un caso de uso es similar a otro (en sus características).
- **uses:** Se recomienda utilizar cuando se tiene un conjunto de características que son similares en más de un caso de uso y no se desea mantener copiada la descripción de la característica.



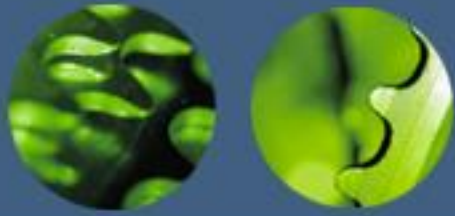
Relaciones de los Casos de Uso





Agrupamiento

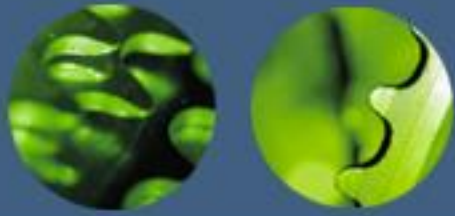
- Cuando un Sistema consta de varios Sub-Sistemas, o cuando se realiza toma de requerimientos a varios usuarios, necesitamos organizar los Casos de uso por Categorías o Tipos de Sistemas, la mejor forma de organizarlo son con los Paquetes.



Casos de Uso - Utilidad

- Modelar el comportamiento de un elemento (sistema, subsistema, clase):
 - Centrarse en qué hace el elemento, NO en cómo lo hace.
 - 1º) Sirven para intercambiar opiniones los expertos del dominio, los usuarios finales y los desarrolladores.
 - Los expertos del dominio especifican su vista externa para que los desarrolladores construyan su vista interna.

Los expertos del dominio especifican su vista externa para que los desarrolladores construyan su vista interna.

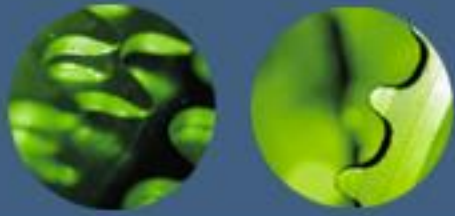


Casos de Uso - Utilidad

- 2º) El creador del elemento comunica cómo se debería usar.

El elemento puede ser complejo y tener muchas operaciones.

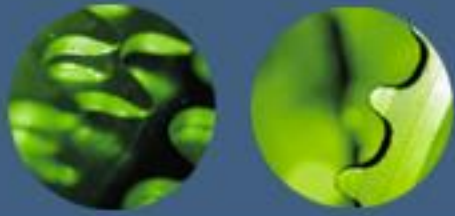
- 3º) Sirven de base para probar el sistema una vez implementado.



Casos de Uso

Pasos a seguir:

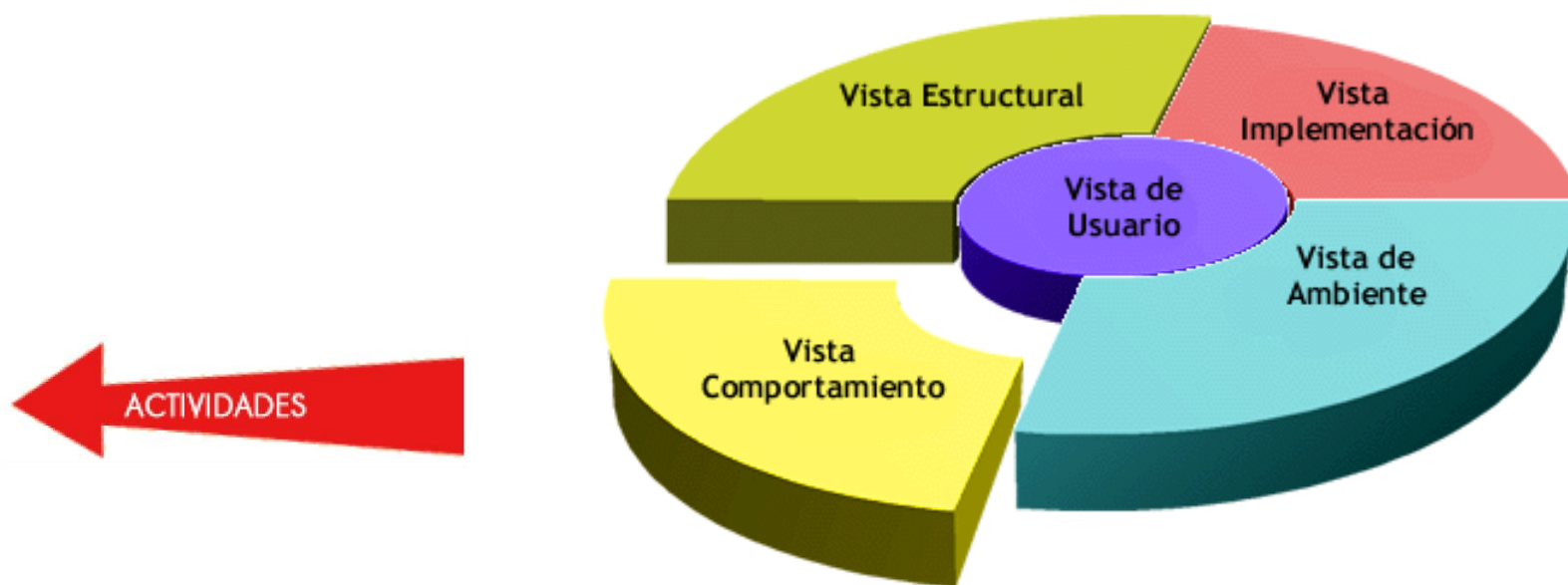
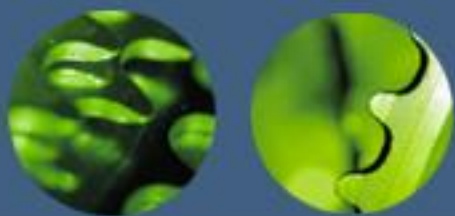
- Identificar los actores que interactúan con el elemento.
- Organizar los actores (roles generales, roles especializados, ...).
- Considerar las formas más importantes que tiene cada actor de interactuar con el elemento.
- Considerar las formas excepcionales que tiene cada actor de interactuar con el elemento.
- Organizar estos comportamientos utilizando las relaciones entre casos de uso vistas.
- Especificar cada caso de uso con texto y trazas de eventos.



Casos de Uso

Sugerencias y consejos:

- Cada caso de uso debe representar un comportamiento distinto e identificable del sistema (razonablemente atómico).
- Factorizar el comportamiento común: include.
- Factorizar las variantes de comportamiento: extends.
- Describir el flujo de eventos de manera suficientemente clara para que alguien externo lo entienda.
- Mostrar sólo los importantes para comprender el comportamiento del sistema.
- Mostrar sólo los actores implicados.



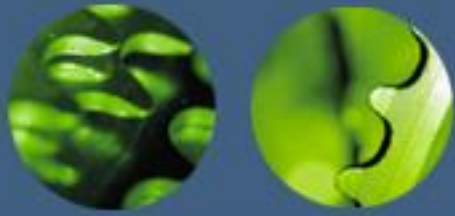
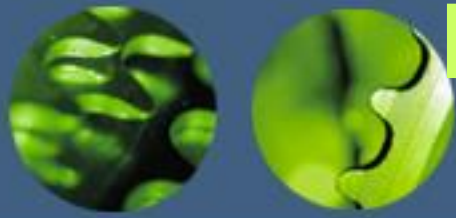
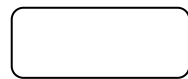


Diagrama de Actividades

- Diagrama de flujo que describe el orden de las actividades de un proceso.
- Describen las actividades que ocurren dentro de un Caso de Uso.
- Ha sido diseñado para mostrar una visión simplificada de lo que ocurre dentro de un proceso u operación.
- Este diagrama es una Extensión del Diagrama de Estado.



Elementos del Diagrama de Actividades



Actividad



Flujo



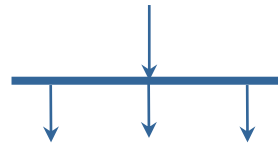
Inicio



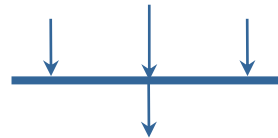
Fin



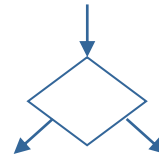
Separador



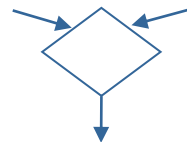
Bifurcación



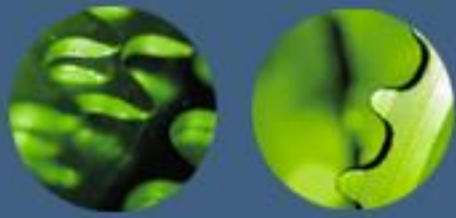
Unión



Subdivisión

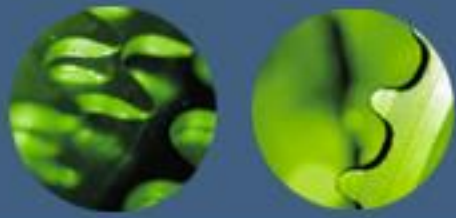


Unión

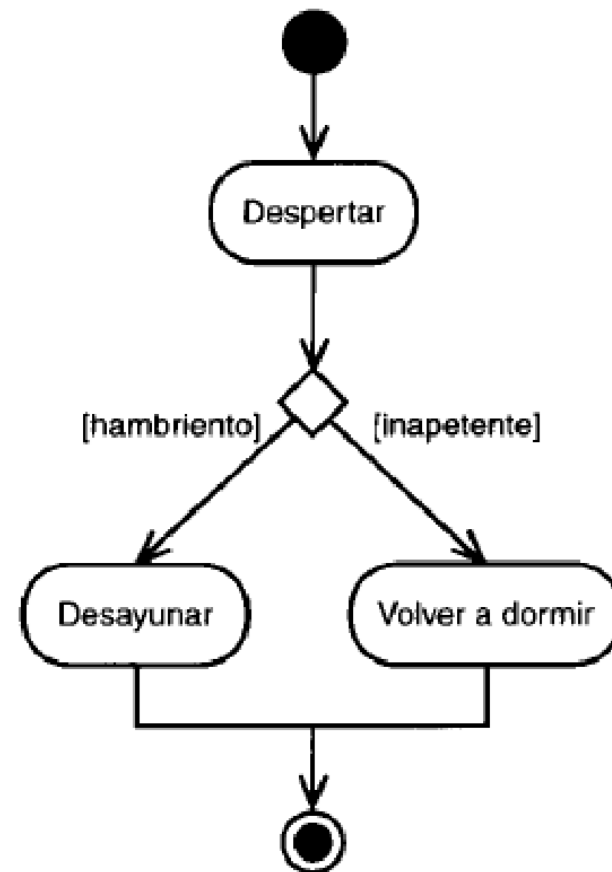
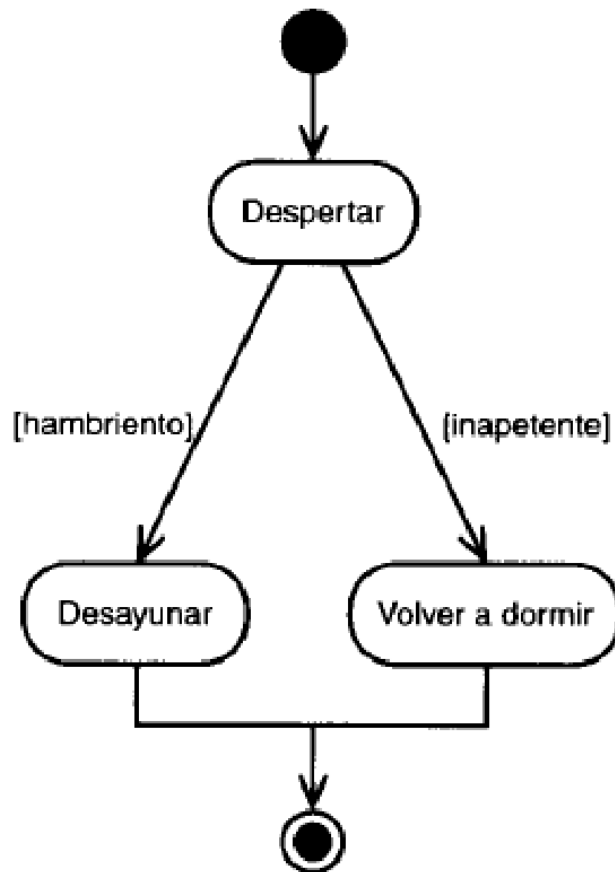


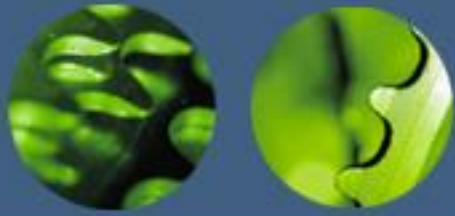
Decisiones en el Diagrama de Actividades

- Casi siempre en un Diagrama de Actividades se llegara a un punto donde se realizara alguna decisión, donde una lo llevara por un camino y otra por otro camino.
- Existen dos formas de representar los puntos de decisión:
 - La primera es mostrar las rutas posibles que parten directamente una actividad.
 - La segunda es llevar la transición hacia un rombo.



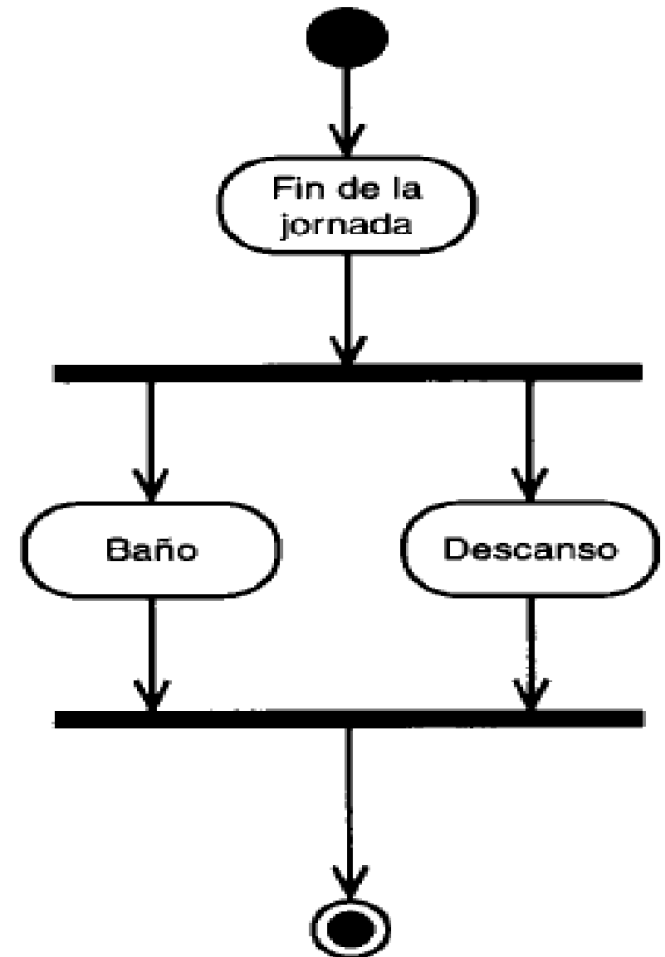
Decisiones en el Diagrama de Actividades

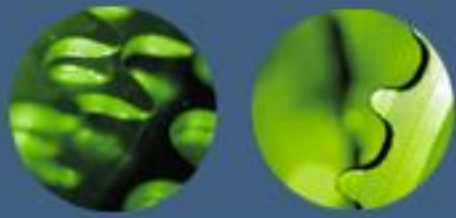




Rutas Concurrentes en el Diagrama de Actividades

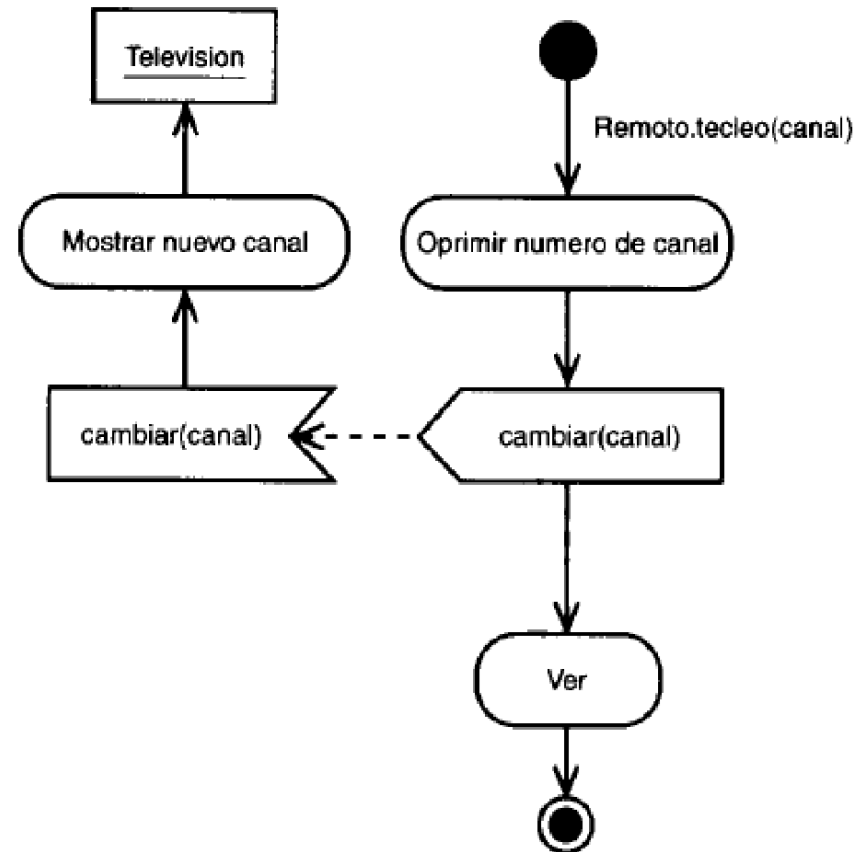
- Conforme como se modele las actividades, se tendrá la oportunidad de separar la transición en dos rutas que se ejecutan al mismo tiempo (en forma concurrente) y luego se reúna.





Indicaciones en el Diagrama de Actividades

- También es posible enviar una indicación. Cuando se reciba, la indicación provocara que se ejecute una actividad.
- El símbolo para enviar la indicación es un pentágono convexo y el que recibe un pentágono cóncavo.



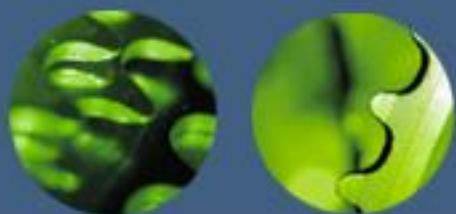
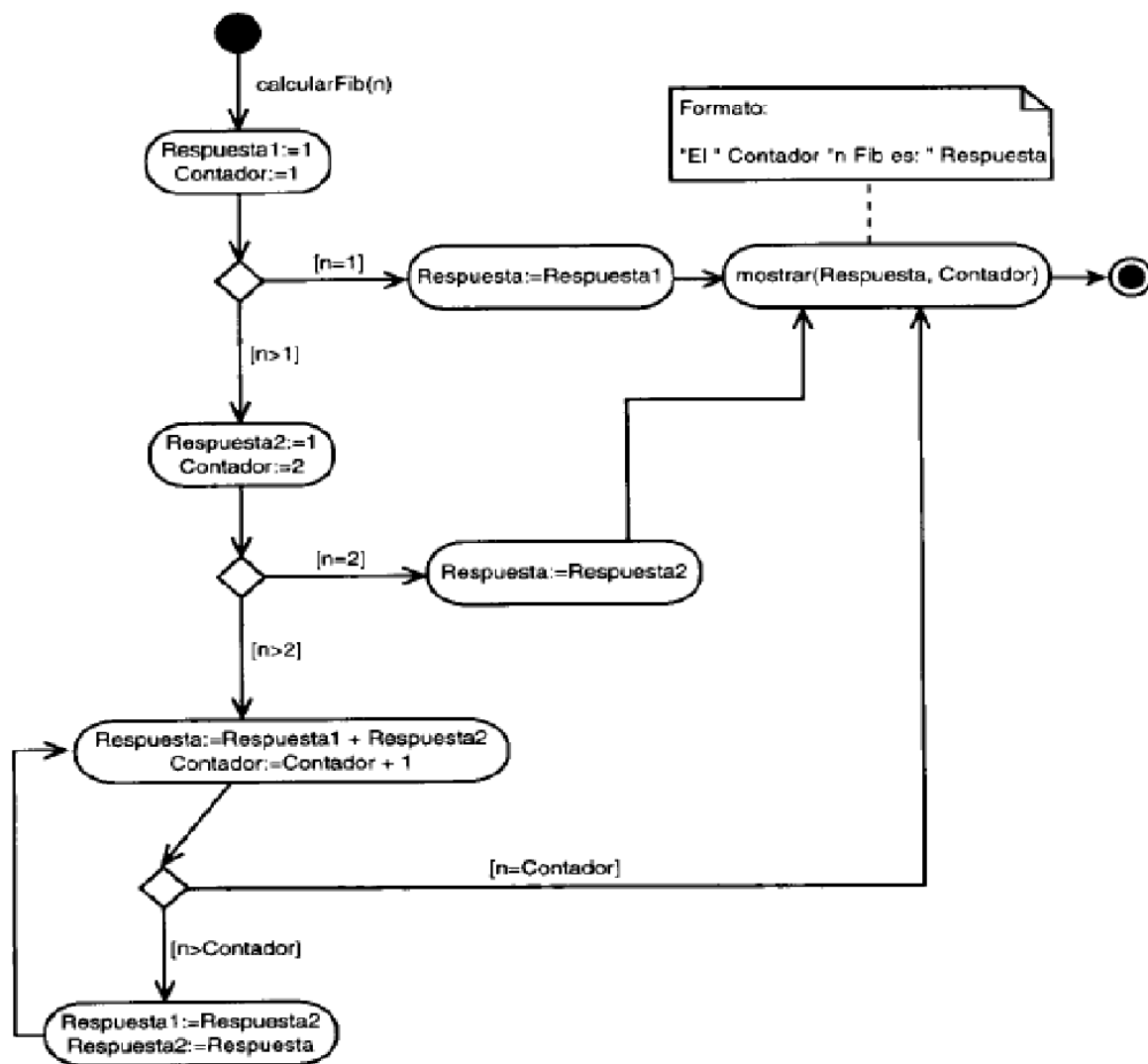


Diagrama de Actividades

Ejemplo Serie de Fibonacci



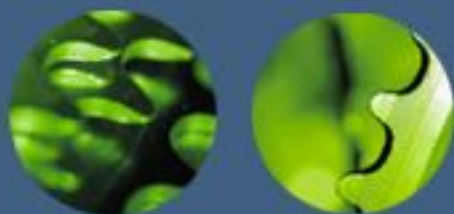


Diagrama de Actividades

Proceso de Creación de un Documento



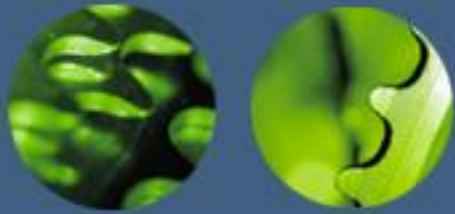


Diagrama de Actividades Híbrido

Proceso de Creación de un Documento

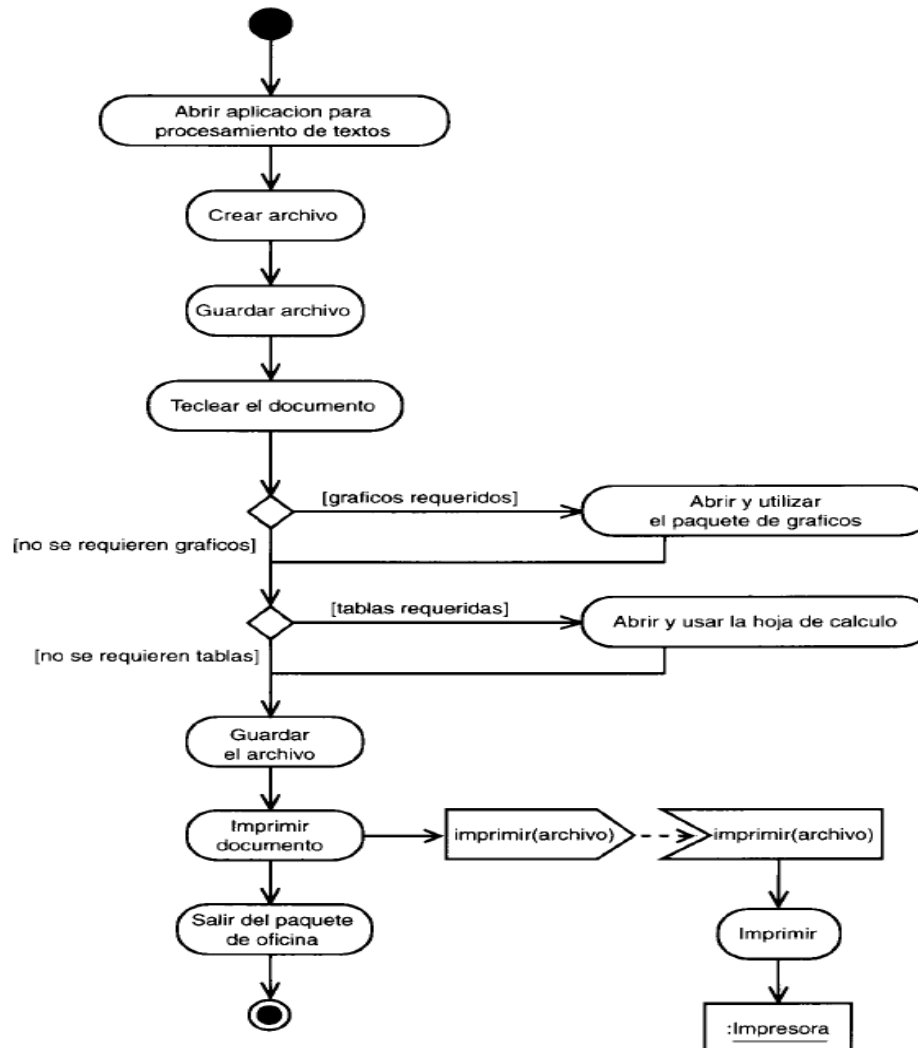
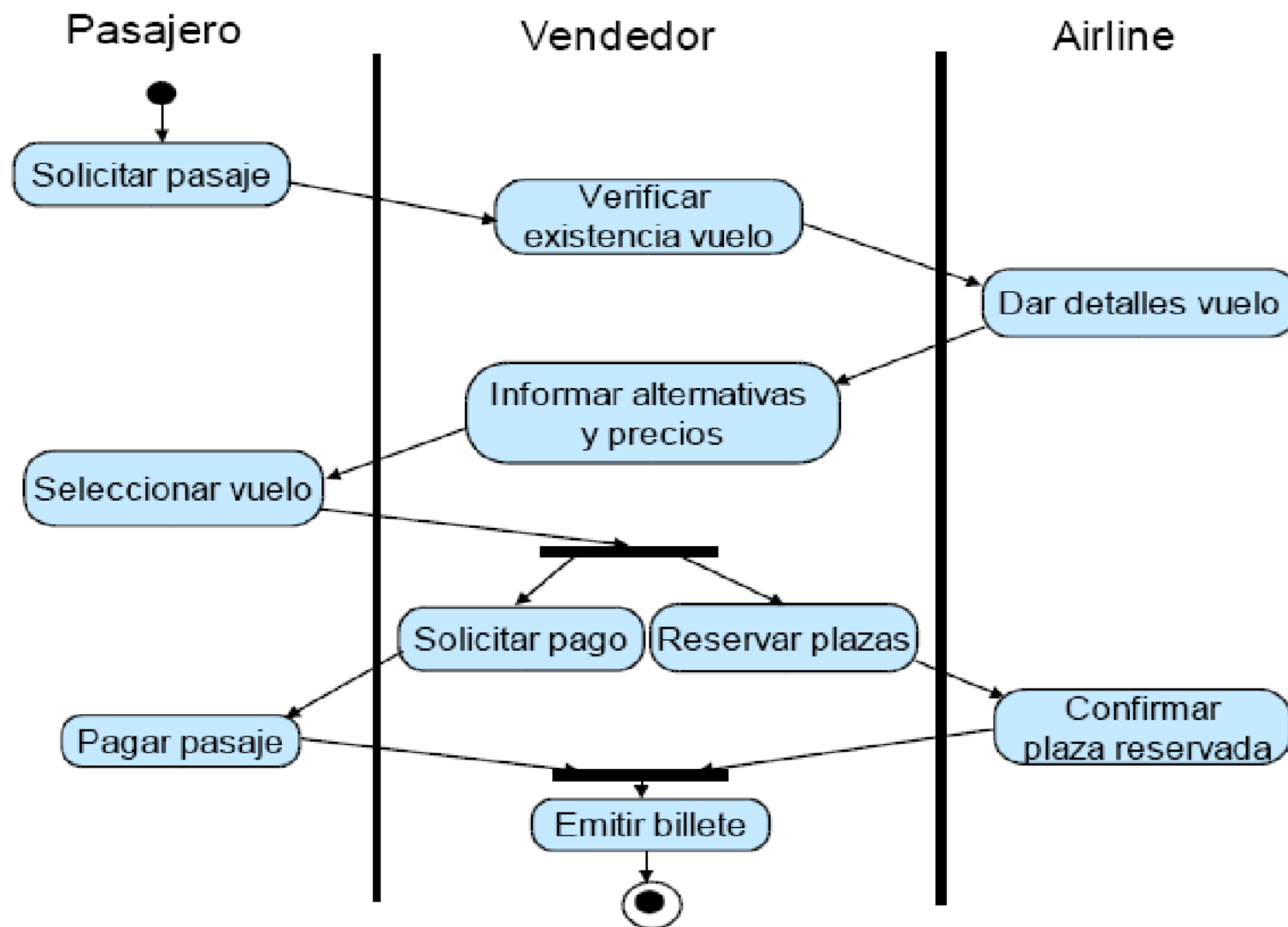
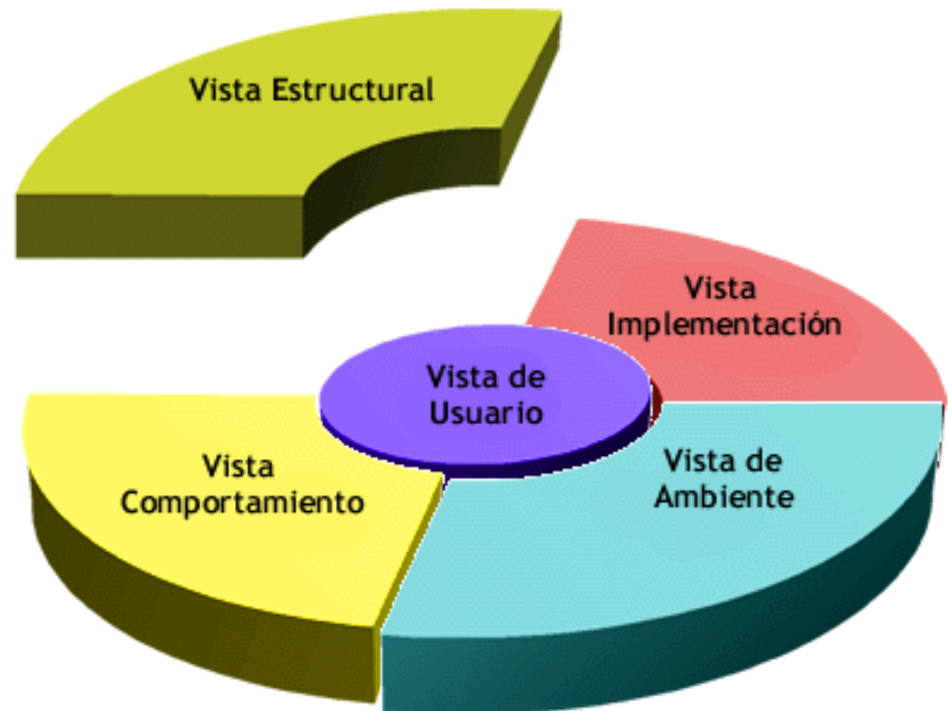
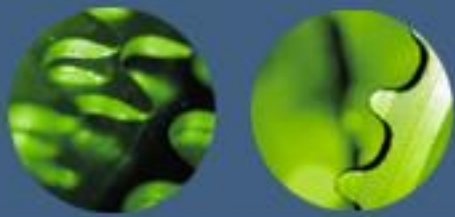




Diagrama de Actividades

Proceso de una Aerolínea con marcos de Responsabilidades





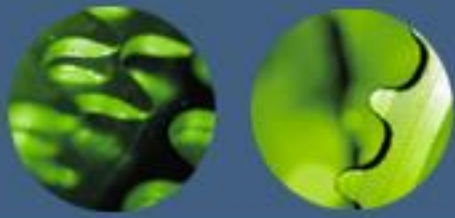


Diagrama de Clases

- El Diagrama de Clases es el diagrama principal para el análisis y diseño.
- Un diagrama de clases presenta las clases del sistema con sus relaciones estructurales y de herencia.
- La definición de clase incluye definiciones para atributos y operaciones.
- El modelo de casos de uso aporta información para establecer las clases, objetos, atributos y operaciones.
- Los diagramas de clases son utilizados para ilustrar las relaciones entre clases y son el fundamento para el proceso de diseño

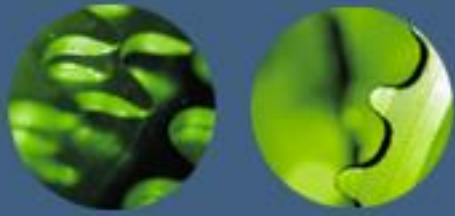


Diagrama de Clases

- Modela los conceptos del dominio de la aplicación.
- Un diagrama de clases esta compuesto por los siguientes elementos:
 - Clases: atributos, operaciones y visibilidad.
 - Relaciones: Herencia, Composición, Agregación, Asociación y Uso.
 - Responsabilidades



Pasos para dibujar un Diagrama de Clases

- Paso 1: Dibuje los Nodos de las Clases.
- Paso 2: Dibuje las Asociaciones.
- Paso 3: Coloque los Nombres y Roles de las Asociaciones.
- Paso 4: Coloque la Multiplicidad de las Asociaciones.
- Paso 5: Dibuje las flechas de navegación.
- Paso 6: Dibuje las Clases Asociadas (si existen).
- Paso 7: Validar el modelo del Dominio.

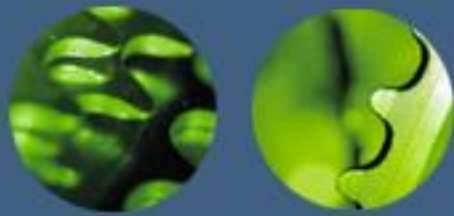
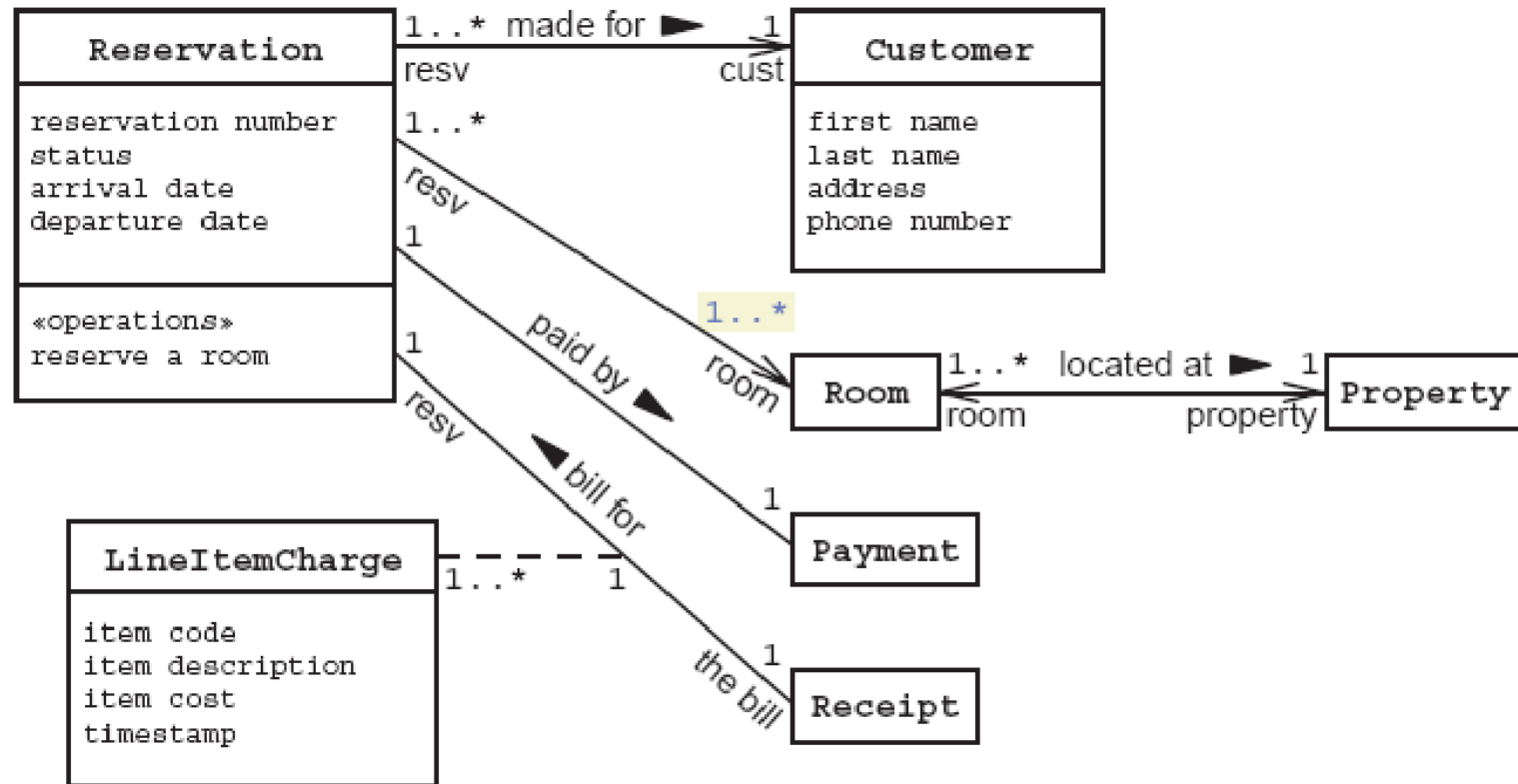
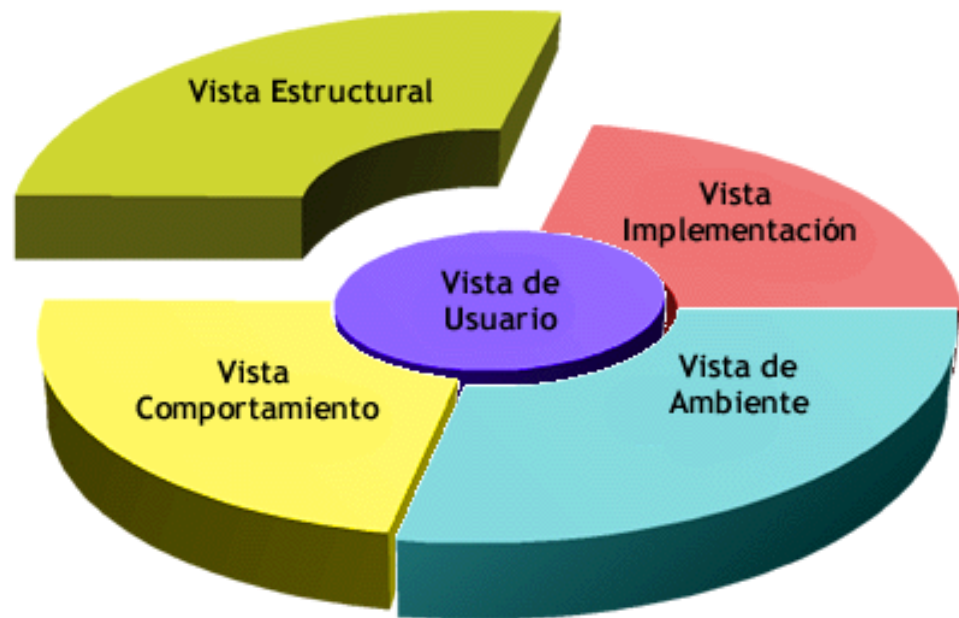
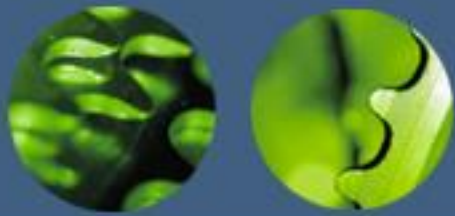


Diagrama de Clases





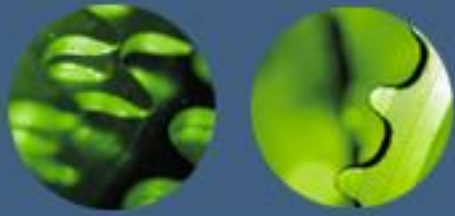


Diagrama de Objetos

- El Diagrama de Objetos es una instancia de un Diagrama de Clases y presenta los detalles de un estado del sistema en un punto del tiempo determinado. Se utilizan para validar el modelo del dominio.
- Para validar el modelo del dominio es necesario ejecutar los siguientes pasos:
 - Elegir uno o más casos de uso que estén altamente relacionados con el modelo del dominio.
 - Elegir uno o más escenarios de los casos de uso seleccionados en el punto anterior. Es recomendable elegir escenarios que exploren diferentes situaciones.

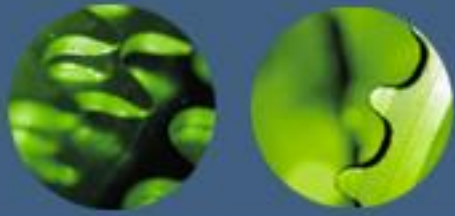


Diagrama de Objetos

- Ir a través de cada escenario en forma separada, y construir los objetos con los datos mencionados en el escenario.
- Comparar cada diagrama de objetos con el modelo del dominio para analizar si se han violado algunas restricciones.

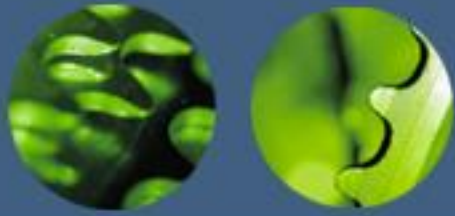
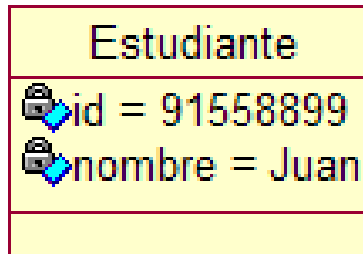


Diagrama de Objetos

Ejemplo Sistema Académico

Creando el diagrama de objetos desde el escenario: Juan ingresa su identificación 91558899 la cual el sistema valida.



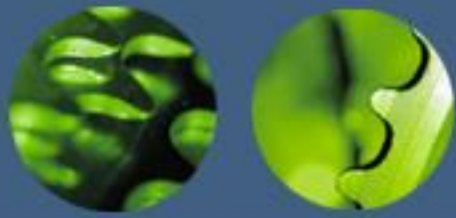
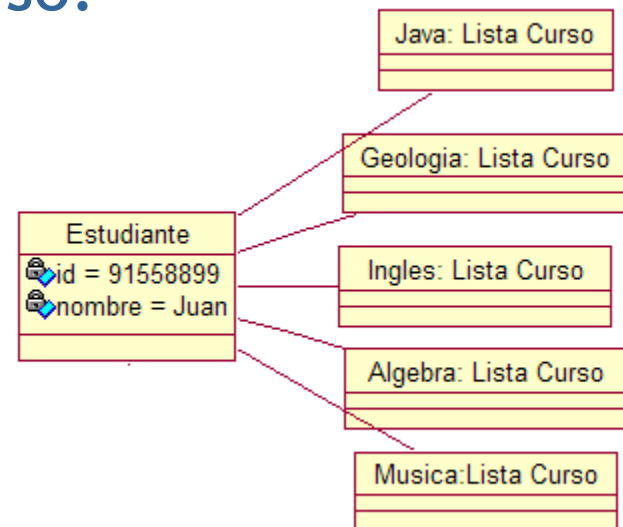


Diagrama de Objetos

Ejemplo Sistema Académico

De un catálogo de cursos disponibles, Juan selecciona como cursos principales Inglés, Geología, Historia y Algebra. También selecciona Música y Java como materias alternativas. El sistema determina que Juan cumple con los pre-requisitos necesarios y lo agrega a la lista de estudiantes de ese curso.



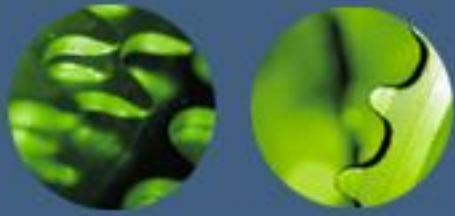
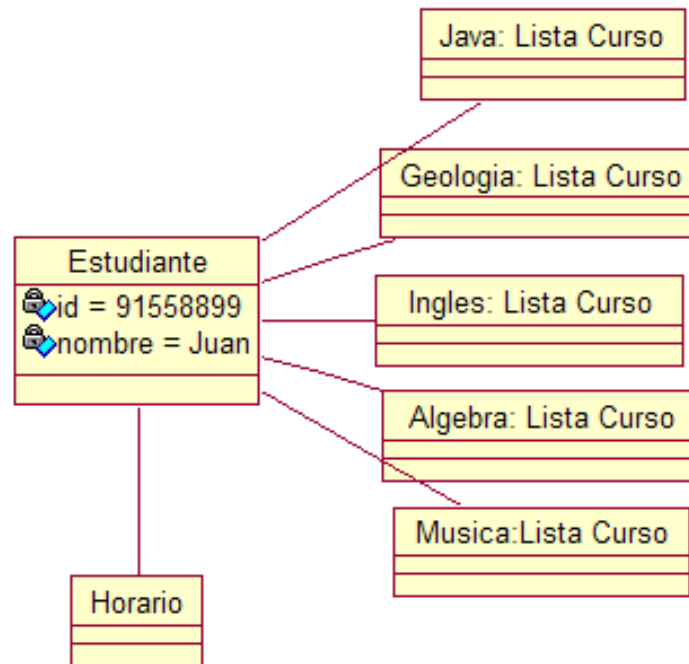
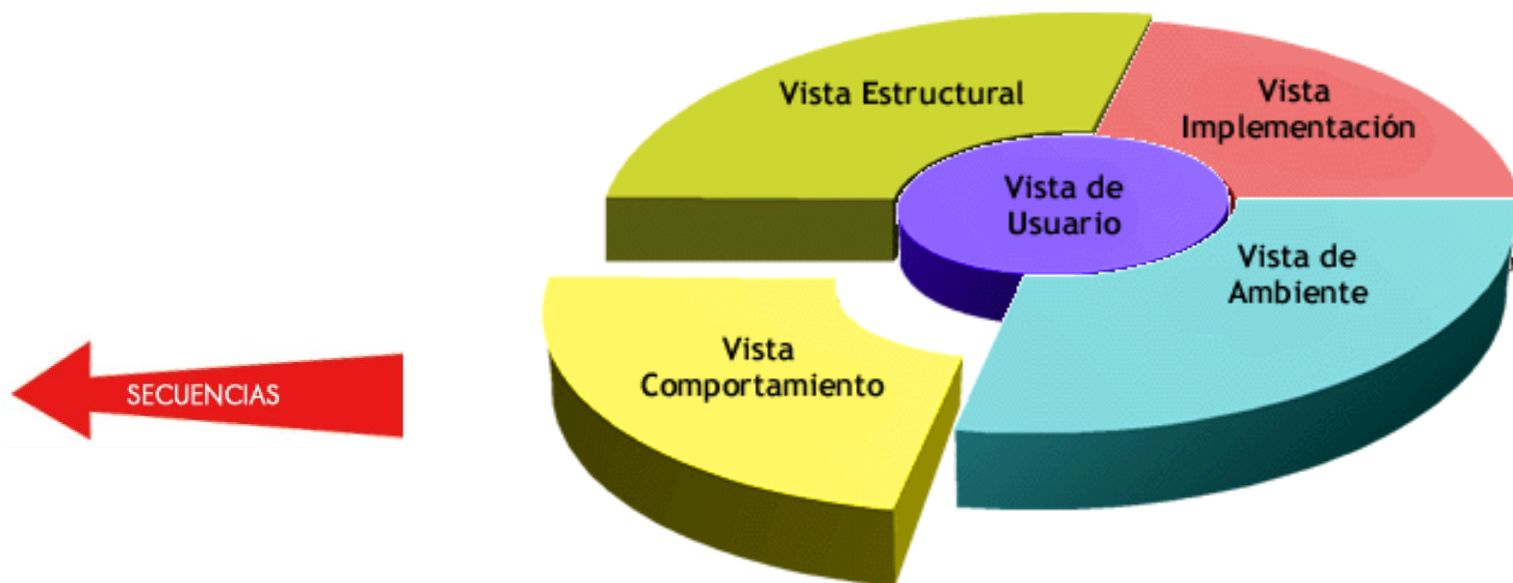
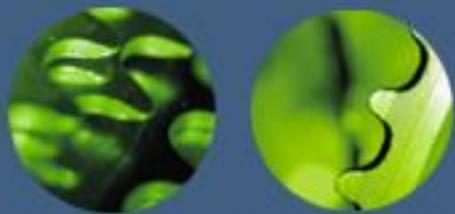


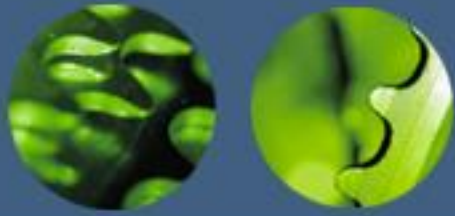
Diagrama de Objetos

Ejemplo Sistema Académico

El sistema indica que la actividad se ha completado, imprime el horario del estudiante y le envía la información correspondiente al sistema financiero.

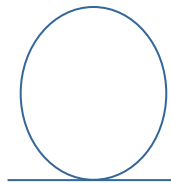




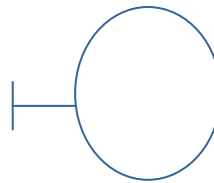


Tipos de Clases

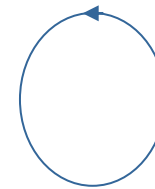
Cada Clase en UML tiene su propia notación.



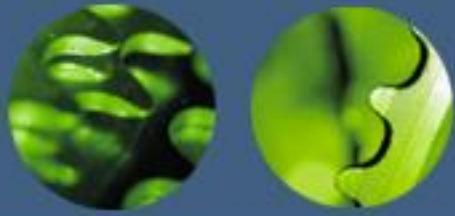
**Clase
Entidad**



**Clase
Interfaz**



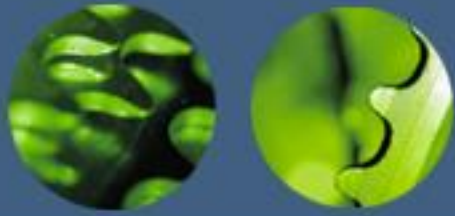
**Clase
Control
(Servicio)**



Tipos de Clases

Clase de Entidad

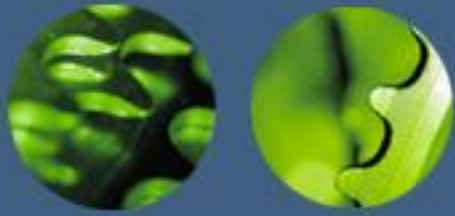
- Representa la información que va a ser persistente.
 - Para ser utilizada en tareas internas del sistema.
 - Su comportamiento es independiente
 - El valor de sus atributos generalmente es proporcionado por un actor.



Tipos de Clases

Clase de Límite (Interfaz)

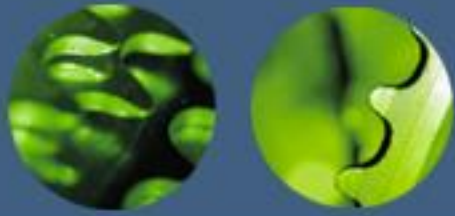
- Modelan la comunicación entre los límites del sistema y sus entradas de trabajo: formas, ventanas de diálogo, protocolos de comunicación, dispositivos.
- También usadas para la comunicación entre otros sistemas.



Tipos de Clases

Clase de Control (Servicio)

- Modela el comportamiento específico de uno o más casos de uso.
- Una clase de control:
 - Crea, inicializa y elimina objetos controlados.
 - Controla la secuencia o coordinación de ejecución de los objetos controlados.
 - Es la implementación de un objeto intangible.



Interacción entre Objetos

- **Diagramas de Secuencia:** interacción a través del tiempo
- **Diagramas de Colaboración:** encadenamiento entre objetos.

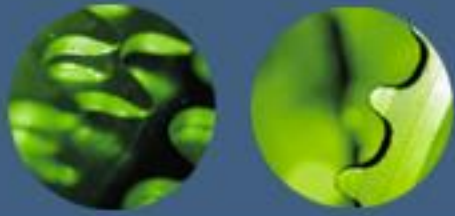


Diagrama de Secuencia

- Representa los mensajes intercambiados por un conjunto de objetos durante un escenario
- Consta de Actores, Objetos o Clases, mensajes y tiempo, donde se enfocan en los diferentes estados de un Objeto.

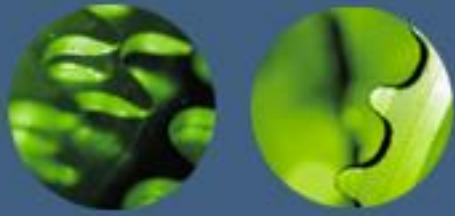


Diagrama de Secuencia

- Los Mensajes es la comunicación existente entre un Objeto a otro.
- Los mensajes pueden ser:
 - Simple: es la transferencia normal del control entre un Objeto a otro.
 - Sincrónico: Es la espera la respuesta de un mensaje antes de continuar con su trabajo.
 - Asincrónico: no espera respuesta de un mensaje para continuar con su trabajo.



Simple



Sincrónico



Asincrónico

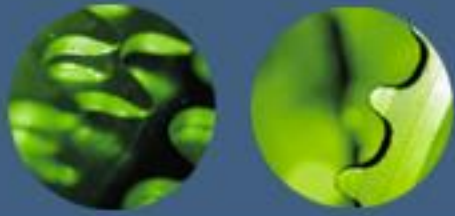


Diagrama de Secuencia

- El Tiempo representa la duración de la ejecución de un mensaje.
- Se representa con una barra vertical.
- Puede mostrar los Estados de un Objeto.
- En ocasiones un objeto cuenta con una operación que se invoca así misma, esto se llama “*Recursividad*”.

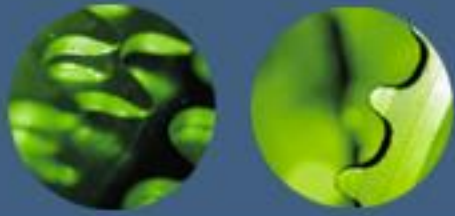


Diagrama de Secuencia

Los pasos para elaborar este tipo de diagramas son:

- Seleccione un caso de uso
- Coloque el actor en el diagrama
- Identifique las clases de interfaz
- Identifique las clases de control
- Identifique las clases de entidad

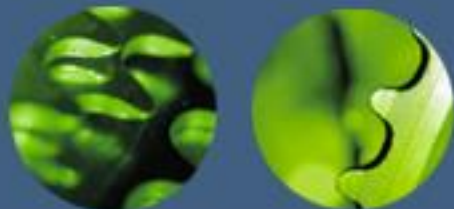
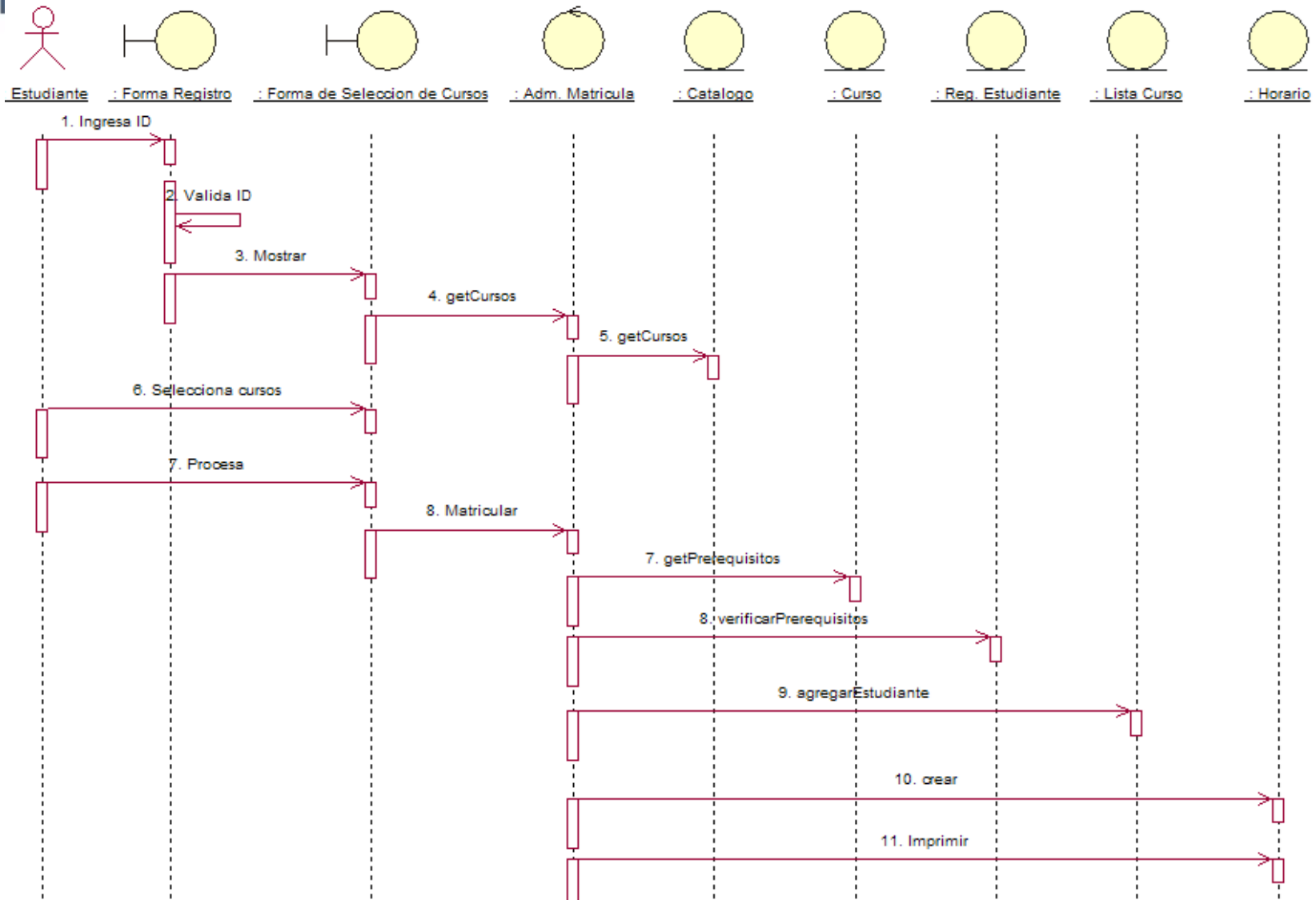
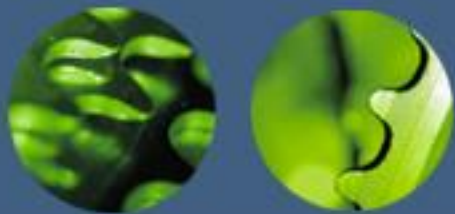


Diagrama de Secuencia

Ejemplo Caso de Uso Matricular





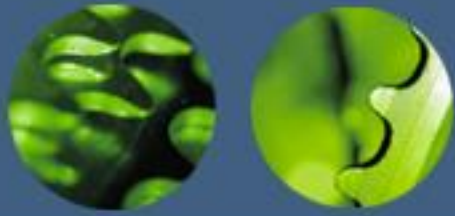


Diagrama de Colaboración

- Este Diagrama es Similar al Diagrama de Secuencia, pero de una mirada diferente.
- Es la forma de cómo los Objetos se colaboran entre si, tal como se muestra en el Diagrama de Secuencia.
- Destaca la organización de los Objetos que participan en una interacción y sus relaciones.

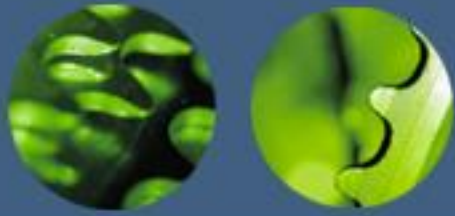


Diagrama de Colaboración

- Cuenta con dos características que lo diferencia del Diagrama de Secuencia:
 - **El Camino:** Indica como se enlaza entre un Objeto a otro.
 - **Numero de Secuencia:** Indica la ordenación temporal de un mensaje, se precede de un número y que incrementa secuencialmente por cada nuevo mensaje en el flujo de control. También se cuenta la representación por anidamiento, utilizando la numeración decimal de Dewey.



Diferencias entre el Diagrama de Secuencia y Colaboración



- El Diagrama de Secuencia muestra la sucesión de las interacciones y el de Colaboración destacan el Contexto y la Organización general de los Objetos que interactúan.
- El Diagrama de Secuencia se organiza de acuerdo al tiempo y el de Colaboración de acuerdo al espacio.

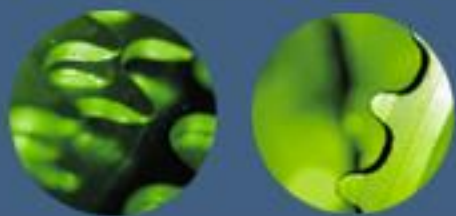
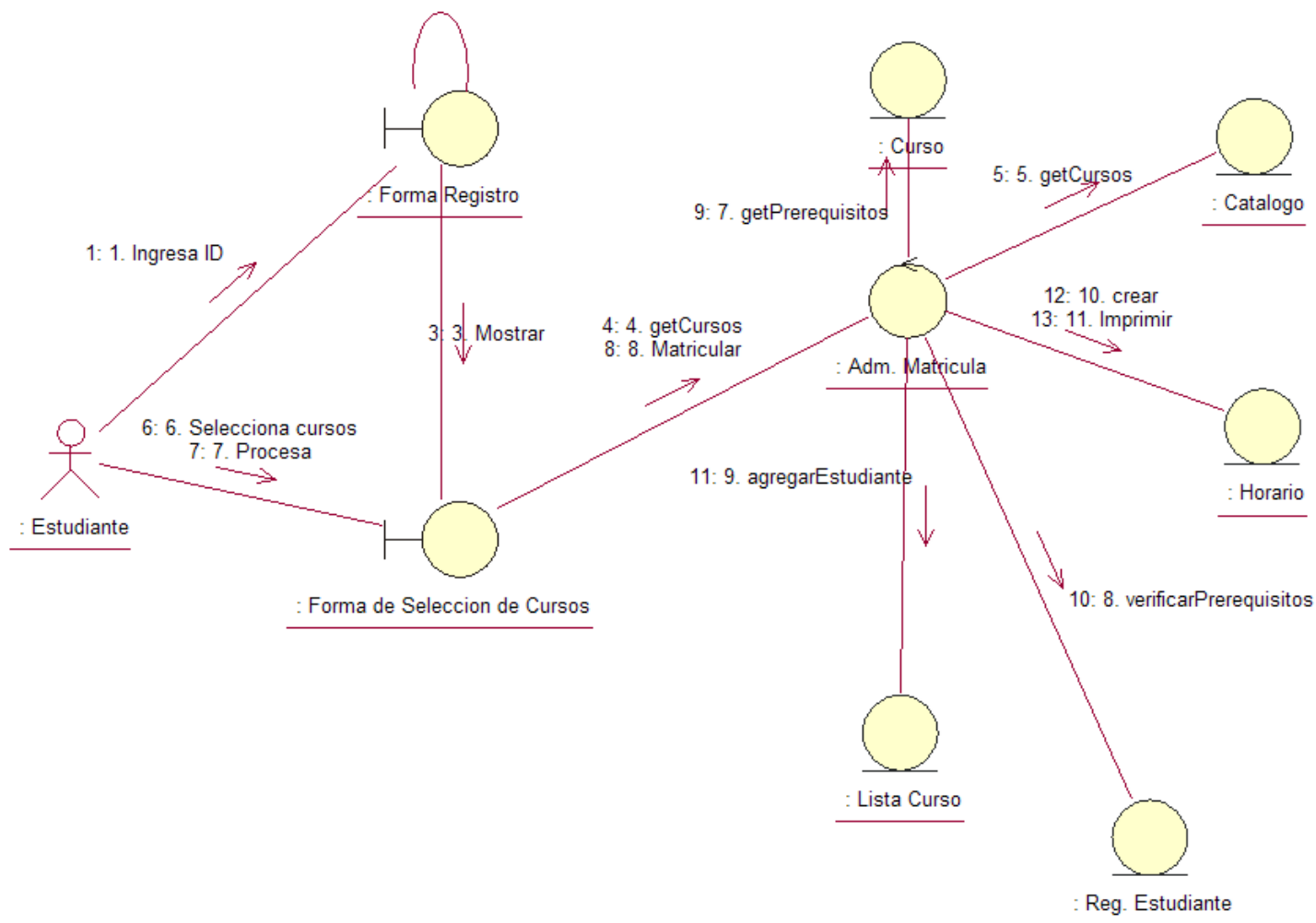
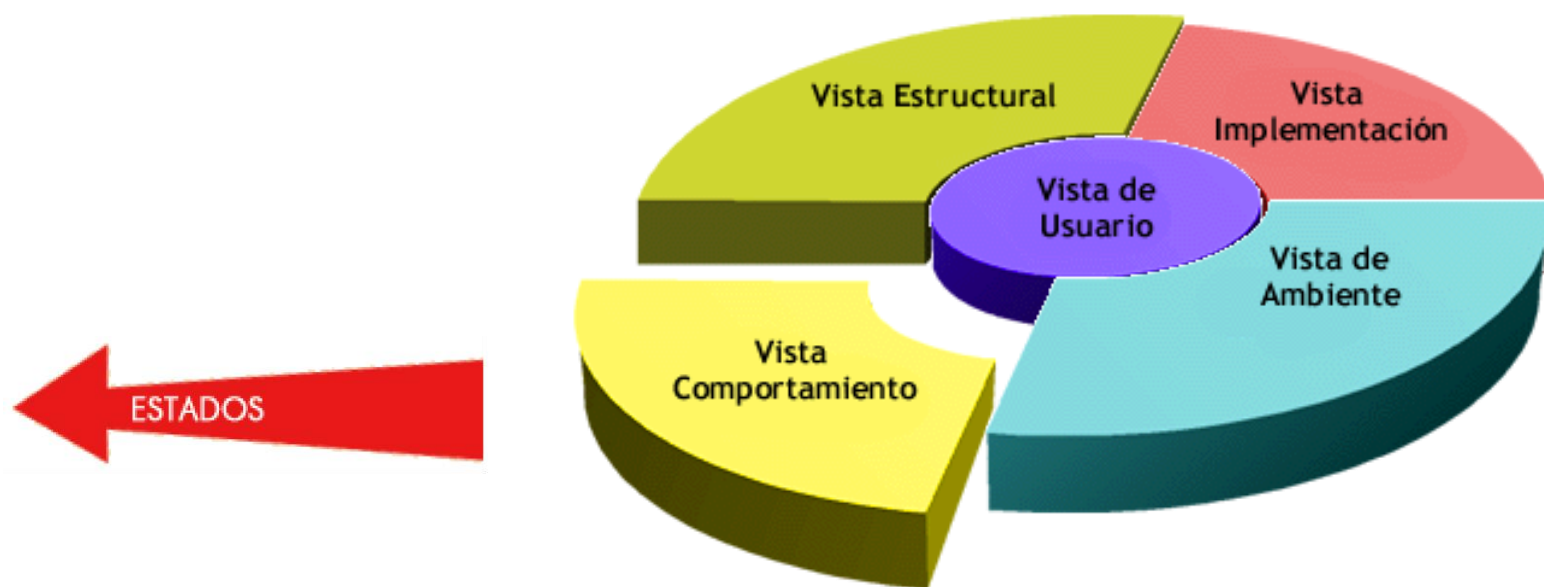
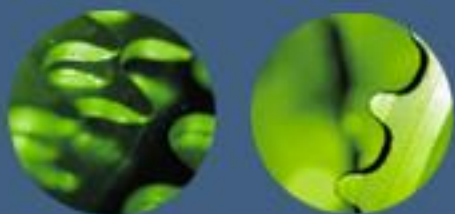


Diagrama de Colaboración

Ejemplo Caso de Uso Matricular





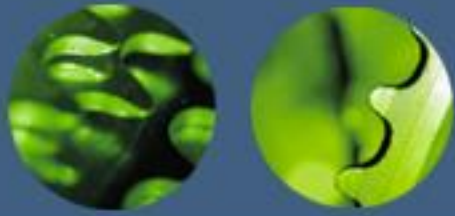


Diagrama de Estado

- Muestra el conjunto de estados por los cuales pasa un objeto durante su vida en una aplicación, junto con los cambios que permiten pasar de un estado a otro.
- Presenta los Estados que puede encontrarse un Objeto junto con las transiciones entre los estados, y muestro los puntos inicial y final de una secuencia de cambios de estados.
- Un Diagrama de Estado también se le conoce como un Motor de Estado.
- Un Estado de Acción se puede ver como un caso especial de un estado de actividad.

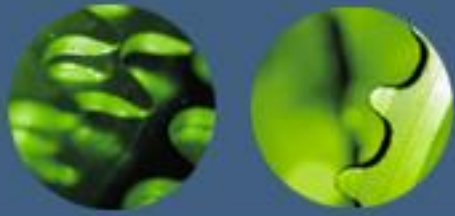


Diagrama de Estado

- También se conoce como Diagrama de Transición.
- Es usado para mostrar la vida de una clase determinada a través de todo el sistema, los eventos causan una transición de un estado a otro, y las acciones que resultan del cambio de estado.
- Un estado de un objeto es una de las posibles condiciones en las cuales puede existir.

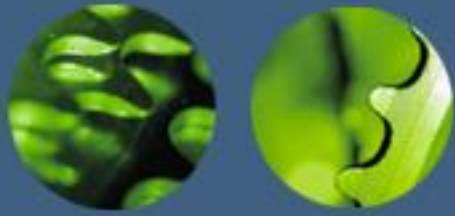


Diagrama de Estado

- Los Elementos de una Estado son:
 - **Estado:** Es una condición o situación en la vida de un objeto durante la cual se satisface alguna condición, realiza alguna actividad o espera algún evento.
 - **Evento.** Es la especificación de un acontecimiento significativo que ocupa un lugar en el espacio y en el tiempo.
 - **Transición.** Es la relación entre dos estados, en la que se indica cómo se pasa de uno a otro.

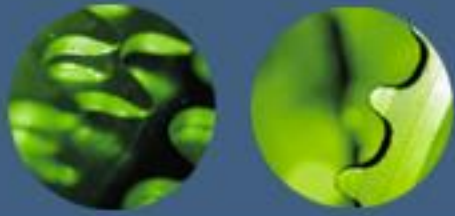


Diagrama de Estado

- **Actividad.** Ejecución atómica en curso dentro de una máquina de estado.
- **Acción.** Computación atómica ejecutable que produce un cambio de estado en el modelo o devuelve un valor.
 - Cuando se crea un objeto, se entra en un estado inicial y cuando se destruye, se llega a un estado inicial.
- **Acciones:** De entrada, salida y durante la actividad.



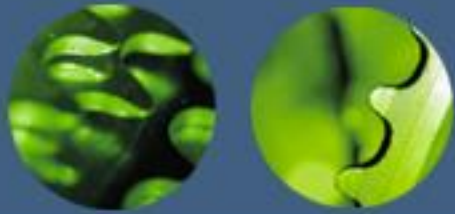
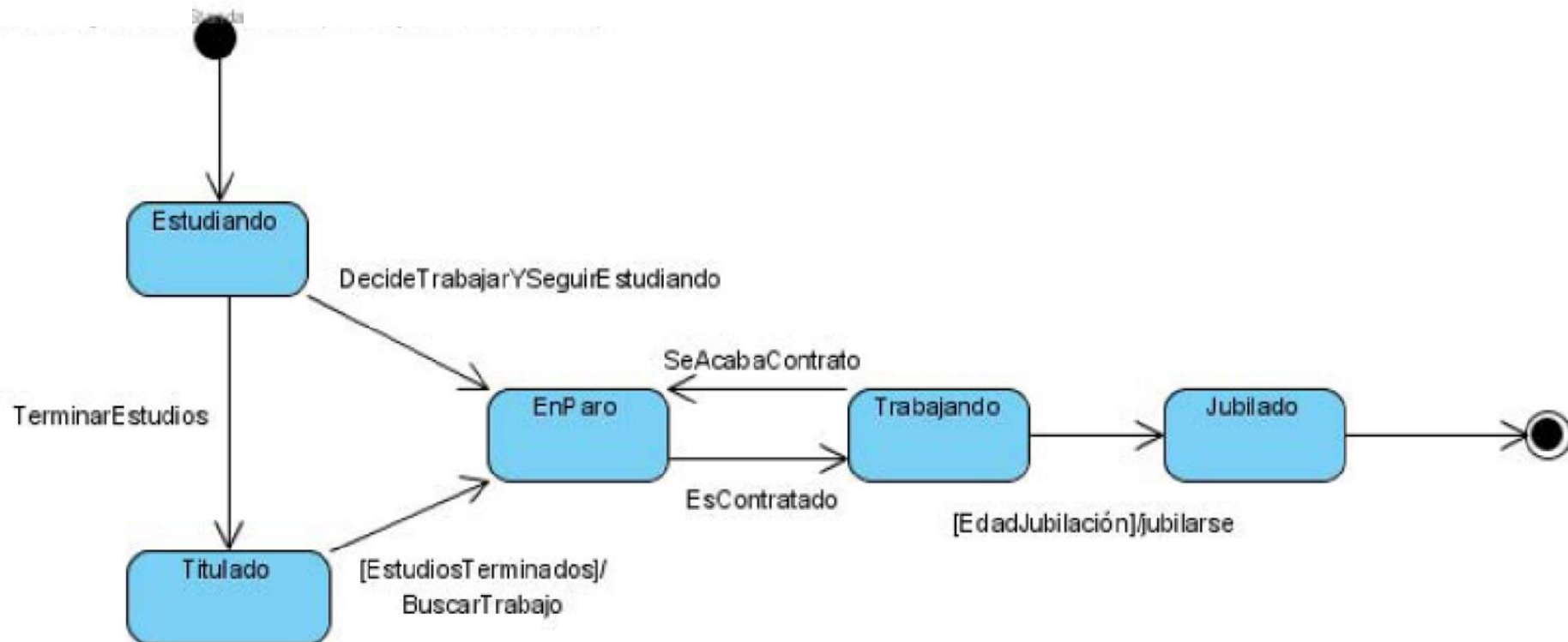


Diagrama de Estado

Ejemplo para el Objeto Empleado:



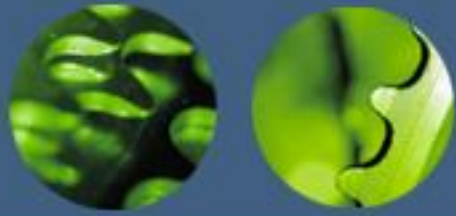
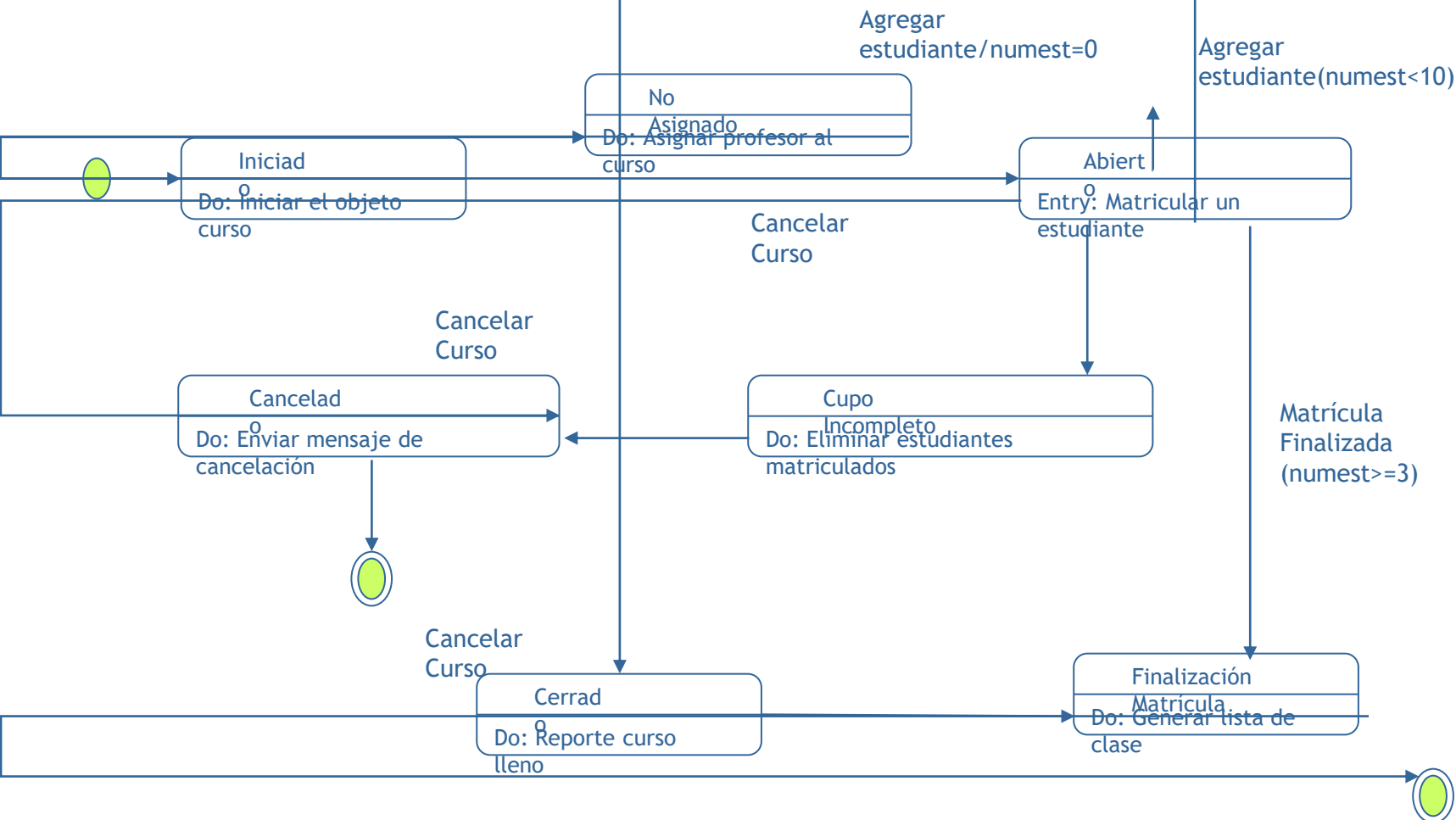


Diagrama de Estado

Ejemplo para la Clase Curso:



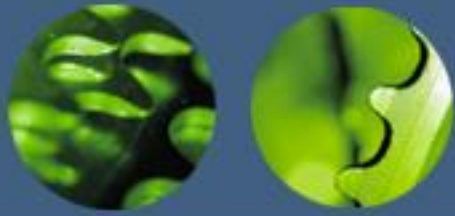
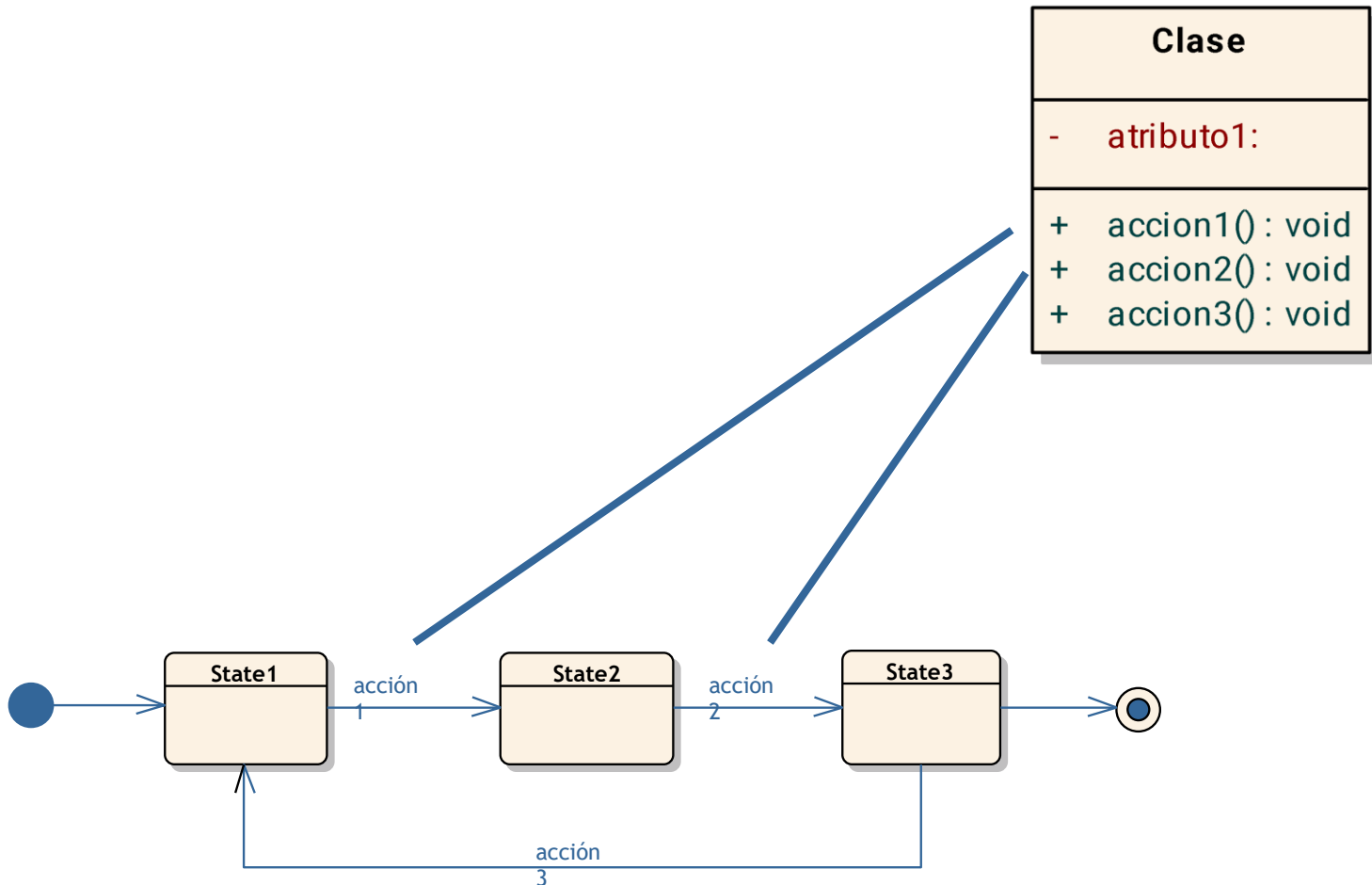


Diagrama de Estado

Interpretación para la Clase Curso:



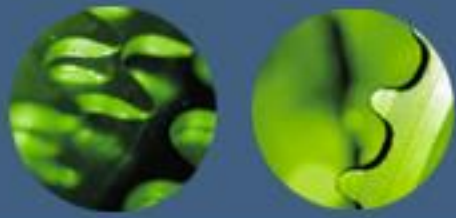
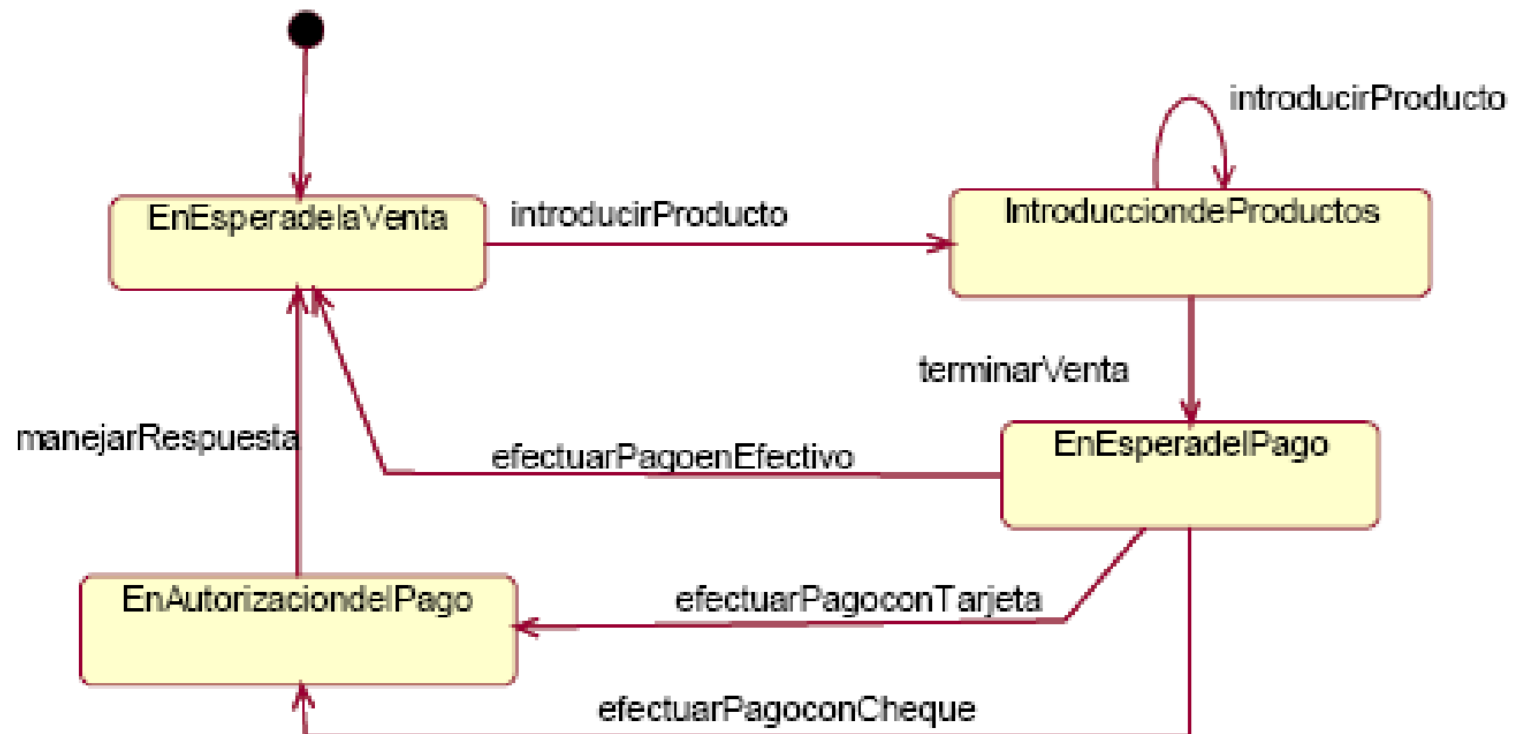


Diagrama de Estado

Ejemplo para el una Caso de Uso Comprar Productos:



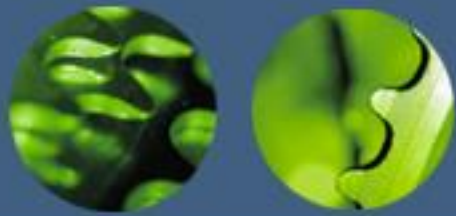
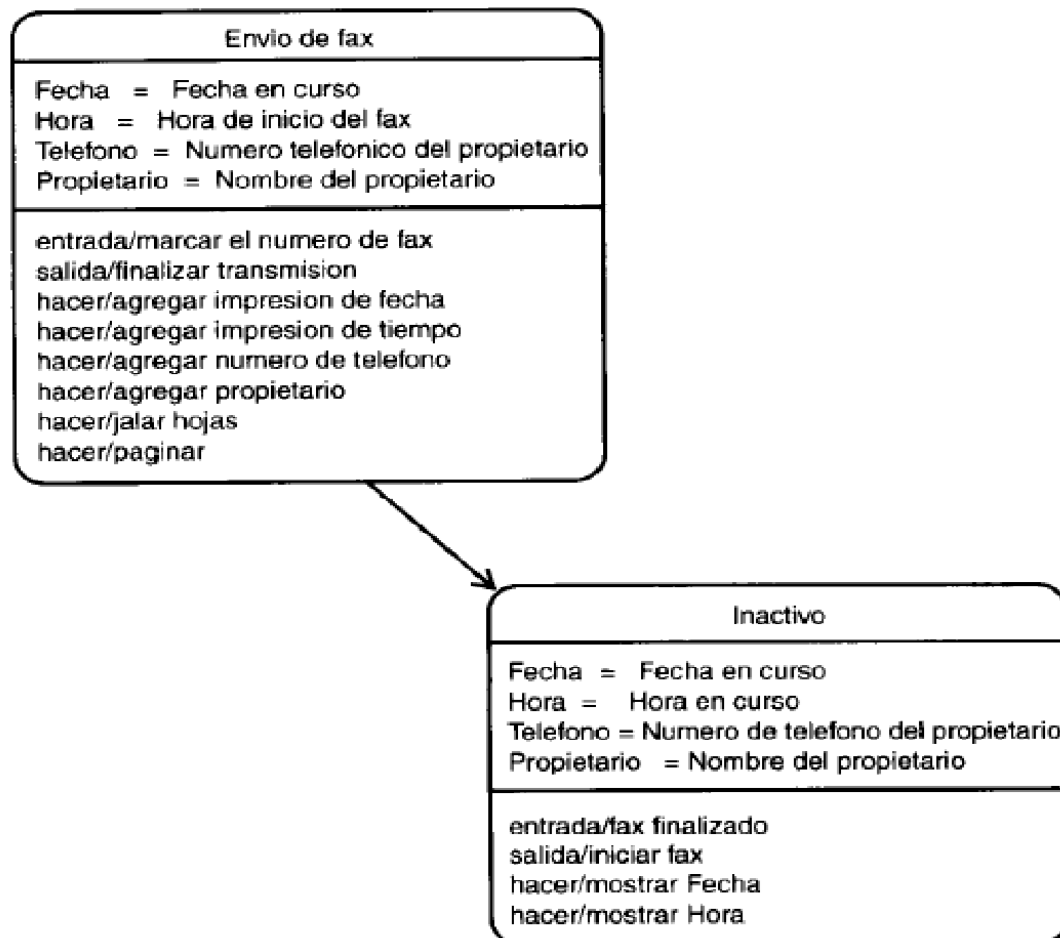


Diagrama de Estado

Ejemplo Maquina de Fax:



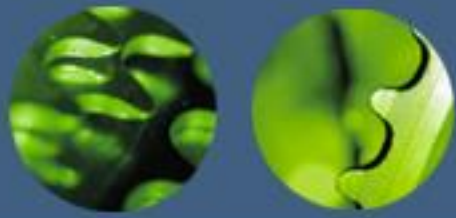
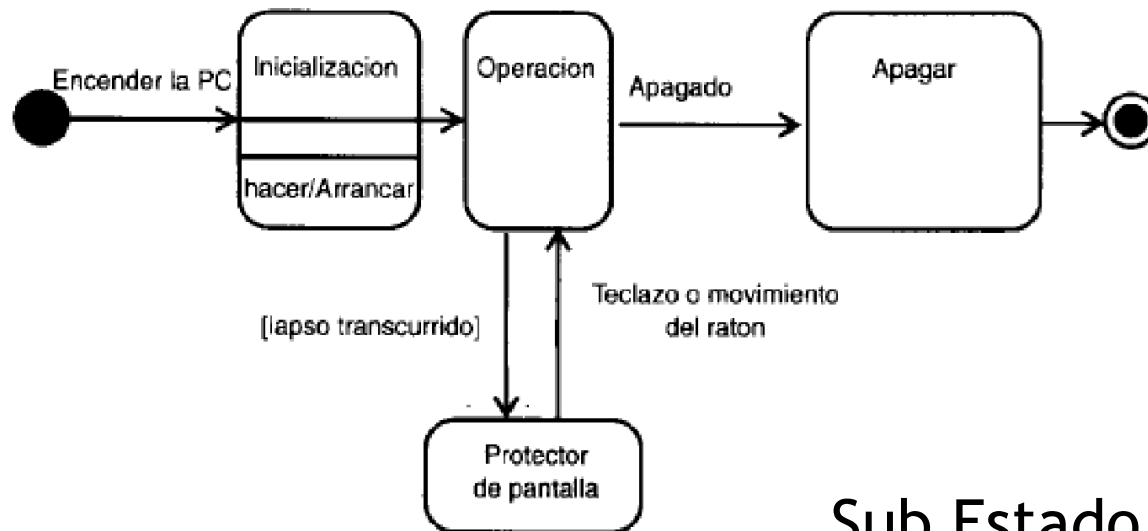


Diagrama de Estado

Ejemplo Protector de Pantalla:



Sub Estado del proceso Operación



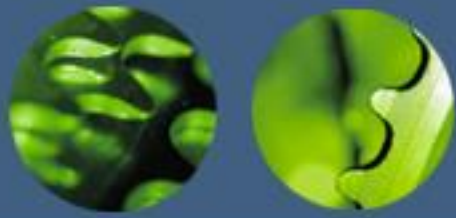
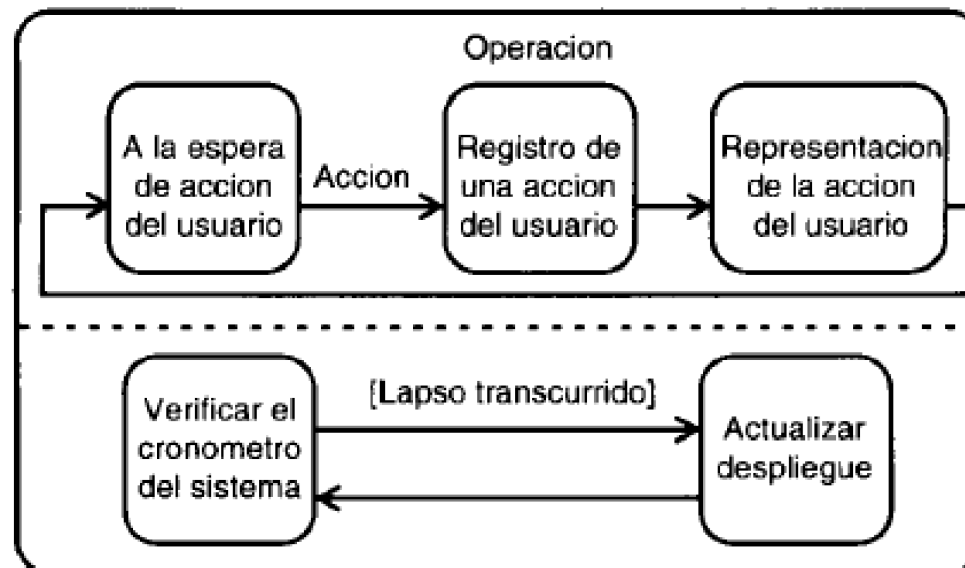
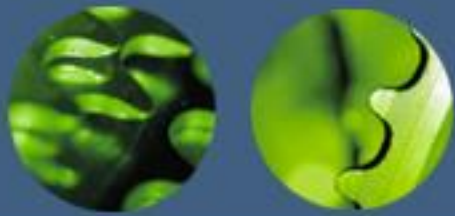


Diagrama de Estado

Ejemplo Protector de Pantalla:

Sub Estado Concurrente del proceso Operación





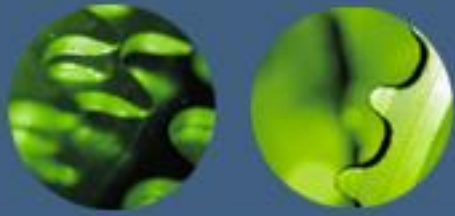


Diagrama de Componentes

- Un Componente de Software es una parte física de un Sistema y se encuentra en la Computadora y no en la mente del Analista.
- Se puede tomar como Componente: tabla, archivo de datos, html, ejecutable, biblioteca de vínculos dinámicos, documentos, etc.

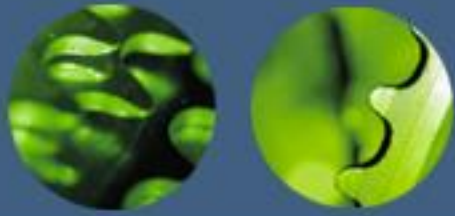


Diagrama de Componentes

- Los Diagramas de Componentes se utilizan para:
 - Los Clientes puedan ver la estructura del Sistema finalizado.
 - Los Desarrolladores cuenten con una estructura con la cual trabajar en adelante.
 - Quienes escriban las notas técnicas y la documentación puedan entender lo que escriben.
 - Ustedes se alisten para volver a utilizar los Componentes.

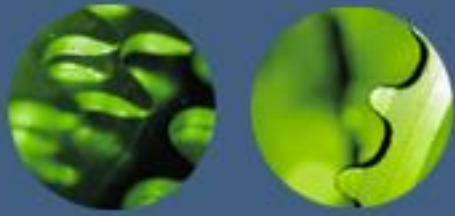


Diagrama de Componentes

- Los Diagramas de Componentes se utilizan para:
 - Modelar Código Fuentes.
 - Modelar Versiones Ejecutables.
 - Modelar Base de Datos Físicas.
 - Modelar Sistemas Adaptables.
- Los componentes representan todos los tipos de elementos software que entran en la Fabricación de aplicaciones informáticas.

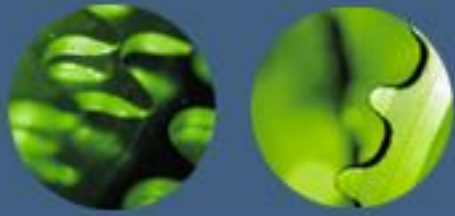
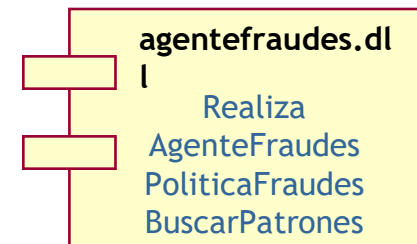
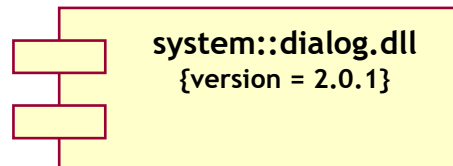
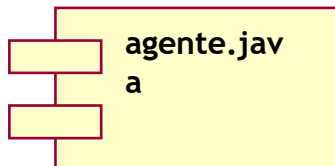
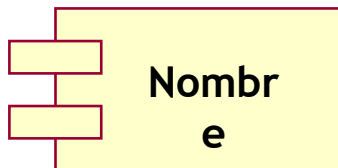


Diagrama de Componentes

- Muestra la organización y las Dependencias entre un conjunto de Componentes.
- Cubren la vista de la Implementación Estática y se relacionan con los Diagramas de Clases ya que en un Componente suele tener una o mas Clases, interfaces o Colaboraciones.
- Cuando se habla del Diagrama de Componentes, se trata obviamente de, Componentes, Interfaces y Relaciones.



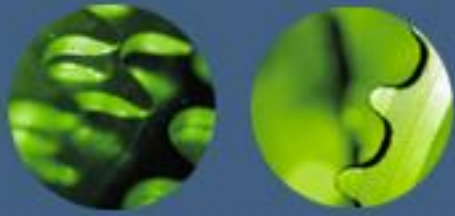
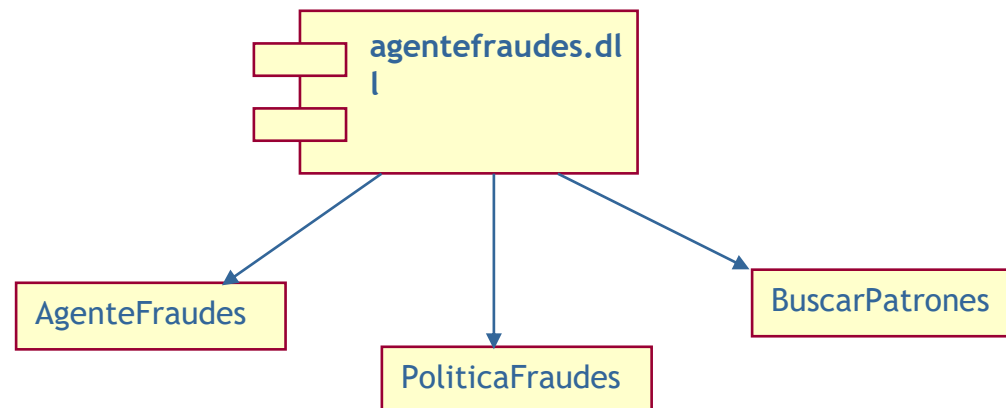


Diagrama de Componentes

Componentes y Clases

Las clases representan abstracciones lógicas. Los componentes son elementos físicos del mundo real. Un componente es la implementación física de un conjunto de otros elementos lógicos, como clases y colaboraciones.



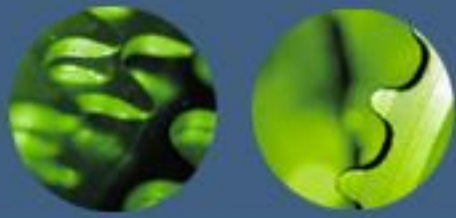


Diagrama de Componentes

Componentes y Clases

UML definen cinco Estereotipos estándar que se aplican a los Componentes:

- **Executable:** Especifica un componente que se puede ejecutar en un Nodo.
- **Library:** Especifica una biblioteca de Objetos Estática o Dinámica.
- **Table:** Especifica un Componente que representa una tabla de una Base de Datos.
- **File:** Especifica un Componente que representa un Archivo de Código Fuente o Archivo de Datos.
- **Document:** Especifica un Componente que representa un documento.

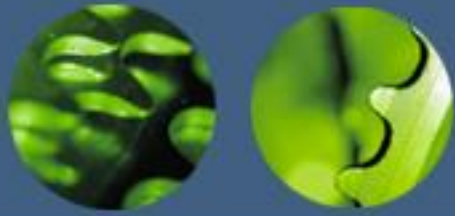
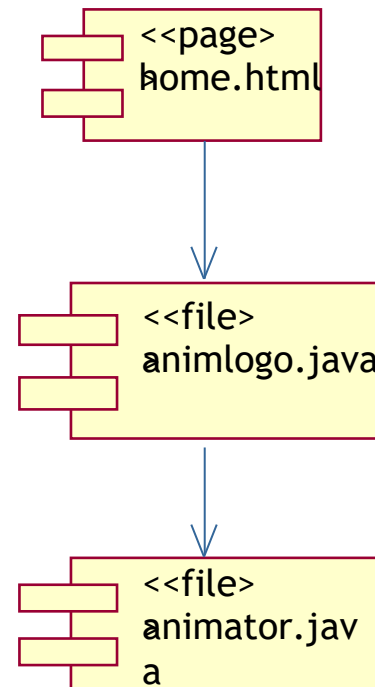


Diagrama de Componentes

Dependencias entre Componentes

La *dependencia* entre dos componentes se muestra como una flecha punteada. La *dependencia* quiere decir que una componente necesita de la otra para completar su definición, ósea, los Servicios ofrecidos por otro Componente .



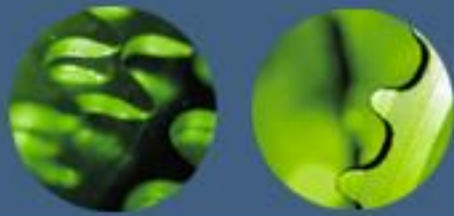
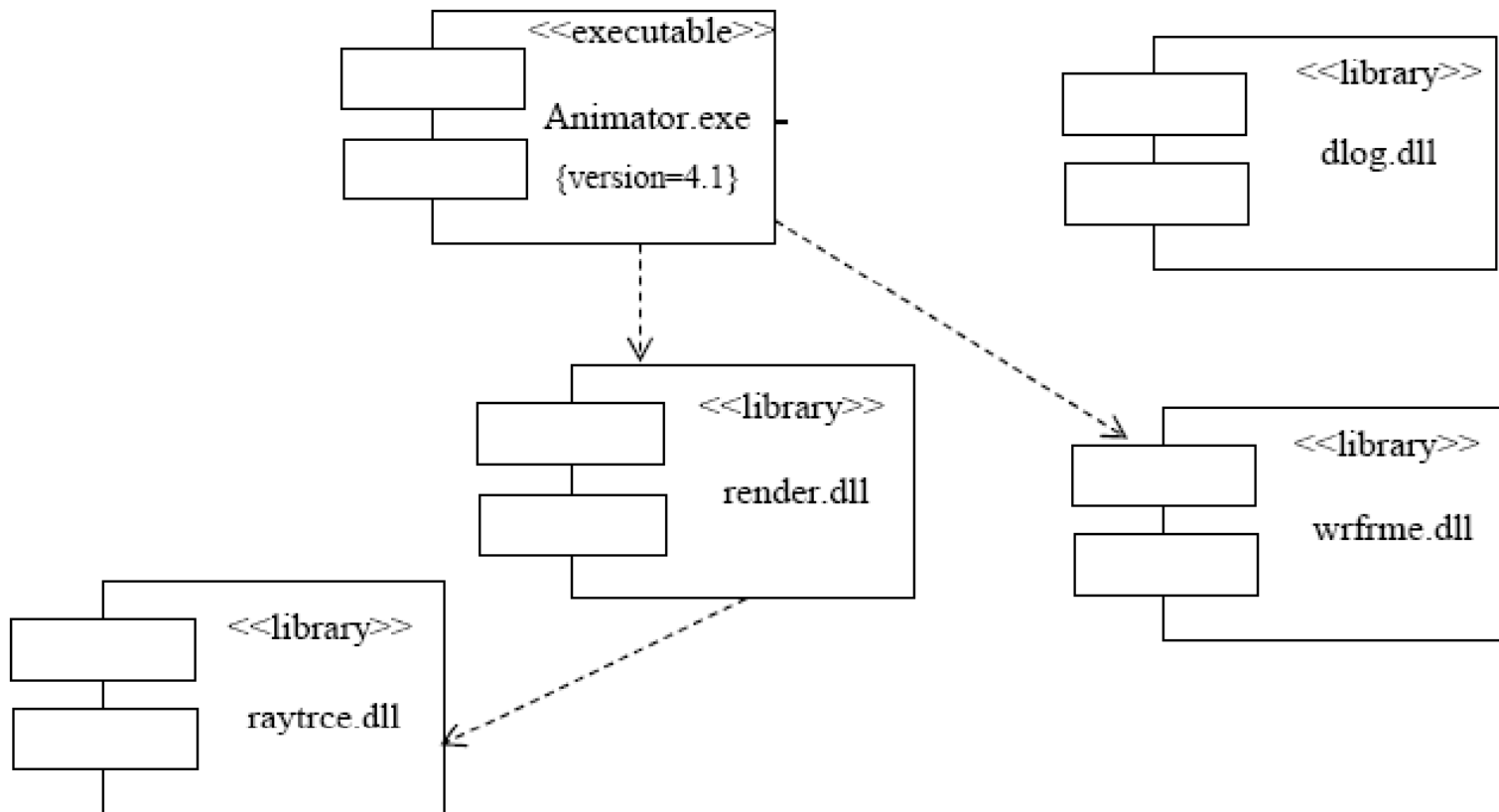


Diagrama de Componentes

Ejemplo



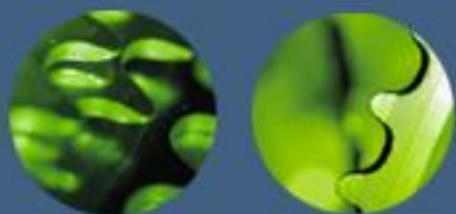
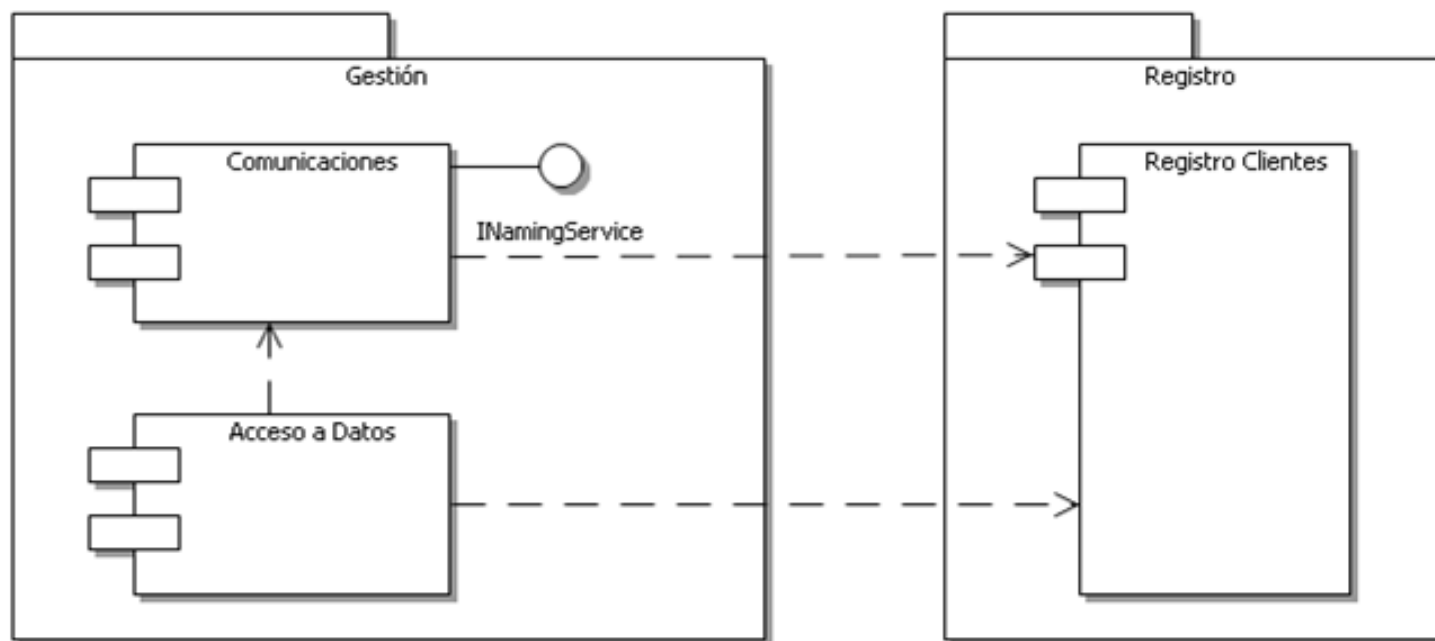
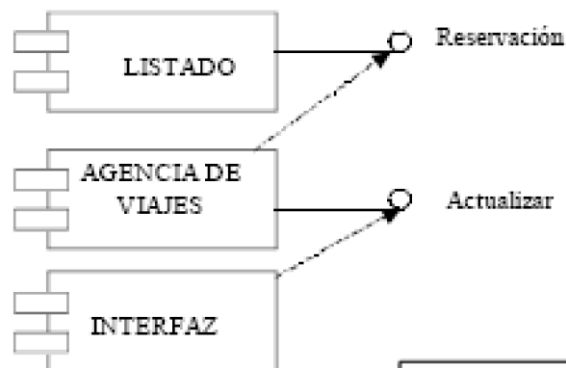


Diagrama de Componentes

Ejemplo



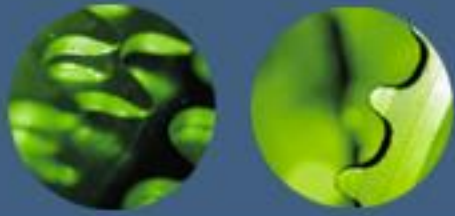


Diagrama de Componentes

Sub Sistemas

- Los distintos componentes pueden agruparse en paquetes según un criterio lógico y con vistas a simplificar la implementación.
- Son paquetes estereotipados en `<<subsistemas>>` para incorporar la noción de biblioteca de compilación.

```
<<subsistema>>  
NewPackage4
```

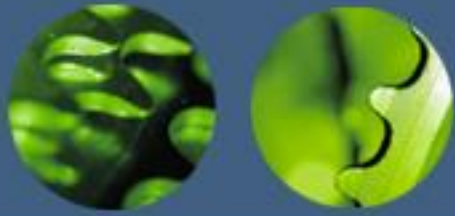
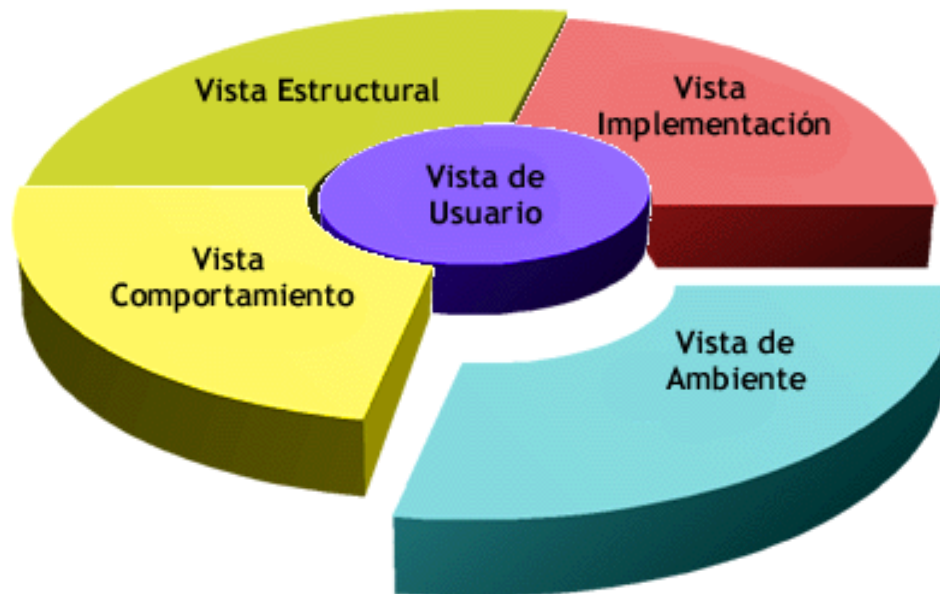
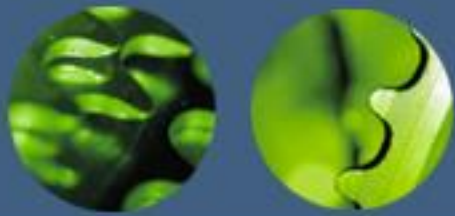


Diagrama de Componentes

Sub Sistemas

- Los subsistemas organizan la vista de realización de un sistema.
- Cada subsistema puede contener componentes y otros subsistemas.
- La descomposición en subsistemas no es una descomposición funcional.
- La relación entre paquetes y clases en el nivel lógico es el que existe entre subsistemas y componentes en el nivel físico.



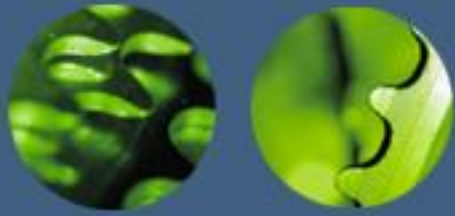


Diagrama de Despliegue o Distribución

- Los Diagramas de Despliegue o Distribución muestran la disposición física de los distintos nodos que componen un sistema y el reparto de los componentes sobre dichos nodos.
- Los Diagramas de Despliegue o Distribución modelan la topología del hardware sobre el que se ejecuta el Sistema Software.
- Este tipo de diagramas suele utilizarse para modelar Sistemas Distribuidos o Sistemas Empotrados. En los sistemas monolíticos, generalmente, resultan innecesarios.

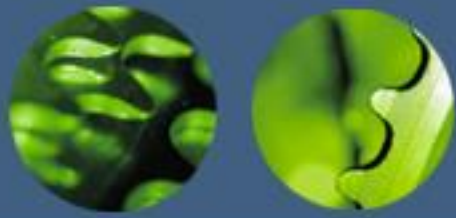


Diagrama de Despliegue o Distribución

- Representa los Dispositivos y Equipos, mostrar sus interconexiones y el Software que se encuentra en cada maquina.
- Modela la distribución en tiempo de ejecución de los elementos de procesamiento y componentes de software, junto a los procesos y objetos asociados.

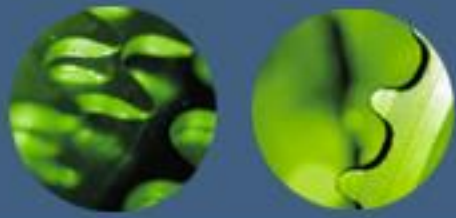


Diagrama de Despliegue o Distribución

- Un nodo es un recurso de ejecución, representa un recurso de ejecución tal como:
 - Dispositivos
 - Procesadores
 - Memoria
 - Sistema Operativos
 - Bases de Datos

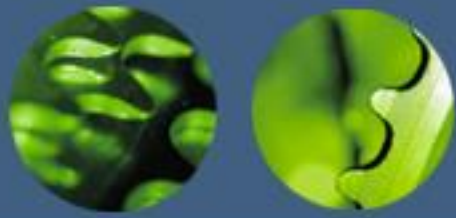


Diagrama de Despliegue o Distribución

- Un Nodo es un elemento físico, que existe en tiempo de ejecución y representa un recurso computacional que generalmente tiene alguna memoria y, a menudo, capacidad de procesamiento.
- Cada nodo puede contener instancias de componentes.
- Los nodos se interconectan mediante soportes bidireccionales que pueden a su vez estereotiparse.
- Esta vista permite determinar las consecuencias de la distribución y la asignación de recursos.

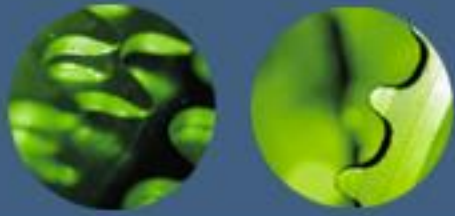
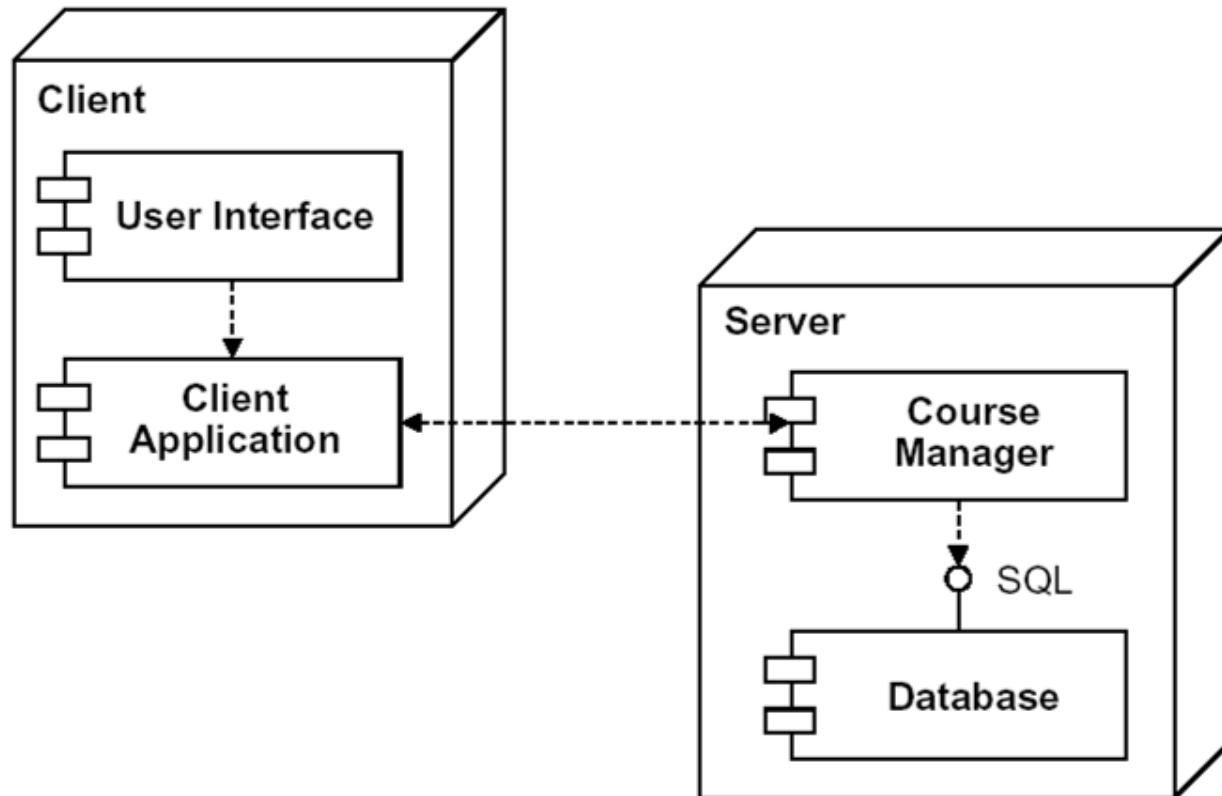


Diagrama de Despliegue o Distribución



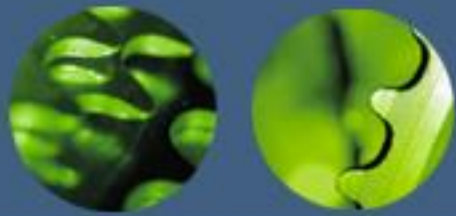
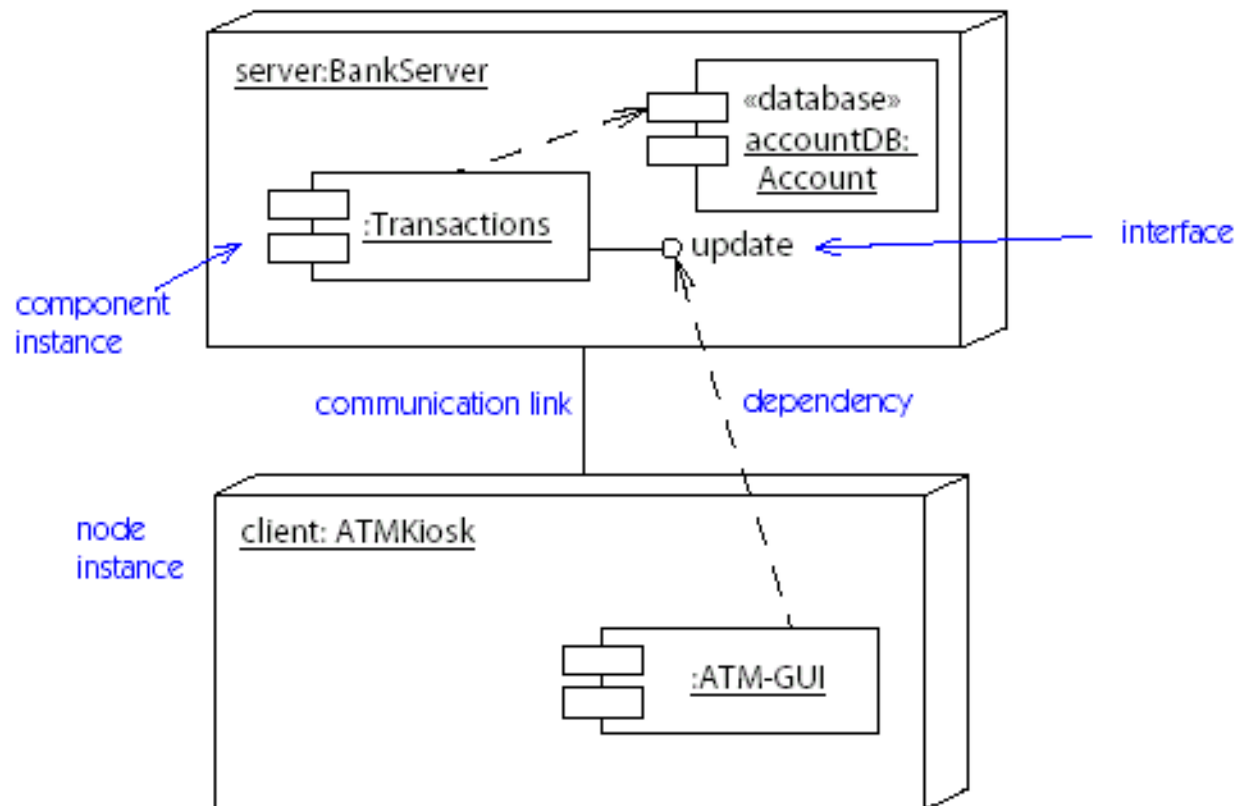


Diagrama de Despliegue o Distribución



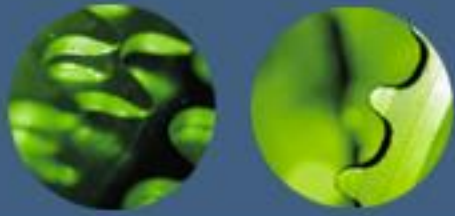
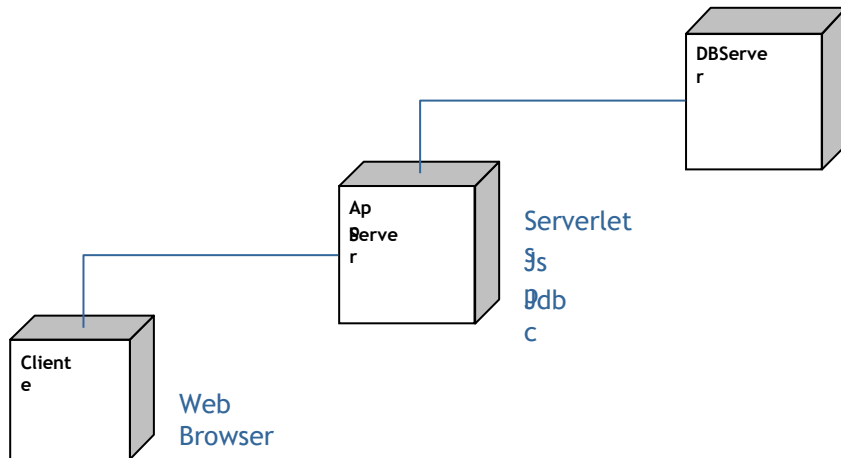
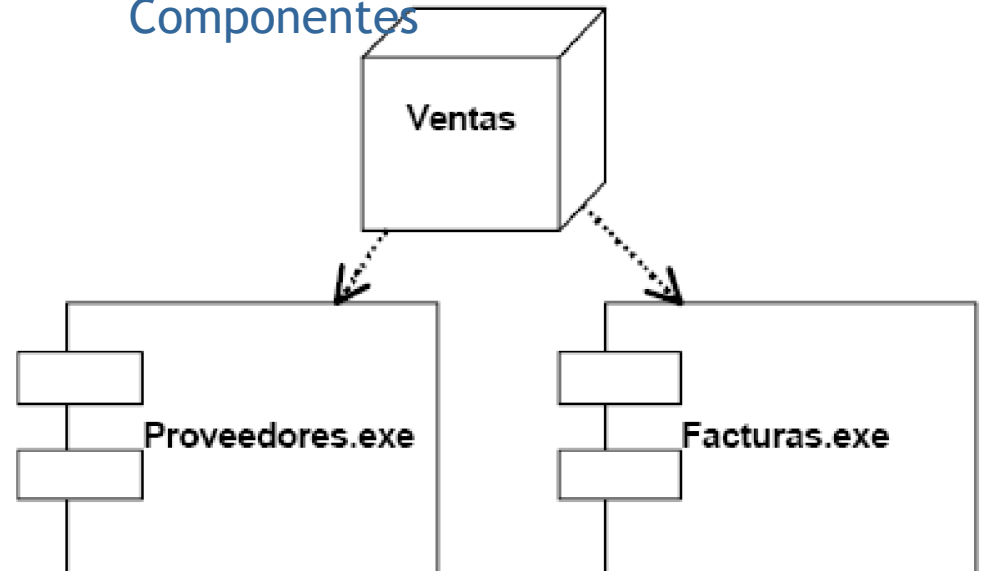


Diagrama de Despliegue o Distribución



Relación entre Nodos y Componentes



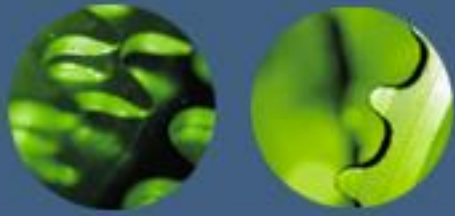
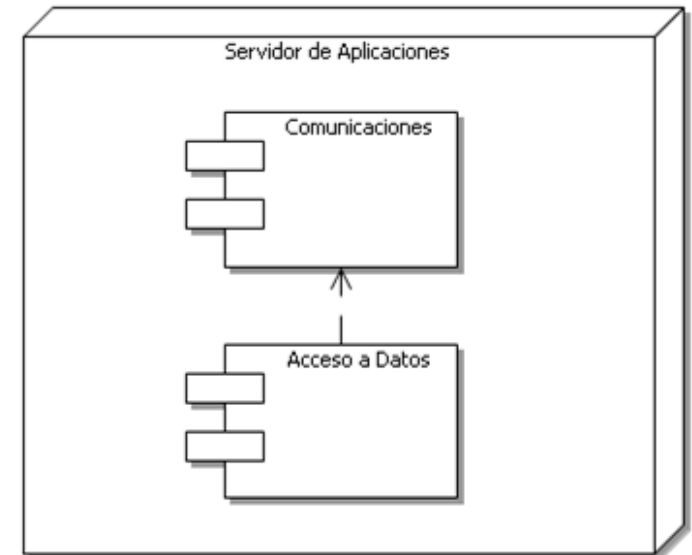
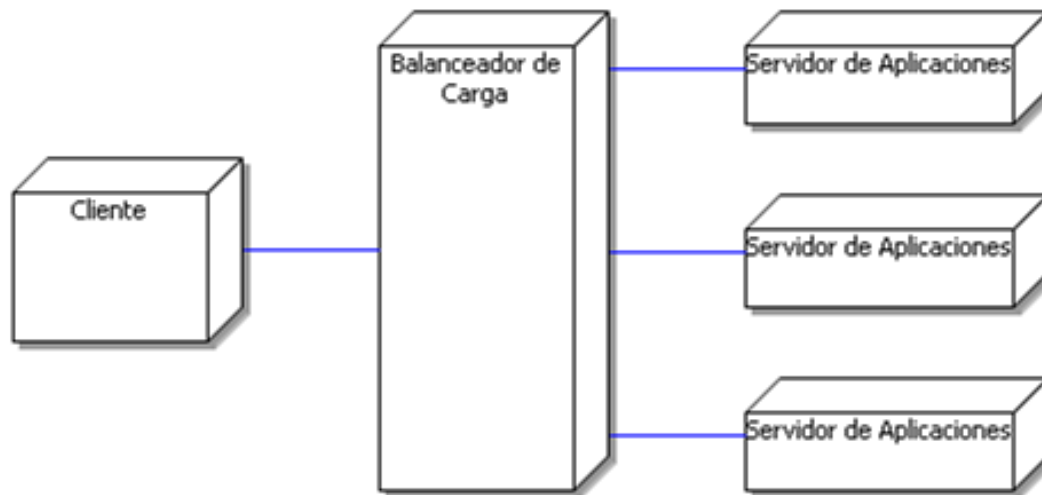


Diagrama de Despliegue o Distribución



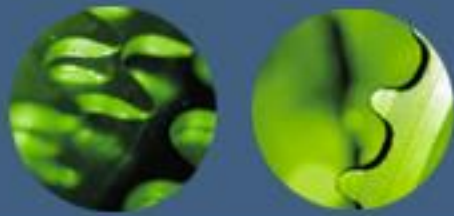
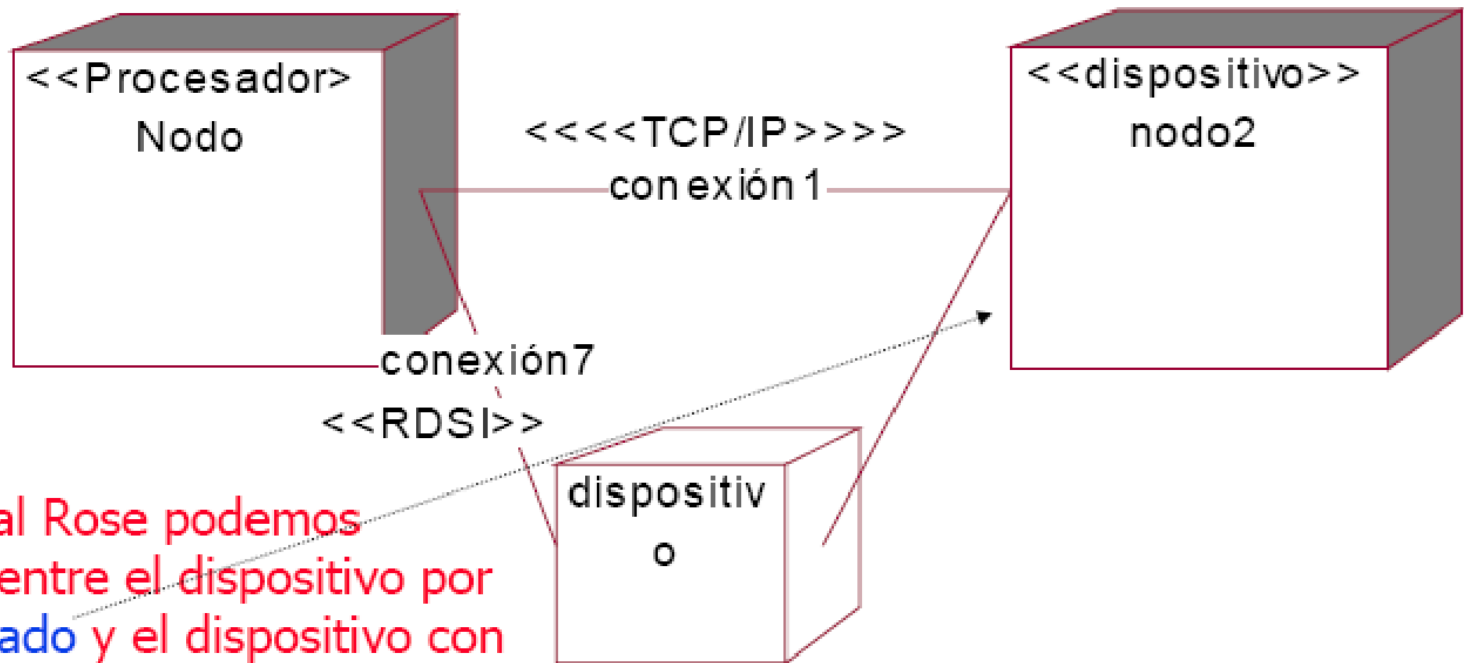


Diagrama de Despliegue o Distribución



En Rational Rose podemos distinguir entre el dispositivo por estereotipado y el dispositivo con su propio símbolo

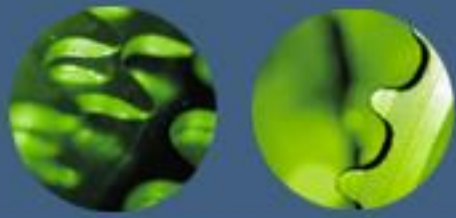
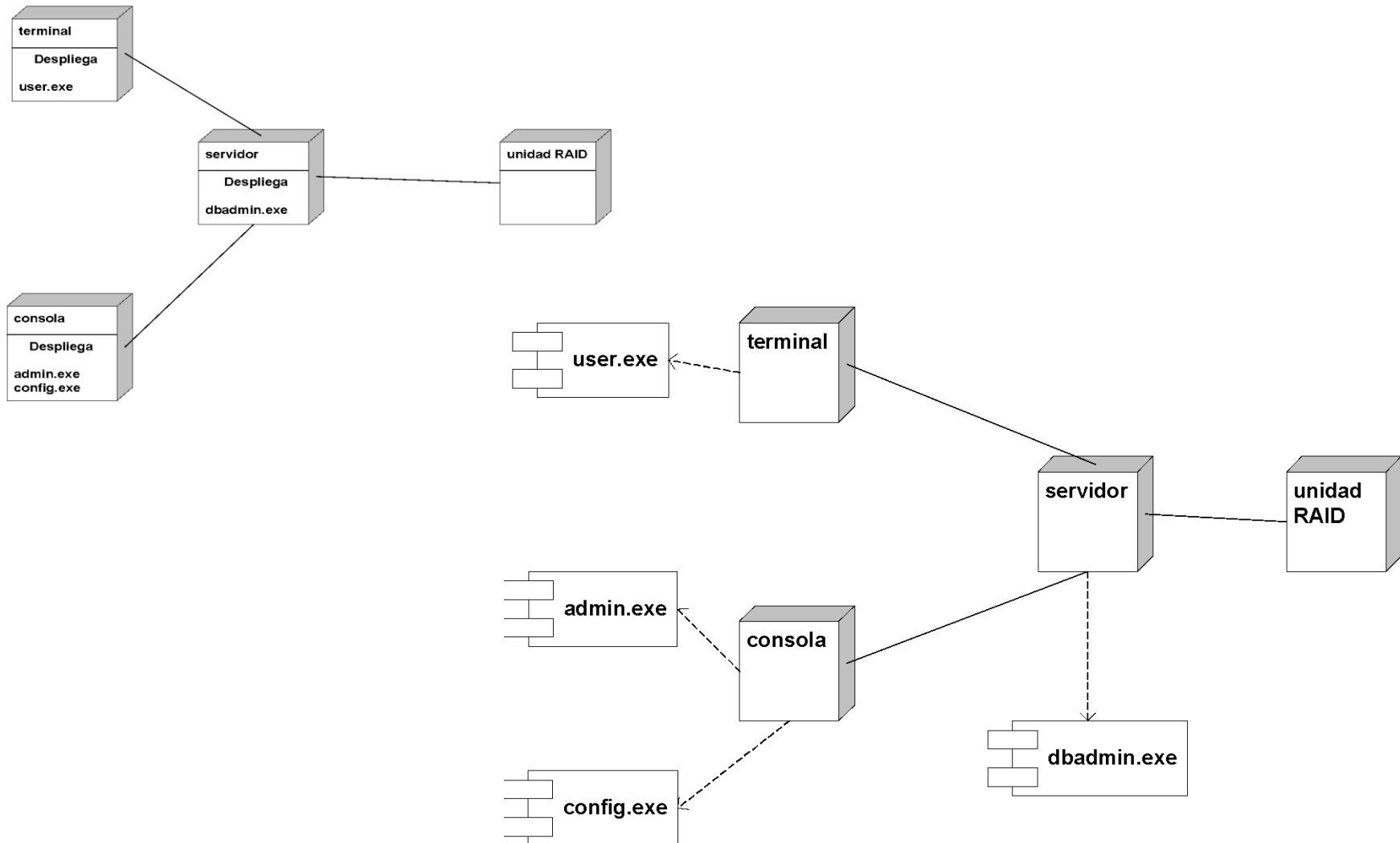
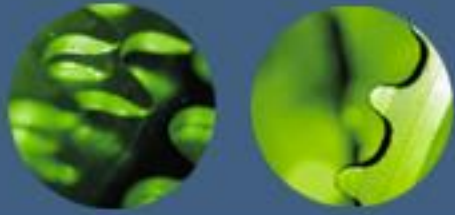
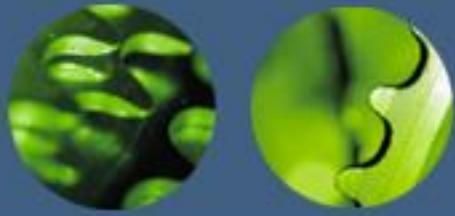


Diagrama de Despliegue o Distribución



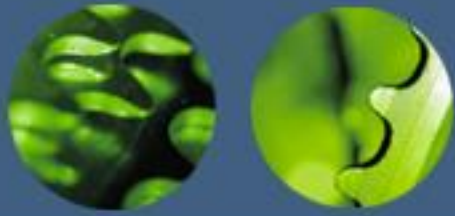


Conclusiones



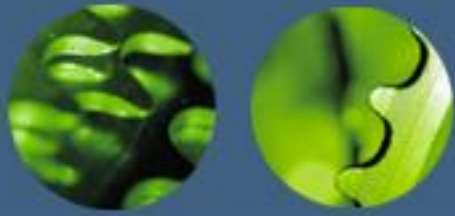
Conclusión

- El UML es un lenguaje reconocido mundialmente por la industria de construcción de software.
- El Modelamiento visual es una de las técnicas probadas que brinda mejores resultados.
- Todos los sistemas tienen una estructura estática y comportamiento dinámico.
- La estructura se describe con los diagramas de clases, componentes y despliegue.
- El comportamiento dinámico del sistema se describe con diagramas de estados, secuencias, colaboración y actividades.



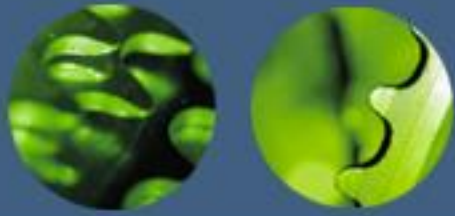
Conclusión

- UML define una notación que se expresa como diagramas sirven para representar modelos/subsistemas o partes de ellos.
- El 80% de la mayoría de los problemas pueden modelarse usando alrededor del 20% de UML- Grady Booch



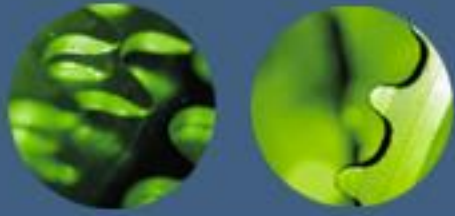
Herramientas CASE

- Rational Rose (www.rational.com)
- Rational XDE (www.rational.com)
- Borland Together (www.borland.com/together/)
- Embarcadero Describe (www.embarcadero.com/)



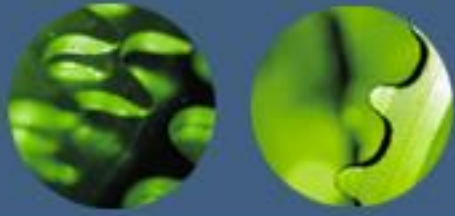
Herramientas CASE - Libre

- Argo UML (argouml.tigris.org)
- Poseidon (www.gentleware.com)
- Dome (www.htc.honeywell.com/dome)
- Comparativa:
 - <http://www.diatel.upm.es/malvarez/UML/Comparativa.html>



¿Preguntas?





¡Muchas Gracias!

